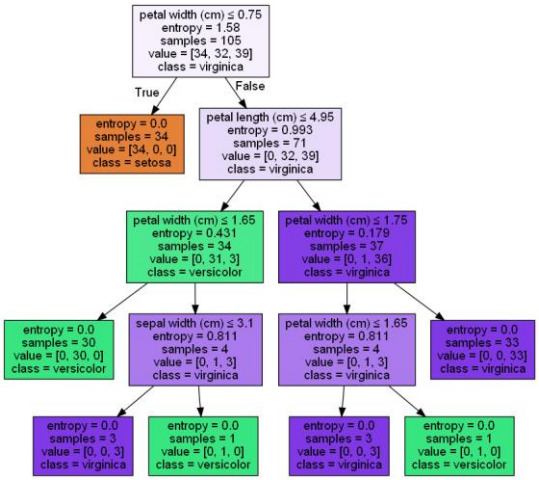
## 7.分类

### 决策树的图片展示——IO 流存储 dot

```
from sklearn import datasets
from sklearn import tree
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.tree import export_graphviz
from six import StringIO
import pydotplus

Iris = datasets.load_iris()  # 导入鸢尾花数据集
X = Iris.data  # 获得样本特征向量
Y = Iris.target  # 获得样本 label
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.3,
random_state=0)
print("target 的名称:%s" % Iris.target_names)

tree = tree.DecisionTreeClassifier(criterion='entropy')  # 使用的划分标准是信息熵
entropy
tree.fit(x_train, y_train)
print("决策树模型的训练集准确率:%.3f" % tree.score(x_train, y_train))
print("决策树模型的测试集准确率:%.3f" % tree.score(x_test, y_test))
y_hat = tree.predict(x_test)
print(classification_report(y_test, y_hat, target_names=Iris.target_names))

# 生成决策树可视化的 DOT 文件
dot_data = StringIO()
export_graphviz(tree, out_file=dot_data, filled=True,
feature_names=Iris.feature_names, class_names=Iris.target_names,
                fontname='Arial', special_characters=True)
dot_data = dot_data.getvalue()
dot_data = dot_data.replace('\n', '')  #  去除右上角黑块
# 使用 pydotplus 生成图像
graph = pydotplus.graph_from_dot_data(dot_data)
graph.write_png("output/iris_decision_tree.png")  # 将决策树保存为 PNG 文件
```

输出

```
target 的名称:['setosa' 'versicolor' 'virginica']
决策树模型的训练集准确率:1.000
决策树模型的测试集准确率:0.978
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        16
  versicolor       1.00      0.94      0.97        18
   virginica       0.92      1.00      0.96        11

    accuracy                           0.98        45
   macro avg       0.97      0.98      0.98        45
weighted avg       0.98      0.98      0.98        45
```




在训练集上准确率为 100%，表示模型能完美地对训练数据分类，但也暗示可能过拟合。

在测试集上准确率为 97.8%。表明模型在未见过的数据上表现很好，具有较高泛化能力。

第一个矩阵代表三个 target 的**查准率、查全率、F1 度量、支持的样本数量**。

第二个矩阵 Accuracy **精度**：整个模型在所有类别上的正确分类比例。

Macro average **宏平均**：对每个类别的评估指标取平均值，不考虑类别的样本数量，以 precision 为例，$\frac{1.00+1.00+0.92}{3}=0.97333$。

Weighted average **加权平均**：对每个类别的评估指标取加权平均值，考虑不同类别的样本数量。以 precision 为例，$1.00\times\frac{16}{45}+1.00\times\frac{18}{45}+0.92\times\frac{11}{45}=0.98044$。

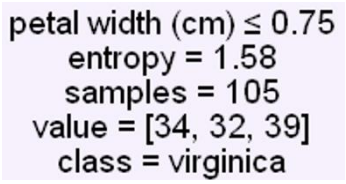
**决策树图形的参数讲解：**

**第一行**：划分依据。**第二行**：熵。

**第三行**：本次划分前有 105 个样本。

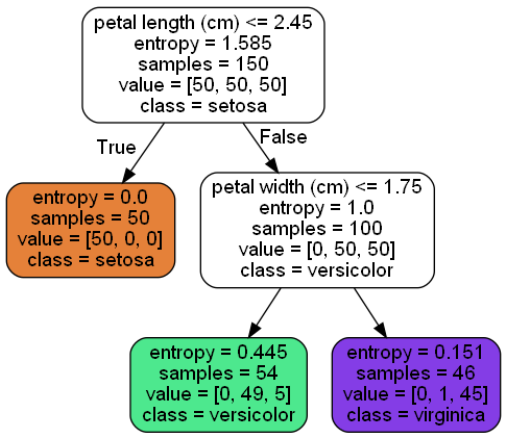
**第四行**：样本标签有三个，数量依次为 34,32,39。

**第五行**：代表类别(叶子结点的 class 才代表最终标签)。




**决策树的图片展示——本地存储 dot**

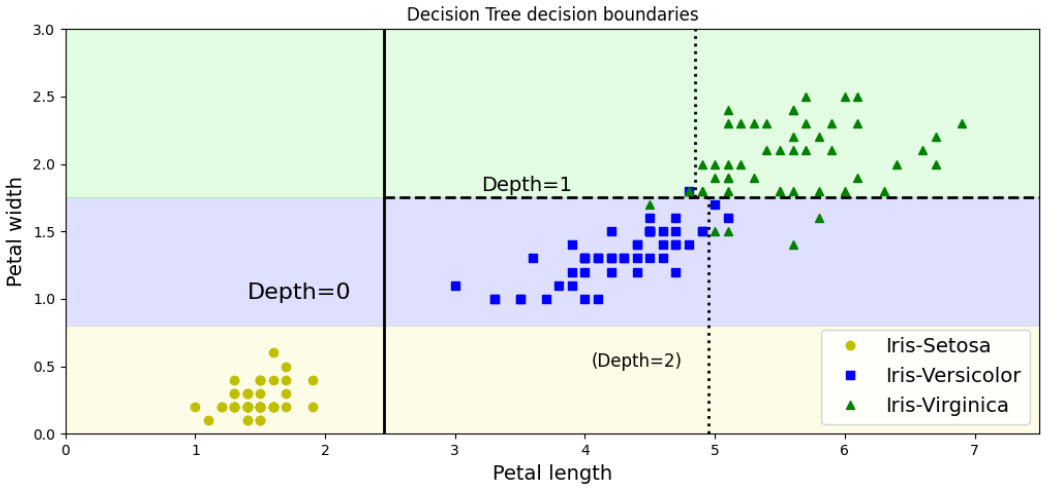
```
import pydotplus
import os
from sklearn.datasets import load_iris
from sklearn.datasets import make_moons
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_graphviz
from matplotlib.colors import ListedColormap
import numpy as np
import matplotlib.pyplot as plt


Iris = load_iris()
X = Iris.data[:, 2:]  # petal length and width
y = Iris.target

tree_clf = DecisionTreeClassifier(max_depth=2, criterion='entropy')
tree_clf.fit(X, y)

os.makedirs("output/决策树/", exist_ok=True)  # 创建目录，如果不存在则创建
export_graphviz(
    tree_clf,
    out_file="output/决策树/Iris_tree.dot",
    feature_names=Iris.feature_names[2:],
    class_names=Iris.target_names,
    rounded=True,
    filled=True
)
graph = pydotplus.graph_from_dot_file("output/决策树/Iris_tree.dot")
dot_data = graph.to_string()
dot_data = dot_data.replace('"\\r\\n";', '').replace('\n', '')  # 去除黑块
graph = pydotplus.graph_from_dot_data(dot_data)
graph.write_png("output/决策树/Iris_tree.png")
```




注意两种方式去除黑块的不同，因为存到本地后多了一个"\\r\\n";，所以不一样。

## 决策树的决策边界展示

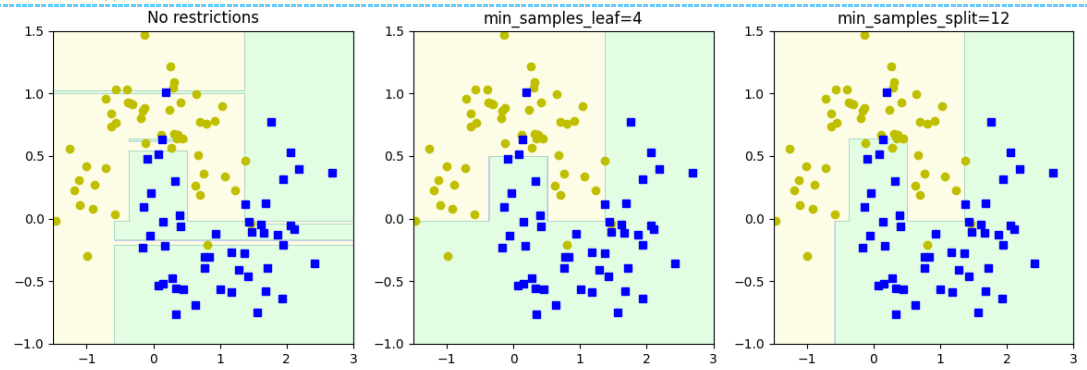
```
def plot_decision_boundary(clf, X, y, plot_boundary, draw_boundary=False,
label="other"):
    x1s = np.linspace(plot_boundary[0], plot_boundary[1], 1000)
    x2s = np.linspace(plot_boundary[2], plot_boundary[3], 1000)
    x1, x2 = np.meshgrid(x1s, x2s)
    X_new = np.c_[x1.ravel(), x2.ravel()]
    y_pred = clf.predict(X_new).reshape(x1.shape)
    custom_cmap = ListedColormap(['#fafab0', '#9898ff', '#a0faa0'])
    plt.contourf(x1, x2, y_pred, alpha=0.3, cmap=custom_cmap)

    plt.plot(X[:, 0][y == 0], X[:, 1][y == 0], "yo", label="Iris-Setosa")
    plt.plot(X[:, 0][y == 1], X[:, 1][y == 1], "bs", label="Iris-Versicolor")
    plt.plot(X[:, 0][y == 2], X[:, 1][y == 2], "g^", label="Iris-Virginica")

    if label == "Iris":
        plt.xlabel("Petal length", fontsize=14)
        plt.ylabel("Petal width", fontsize=14)
        plt.title('Decision Tree decision boundaries')
        plt.legend(loc="lower right", fontsize=14)

    if draw_boundary:
        plt.plot([2.45, 2.45], [0, 3], "k-", linewidth=2)  # 数据由决策树的树图像给出
        plt.plot([2.45, 7.5], [1.75, 1.75], "k--", linewidth=2)
        plt.plot([4.95, 4.95], [0, 1.75], "k:", linewidth=2)
        plt.plot([4.85, 4.85], [1.75, 3], "k:", linewidth=2)
        plt.text(1.40, 1.0, "Depth=0", fontsize=16)
        plt.text(3.2, 1.80, "Depth=1", fontsize=14)
        plt.text(4.05, 0.5, "(Depth=2)", fontsize=12)  # 加括号代表这其实是下一层决策树
才能给出具体的分割数字

plt.figure()
plot_decision_boundary(tree_clf, X, y, draw_boundary=True, plot_boundary=[0, 7.5,
0, 3], label="Iris")
plt.show()
```




## 决策树的“正则化”——限制决策树形状，避免过拟合

```
X, y = make_moons(n_samples=100, noise=0.25, random_state=42)
tree_clf1 = DecisionTreeClassifier(random_state=42)
tree_clf2 = DecisionTreeClassifier(min_samples_leaf=4, random_state=42)
tree_clf3 = DecisionTreeClassifier(min_samples_split=12, random_state=42)
tree_clf1.fit(X, y)
tree_clf2.fit(X, y)
tree_clf3.fit(X, y)
plt.figure(figsize=(12, 4))
plt.subplot(131)
```

```
plot_decision_boundary(tree_clf1, X, y, plot_boundary=[-1.5, 3, -1, 1.5])
plt.title('No restrictions')
plt.subplot(132)
plot_decision_boundary(tree_clf2, X, y, plot_boundary=[-1.5, 3, -1, 1.5])
plt.title('min_samples_leaf=4')
plt.subplot(133)
plot_decision_boundary(tree_clf3, X, y, plot_boundary=[-1.5, 3, -1, 1.5])
plt.title('min_samples_split=12')
plt.show()
```




**常见参数：**

max_depth(树最大的深度)

min_samples_split（节点在分割之前必须具有的最小样本数）

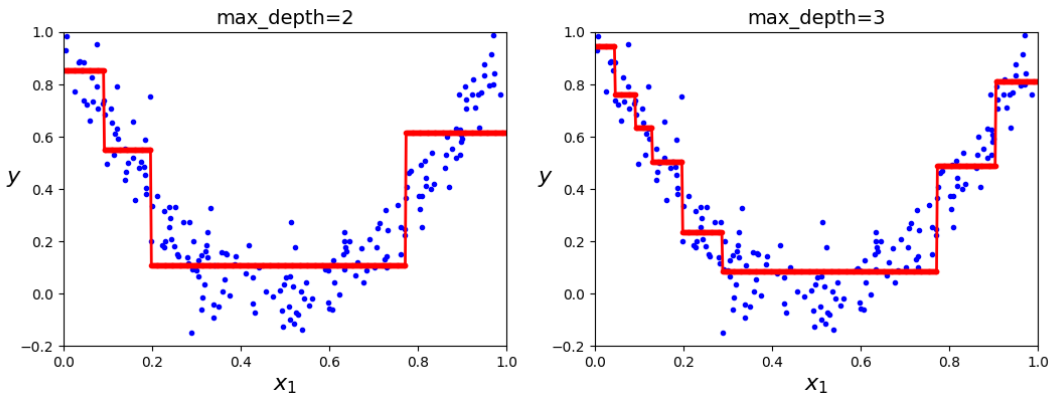
min_samples_leaf（叶子节点必须具有的最小样本数）

max_leaf_nodes（叶子节点的最大数量）

max_features（在每个节点处评估用于拆分的最大特征数）

**决策树解决回归问题**

```
np.random.seed(42)
m = 200
X = np.random.rand(m, 1)
y = 4 * (X - 0.5) ** 2
y = y + np.random.randn(m, 1) / 10
tree_reg1 = DecisionTreeRegressor(random_state=42, max_depth=2)
tree_reg2 = DecisionTreeRegressor(random_state=42, max_depth=3)
tree_reg1.fit(X, y)
tree_reg2.fit(X, y)

def plot_regression_predictions(tree_reg, X, y, axes=(0, 1, -0.2, 1),
ylabel="$y$"):
    # 因为默认参数在函数定义时只被计算一次，使用元组避免 axes 被函数内部改变。
    # 当默认参数是可变对象(比如列表或字典)时，如果在函数中修改了这个对象，那么这个修改会在
函数的后续调用中被保留。
    x1 = np.linspace(axes[0], axes[1], 500).reshape(-1, 1)
    y_pred = tree_reg.predict(x1)
    plt.axis(axes)
    plt.xlabel("$x_1$", fontsize=16)
    plt.ylabel(ylabel, fontsize=16, rotation=0)
    plt.plot(X, y, "b.")
    plt.plot(x1, y_pred, "r.-", linewidth=2, label=r"$\hat{y}$")

plt.figure(figsize=(11, 4))
plt.subplot(121)
plot_regression_predictions(tree_reg1, X, y)
plt.title("max_depth=2", fontsize=14)
plt.subplot(122)
plot_regression_predictions(tree_reg2, X, y)
plt.title("max_depth=3", fontsize=14)
plt.show()
```


max_depth=2
max_depth=3
y
$x_1$
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
0.0
0.2
0.4
0.6
0.8
1.0

## KNN

### 距离度量-欧氏距离的计算

```
import numpy as np

vector1 = np.mat([1, 2, 3])  # [[1 2 3]]
vector2 = np.mat([4, 5, 6])  # [[4 5 6]]
distance = np.sqrt((vector1-vector2) * (vector1-vector2).T)  # [[-3 -3 -3]] * [[-3]
[-3] [-3]] = [[5.19615242]]
print(distance)
```

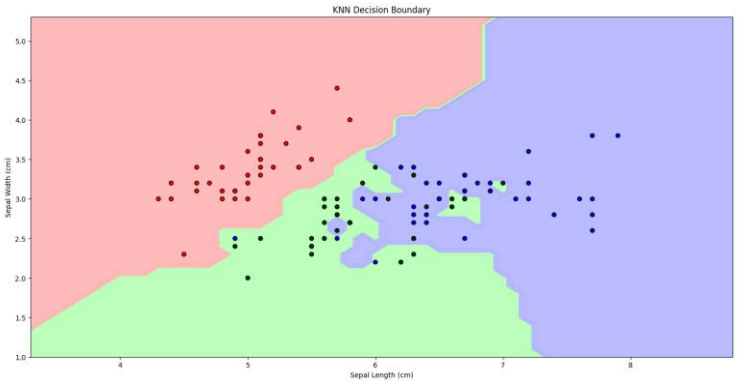
### KNN-模型拟合、样本预测、二维绘图

```
from sklearn import datasets
from sklearn import neighbors
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap

Iris = datasets.load_iris()  # 导入鸢尾花数据集
X = Iris.data  # 获得样本特征向量
Y = Iris.target  # 获得样本 label
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.3,
random_state=0)
print("target 的名称:%s" % Iris.target_names)

KNN = neighbors.KNeighborsClassifier(n_neighbors=3)
KNN.fit(x_train, y_train)
print("KNN 模型的训练集准确率:%.3f" % KNN.score(x_train, y_train))
print("KNN 模型的测试集准确率:%.3f" % KNN.score(x_test, y_test))
y_hat = KNN.predict(x_test)
print(classification_report(y_test, y_hat, target_names=Iris.target_names))

new_sample = np.array([5.1, 3.5, 1.4, 0.2]).reshape(1, -1)  # 新样本的特征值(其实是
Iris 数据集的第一个数据)
predicted_class = KNN.predict(new_sample)  # 运行后得知 predicted_class 的值为[0]，因此
下一行代码里的 predicted_class[0]就是 0。
predicted_class_name = Iris.target_names[predicted_class[0]]
print("样本[5.1, 3.5, 1.4, 0.2]预测的类别是:", predicted_class_name)

# Create color maps
cmap_light = ListedColormap(['#FFAAAA', '#AAFFAA', '#AAAAFF'])  # 给不同区域赋以颜色
cmap_bold = ListedColormap(['#FF0000', '#003300', '#0000FF'])  # 给不同属性的点赋以颜
色
# 定义绘图边界
feature1_min, feature1_max = x_train[:, 0].min() - 1, x_train[:, 0].max() + 1  # 第
一个属性 sepal length 花萼长度
feature2_min, feature2_max = x_train[:, 1].min() - 1, x_train[:, 1].max() + 1  # 第
二个属性 sepal width 花萼宽度
xx, yy = np.meshgrid(np.arange(feature1_min, feature1_max, 0.1),
np.arange(feature2_min, feature2_max, 0.1))  # 生成网格点
KNN = neighbors.KNeighborsClassifier(n_neighbors=3)
KNN.fit(x_train[:, :2], y_train)  # 以二维重新预测，属性变为原先训练集的一二个属性，
target 不变
print("二维 KNN 模型的训练集准确率:%.3f" % KNN.score(x_train[:, :2], y_train))
print("二维 KNN 模型的测试集准确率:%.3f" % KNN.score(x_test[:, :2], y_test))
# 将预测的结果在平面坐标中画出其类别区域
Z = KNN.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
# 绘制决策边界和训练样本
plt.contourf(xx, yy, Z, cmap=cmap_light, alpha=0.8)
plt.scatter(x_train[:, 0], x_train[:, 1], c=y_train, marker='o', edgecolors='k',
```

```
cmap=cmap_bold)
plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.xlabel('Sepal Length (cm)')
plt.ylabel('Sepal Width (cm)')
plt.title('KNN Decision Boundary')
plt.show()
```

输出

```
target 的名称:['setosa' 'versicolor' 'virginica']
KNN 模型的训练集准确率:0.962
KNN 模型的测试集准确率:0.978
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        16
  versicolor       1.00      0.94      0.97        18
   virginica       0.92      1.00      0.96        11

    accuracy                           0.98        45
   macro avg       0.97      0.98      0.98        45
weighted avg       0.98      0.98      0.98        45

样本[5.1, 3.5, 1.4, 0.2]预测的类别是: setosa
二维 KNN 模型的训练集准确率:0.857
二维 KNN 模型的测试集准确率:0.733
```




试验可知 n_neighbor 取 5 的时候比 3 在 train 上好一些。

绘制 KNN 边界的时候有一个问题，边界应当是连续的，但数据集是非常离散的。因此需要使用一个间隔(这里是 0.1，也可以取得更小，但运行时间会显著上升)来近似连续。但是对于不存在于数据集的点，如何我们选定的二维空间对应到四维空间里呢？或者说，对于给定的某个点(x,y)，已知的只有花萼长宽，没有花瓣长宽，缺失的数据没办法填补。并且，花瓣长宽不完全取决于花萼长宽，因此可能相同的花萼长宽对应不同的花瓣长宽时，会使得花的 target 发生变化。所以在二维图像里没办法给出全部信息，使得 KNN 只能重新拟合二维，因为剩下两维没办法确定。

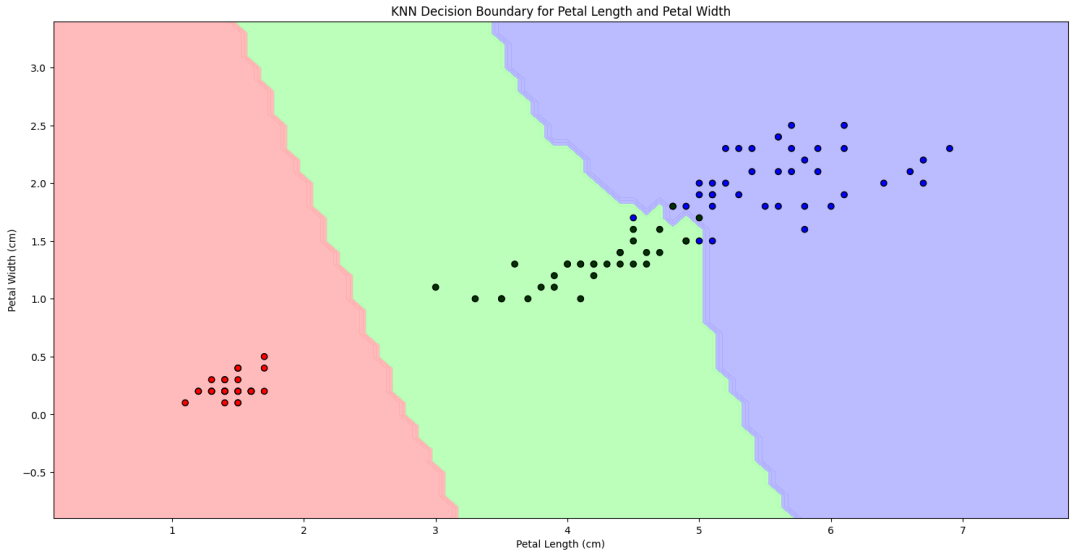
[补充：如何可视化 4 个特征？见 Visualize-ML_Book3_Ch21.2 成对特征散点图]

可以看到使用花萼属性的拟合其实不好，但使用花瓣拟合就很不错（可以预先查看散点图，就能发现花瓣很明显有更好的分类效果）：

```
feature3_min, feature3_max = x_train[:, 2].min() - 1, x_train[:, 2].max() + 1  # 第三个属性 petal length 花瓣长度
feature4_min, feature4_max = x_train[:, 3].min() - 1, x_train[:, 3].max() + 1  # 第四个属性 petal width 花瓣宽度
xx, yy = np.meshgrid(np.arange(feature3_min, feature3_max, 0.1),
np.arange(feature4_min, feature4_max, 0.1))
KNN = neighbors.KNeighborsClassifier(n_neighbors=3)
KNN.fit(x_train[:, 2:4], y_train)
print("后两个属性的 KNN 模型的训练集准确率:%.3f" % KNN.score(x_train[:, 2:4], y_train))
print("后两个属性的 KNN 模型的测试集准确率:%.3f" % KNN.score(x_test[:, 2:4], y_test))
Z = KNN.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
plt.contourf(xx, yy, Z, cmap=cmap_light, alpha=0.8)
plt.scatter(x_train[:, 2], x_train[:, 3], c=y_train, marker='o', edgecolors='k',
cmap=cmap_bold)
plt.xlim(xx.min(), xx.max())
plt.ylim(yy.min(), yy.max())
plt.xlabel('Petal Length (cm)')
plt.ylabel('Petal Width (cm)')
plt.title('KNN Decision Boundary for Petal Length and Petal Width')
plt.show()
```

输出

```
后两个属性的 KNN 模型的训练集准确率:0.962
后两个属性的 KNN 模型的测试集准确率:0.978
```




另外，KNN 要求所有的属性必须经过标准化处理，以免属性之间的数量级相差过大而导致偏差。KNN 可以自动忽略缺失值。对异常数据不敏感，具有较好的抗噪性。

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()  # 创建一个 Z-Score 标准化处理器
x_train_scaled = scaler.fit_transform(x_train)  # 对训练数据进行标准化处理
x_test_scaled = scaler.transform(x_test)  # 对测试数据进行相同的标准化处理
```

其余与之前的基本一致。

## 朴素贝叶斯

```
from sklearn import datasets
from sklearn import naive_bayes
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split

Iris = datasets.load_iris()
X = Iris.data
Y = Iris.target
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.3,
random_state=0)

bayes = naive_bayes.GaussianNB()
bayes.fit(x_train, y_train)
print("贝叶斯模型训练集的准确率:%.3f" % bayes.score(x_train, y_train))
print("贝叶斯模型测试集的准确率:%.3f" % bayes.score(x_test, y_test))
y_hat = bayes.predict(x_test)
print(classification_report(y_test, y_hat, target_names=Iris.target_names))
```

输出

```
贝叶斯模型训练集的准确率:0.943
贝叶斯模型测试集的准确率:1.000
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        16
  versicolor       1.00      1.00      1.00        18
   virginica       1.00      1.00      1.00        11

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

## SVM

```
from sklearn import datasets
from sklearn import neighbors
```

```
from sklearn import svm
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split

Iris = datasets.load_iris()
X = Iris.data
Y = Iris.target
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.3,
random_state=0)

SVM = svm.SVC()  # Support Vector Classification 是用于分类问题的 SVM 实现
SVM.fit(x_train, y_train)
print("SVM 模型训练集的准确率:%.3f" % SVM.score(x_train, y_train))
print("SVM 模型测试集的准确率:%.3f" % SVM.score(x_test, y_test))
y_hat = SVM.predict(x_test)
print(classification_report(y_test, y_hat, target_names=Iris.target_names))
```

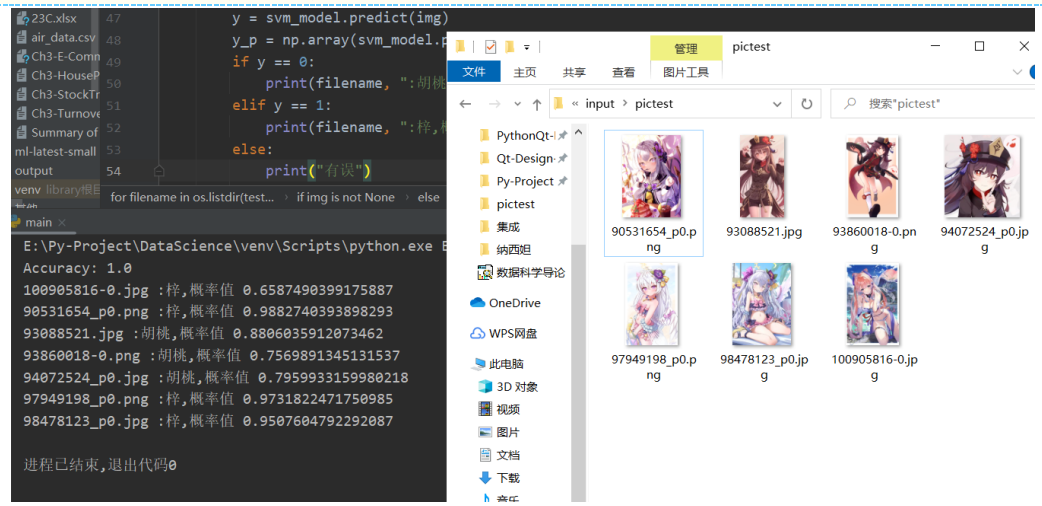
输出

```
SVM 模型训练集的准确率:0.971
SVM 模型测试集的准确率:0.978
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        16
  versicolor       1.00      0.94      0.97        18
   virginica       0.92      1.00      0.96        11

    accuracy                           0.98        45
   macro avg       0.97      0.98      0.98        45
weighted avg       0.98      0.98      0.98        45
```

## SVM 实现图片分类

```
import cv2
import os
from sklearn import svm
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
import numpy as np

def load_images_from_folder(folder):
    images = []
    for filename in os.listdir(folder):
        img_path = os.path.join(folder, filename)
        img = cv2.imread(img_path)
        if img is not None:
            img = cv2.resize(img, (700, 1000))
            img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
            images.append(img)
    return images

hutao_folder = "input/hutao"
azusa_folder = "input/azusa"
test_folder = "input/pictest"

hutao_images = np.array(load_images_from_folder(hutao_folder))  # 大小(12, 1000,
700)
azusa_images = np.array(load_images_from_folder(azusa_folder))
hutao_images = hutao_images.reshape((len(hutao_images), -1))  # 大小(12, 700000)
azusa_images = azusa_images.reshape((len(azusa_images), -1))
hutao_labels = np.zeros(len(hutao_images))  # 标签 0:胡桃，大小(12,)
azusa_labels = np.ones(len(azusa_images))  # 标签 1:梓
X = np.concatenate((hutao_images, azusa_images), axis=0)  # 大小(24, 700000)
y = np.concatenate((hutao_labels, azusa_labels), axis=0)  # 大小(24,)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

```
svm_model = svm.SVC(kernel='rbf', random_state=42, probability=True)
svm_model.fit(X_train, y_train)
y_pred = svm_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

for filename in os.listdir(test_folder):
    img_path = os.path.join(test_folder, filename)
    img = cv2.imread(img_path)
    if img is not None:
        img = cv2.resize(img, (700, 1000))
        img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
        img = np.array(img).reshape(1, -1)
        y = svm_model.predict(img)
        y_p = np.array(svm_model.predict_proba(img))
        if y == 0:
            print(filename, ":胡桃,概率值", str(y_p[0][0]))
        elif y == 1:
            print(filename, ":梓,概率值", str(y_p[0][1]))
        else:
            print("有误")
```

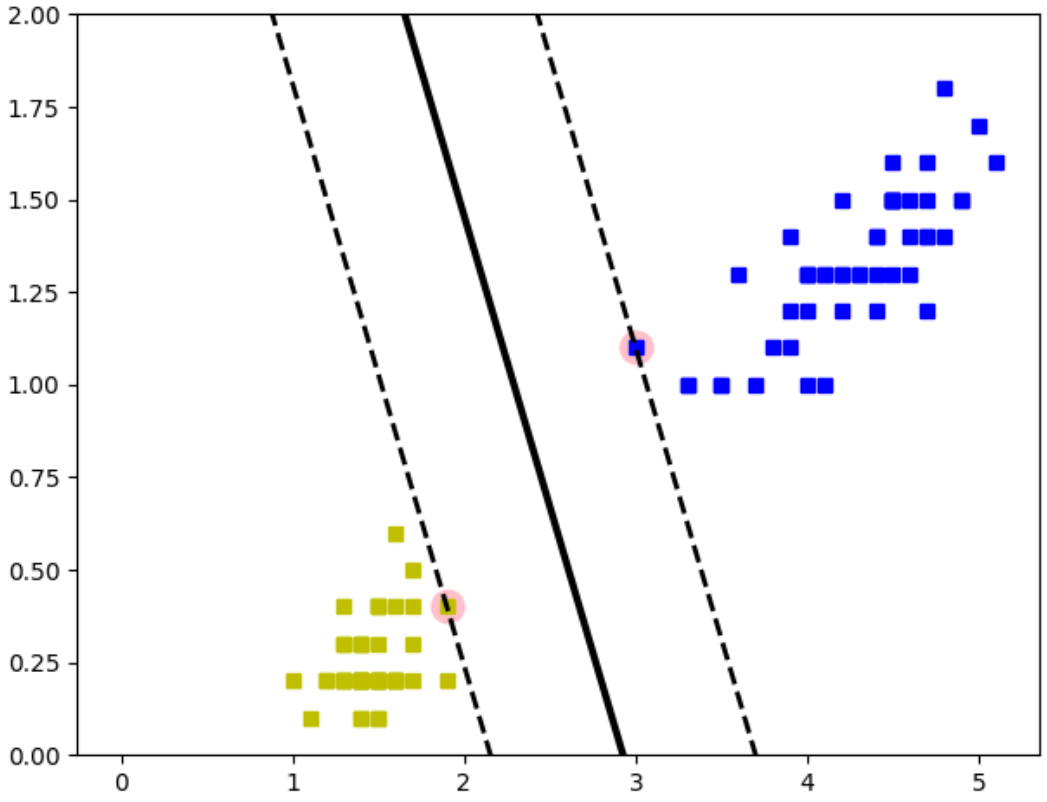


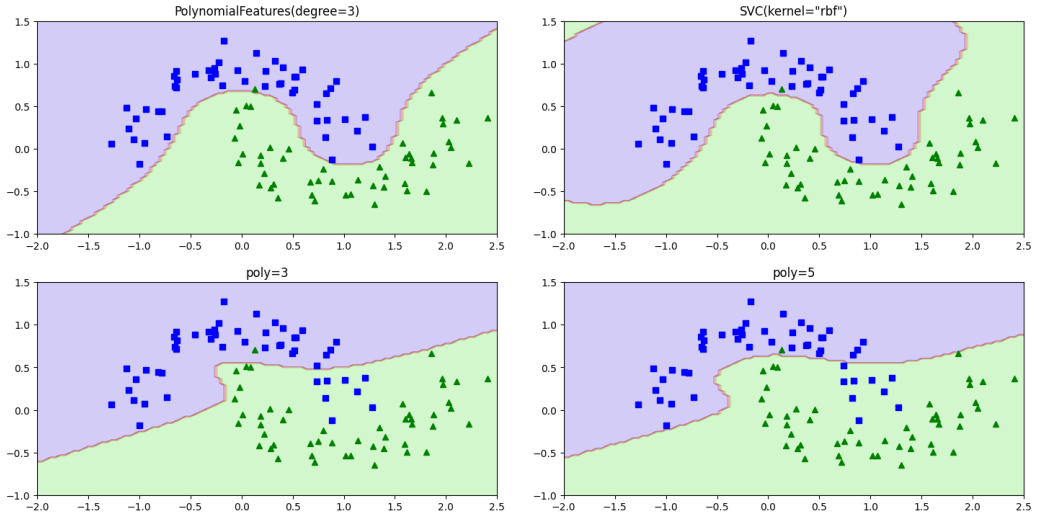

**线性/非线性/RBF 的 SVM**

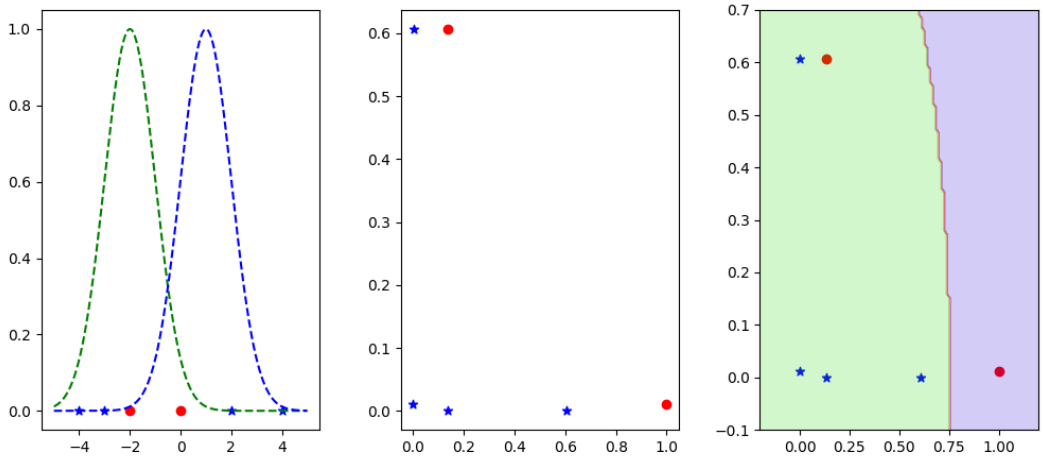
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.svm import SVC
from sklearn.datasets import load_iris, make_moons
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import PolynomialFeatures,StandardScaler

# 线性
Iris = load_iris()
X = Iris['data'][:, (2, 3)]
y = Iris['target']
setosa_and_versicolor = (y == 0) | (y == 1)
X = X[setosa_and_versicolor]
y = y[setosa_and_versicolor]
svm_clf = SVC(kernel='linear', C=100)
svm_clf.fit(X, y)

def draw_decision_boundary(clf, xlim=(0,5)):
    w = clf.coef_
    b = clf.intercept_
    x0 = np.linspace(xlim[0], xlim[1], 100)
    decision_boundary = -(x0 * w[0][0] + b) / w[0][1]  # 因为预测 x0w0+x1w1+b=0，因此
x1=-(x0w0+b)/w1
```

```
    up = decision_boundary + 1 / w[0][1]  # 因为间隔 d 是 2/||w||，因此 margin 就是
1/||w||
    down = decision_boundary - 1 / w[0][1]
    print(w[0], b)  # w 是[[w0,w1...]]这样的数据，因此先要执行 w[0]
    plt.plot(X[:, 0][y == 1], X[:, 1][y == 1], 'bs')
    plt.plot(X[:, 0][y == 0], X[:, 1][y == 0], 'ys')
    plt.ylim(0, 2)
    plt.plot(x0, decision_boundary, 'k-', linewidth=3)
    plt.plot(x0, up, 'k--', linewidth=2)
    plt.plot(x0, down, 'k--', linewidth=2)
    plt.scatter(clf.support_vectors_[:, 0], clf.support_vectors_[:, 1], s=180,
facecolors='#FFC0CB')

plt.figure()
draw_decision_boundary(svm_clf)
plt.show()

# 非线性
X, y = make_moons(n_samples=100, noise=0.15, random_state=42)
poly_svm_clf = Pipeline([
    ('poly', PolynomialFeatures(degree=3)),
    ('scaler', StandardScaler()),
    ('svm_clf', SVC(kernel='linear', C=100))
])
rbf_svm_clf = Pipeline([
    ('scaler', StandardScaler()),
    ('svm_clf', SVC(kernel='rbf', C=100))
])
poly3_svm_clf = Pipeline([
    ('scaler', StandardScaler()),
    ('svm_clf', SVC(kernel='poly', degree=3, C=100))
])
poly5_svm_clf = Pipeline([
    ('scaler', StandardScaler()),
    ('svm_clf', SVC(kernel='poly', degree=5, C=100))
])
poly_svm_clf.fit(X, y)
rbf_svm_clf.fit(X, y)
poly3_svm_clf.fit(X, y)
poly5_svm_clf.fit(X, y)

def draw_decision_boundary(clf, axes=(-2, 2.5, -1, 1.5)):
    x0s = np.linspace(axes[0], axes[1], 100)
    x1s = np.linspace(axes[2], axes[3], 100)
    x0, x1 = np.meshgrid(x0s, x1s)
    x = np.c_[x0.ravel(), x1.ravel()]
    y_pred = clf.predict(x).reshape(x0.shape)
    plt.contourf(x0, x1, y_pred, cmap='brg', alpha=0.2)
    plt.plot(X[:, 0][y == 0], X[:, 1][y == 0], 'bs')
    plt.plot(X[:, 0][y == 1], X[:, 1][y == 1], 'g^')

plt.subplot(221)
plt.title('PolynomialFeatures(degree=3)')
draw_decision_boundary(clf=poly_svm_clf)
plt.subplot(222)
plt.title('SVC(kernel="rbf")')
draw_decision_boundary(clf=rbf_svm_clf)
plt.subplot(223)
plt.title('poly=3')
draw_decision_boundary(clf=poly3_svm_clf)
plt.subplot(224)
plt.title('poly=5')
draw_decision_boundary(clf=poly5_svm_clf)
plt.show()
```

```python
# rbf
def RBF_cal(x, gamma=0.5, landmark=0):
    """计算高斯核函数，传入列向量，这里默认 γ=0.5"""
    return np.exp(-gamma * np.linalg.norm(x - landmark, axis=1) ** 2)  # 按行计算范数(欧氏距离)

X = np.array((-4, -3, -2, 0, 2, 4)).reshape(-1, 1)
y = np.array((0, 0, 1, 1, 0, 0))

x1 = np.linspace(-5, 5, 1000).reshape(-1, 1)
x2 = RBF_cal(x1, landmark=-2)
x3 = RBF_cal(x1, landmark=1)
X_rbf = np.c_[RBF_cal(X, landmark=-2), RBF_cal(X, landmark=1)]

X_SVC = np.array(((-4, 0), (-3, 0), (-2, 0), (0, 0), (2, 0), (4, 0)))
X_SVC_rbf = SVC(kernel='rbf', gamma=0.5)
X_SVC_rbf.fit(X_SVC, y)

plt.subplot(131)
plt.scatter(X_SVC[:, 0][y == 0], X_SVC[:, 1][y == 0], c='blue', marker="*")
plt.scatter(X_SVC[:, 0][y == 1], X_SVC[:, 1][y == 1], c='red', marker="o")
plt.plot(x1, x2, 'g--')
plt.plot(x1, x3, 'b--')
plt.subplot(132)
plt.scatter(X_rbf[:, 0][y == 0], X_rbf[:, 1][y == 0], c='blue', marker="*")
plt.scatter(X_rbf[:, 0][y == 1], X_rbf[:, 1][y == 1], c='red', marker="o")
plt.subplot(133)
x0s = np.linspace(-0.2, 1.2, 100)
x1s = np.linspace(-0.1, 0.7, 100)
x0, x1 = np.meshgrid(x0s, x1s)
x = np.c_[x0.ravel(), x1.ravel()]
y_pred = X_SVC_rbf.predict(x).reshape(x0.shape)
plt.scatter(X_rbf[:, 0][y == 0], X_rbf[:, 1][y == 0], c='blue', marker="*")
plt.scatter(X_rbf[:, 0][y == 1], X_rbf[:, 1][y == 1], c='red', marker="o")
plt.contourf(x0, x1, y_pred, cmap='brg', alpha=0.2)
plt.show()
```




PolynomialFeatures(degree=3)
SVC(kernel="rbf")
poly=3
poly=5



## 8.回归

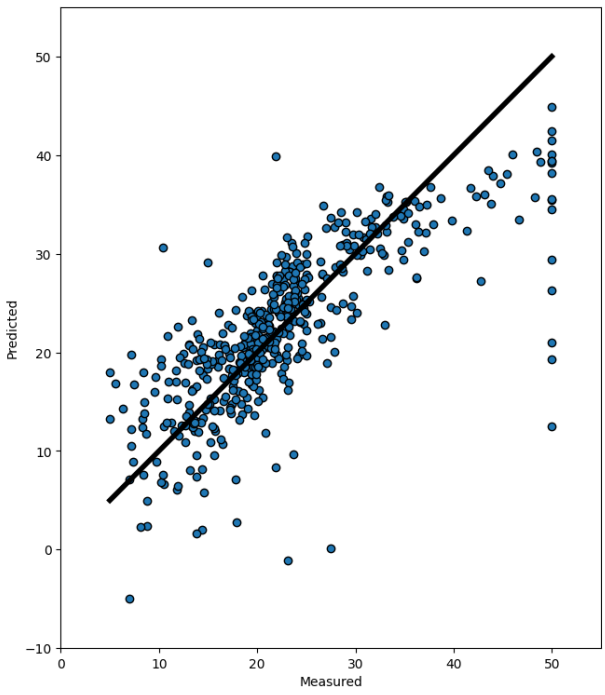
### 线性回归

```
from sklearn.model_selection import cross_val_predict
from sklearn import linear_model
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

data_url = "http://lib.stat.cmu.edu/datasets/boston"
raw_df = pd.read_csv(data_url, sep=r"\s+", skiprows=22, header=None)
data = np.hstack([raw_df.values[::2, :], raw_df.values[1::2, :2]])  # 属性
target = raw_df.values[1::2, 2]  # 预测值
print(data.shape, target.shape)  # 规格(506, 13) (506,)

lr = linear_model.LinearRegression()  # 线性回归
# 使用交叉验证。将数据分成 10 折(由 cv=10 指定)，每次使用 9 折数据训练模型，然后用模型预测第 10 折数据，并将预测结果保存下来。重复 10 次。
predicted = cross_val_predict(lr, data, target, cv=10)

# 查看 target 的最小和最大值以便绘图
print("Target - Min:", min(target))  # 5.0
print("Target - Max:", max(target))  # 50.0
# 查看 predicted 的最小和最大值
print("Predicted - Min:", min(predicted))  # -4.9536551238824345
print("Predicted - Max:", max(predicted))  # 44.912287884174994

fig, ax = plt.subplots()
ax.scatter(target, predicted, edgecolors=(0, 0, 0))
# 绘制线条（从(5,5)到(50,50)）。这条直线表示理想情况下，如果模型的预测完全准确，所有的点都应该落在这条直线上。
ax.plot([target.min(), target.max()], [target.min(), target.max()], 'k-', lw=4)  # k:黑色，-:实线，lw:line width。
ax.set_xlim(0, 55)  # 设置 x 轴范围
ax.set_ylim(-10, 55)  # 设置 y 轴范围
ax.set_xlabel('Measured')
ax.set_ylabel('Predicted')
plt.show()  # 不要问我为什么 target 有那么多 50 的数据点，我也很奇怪啊
```

输出

```
(506, 13) (506,)
Target - Min: 5.0
Target - Max: 50.0
Predicted - Min: -4.9536551238824345
Predicted - Max: 44.912287884174994
```

右图，横轴是准确值，纵轴是预测值。直线代表预测准确的点。预测值在直线之上/下代表预测过高/低。



# 线性回归的三种梯度下降方法

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression

np.random.seed(42)  # 设置固定种子便于实验


""" ---线性回归--- """
""" 生成数据集 & 绘图 """
X = 2 * np.random.rand(727, 1)  # 727 个[0, 1)之间均匀分布的随机数 * 2
y = 4 + 3 * X + np.random.randn(727, 1)  # 4 + 3x + 标准正态分布误差项
plt.plot(X, y, 'b.')
plt.axis([0, 2, 1, 14])
plt.show()

""" 直接使用公式计算 & 绘图 """
X_b = np.c_[np.ones((727, 1)), X]  # 拼接数据，形如[[1.         0.32950847] [1.         0.62878135]...[1.
0.53402584]]
theta_best = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)  # (X^T * X)^(-1) * X^T * y
print("公式计算的 θ: ", theta_best)  # [[4.02650858] [2.96474178]]很接近我们希望的系数 4 和 3
X_HeadAndTail = np.array([[0], [2]])  # [[0] [2]]
X_HeadAndTail_b = np.c_[np.ones((2, 1)), X_HeadAndTail]  # [[1. 0.] [1. 2.]]
y_predicted = X_HeadAndTail_b.dot(theta_best)  # [[4.02650858] [9.95599214]]
plt.plot(X_HeadAndTail, y_predicted, 'r--')
plt.plot(X, y, 'b.')
plt.axis([0, 2, 1, 14])
plt.show()

""" LinearRegression """
lin_reg = LinearRegression()
lin_reg.fit(X, y)
print("偏置参数: ", lin_reg.intercept_)  # 偏置参数 [4.02650858]
print("权重参数: ", lin_reg.coef_)  # 权重参数 [[2.96474178]]

""" 使用 MSE 的批量梯度下降 """
eta = 0.01  # 学习率 learning rate
n_iterations = 10000  # 迭代次数
n = len(X_b)  # 样本数量
theta = np.random.randn(2, 1)  # 形状为(2,1)的数组
for iteration in range(n_iterations):
    gradients = 2 / n * X_b.T.dot(X_b.dot(theta) - y)  # 梯度(使用均方误差损失函数 MSE)
    theta = theta - eta * gradients  # 参数更新
print("MSE 批量梯度下降迭代 10000 次的 θ: ", theta)

""" 三种梯度下降的对比"""
# 批量梯度下降 BGD(学习率对结果的影响)
theta_path_bgd = []  # bgd: Batch Gradient Descent 批量梯度下降
def plot_gradient_descent(theta, eta, store_bgd = None):
    n = len(X_b)
    plt.plot(X, y, 'b.')
    n_iterations = 200  # 迭代次数
    for iteration in range(n_iterations):
        gradients = 2 / n * X_b.T.dot(X_b.dot(theta) - y)  # 梯度(使用均方误差损失函数 MSE)
        theta = theta - eta * gradients  # 参数更新
        if store_bgd is not None:  # 因为 BGD 有三组对比实验，我们只需要存入一个 bgd 的值与后续 sgd,mgd 比较
            store_bgd.append(theta)  # 通过传入 store_bgd=theta_path_bgd 实现向 theta_path_bgd 里传值
        y_predicted = X_HeadAndTail_b.dot(theta)
        plt.plot(X_HeadAndTail, y_predicted, color=(0.1, 1, 0.5, min(8 * iteration / n_iterations, 0.2222 *
iteration / n_iterations + 0.7778)))  # color 的参数依次是红绿蓝和透明度 alpha, alpha 是分段函数
y=8*x(x<=0.1),y=0.2222x+0.7778(x>0.1)
        # print("x:", iteration / n_iterations, "\ty:", min(40*iteration/n_iterations,
0.204*iteration/n_iterations+0.796)) # 查看 color 中的函数对应的 x,y 值
        plt.text(X_HeadAndTail[-1], y_predicted[-1], f'{iteration}', color='red')  # 在当前迭代的最后一个点上添加文本标
签
    plt.xlabel('X')
    plt.ylabel('y')
    plt.axis([0, 2, 1, 14])
    plt.title('eta={}'.format(eta))
theta = np.random.randn(2, 1)
plt.figure(figsize=(10, 4))
plt.subplot(131)
plot_gradient_descent(theta, eta=0.01, store_bgd=theta_path_bgd)
plt.subplot(132)
plot_gradient_descent(theta, eta=0.1)
plt.subplot(133)
plot_gradient_descent(theta, eta=0.3)
plt.show()

# 随机梯度下降 SGD
theta_path_sgd = []
n = len(X_b)  # 数据个数
n_epochs = 50  # 迭代的轮数，即整个数据集被遍历的次数
t0 = 5
t1 = 50  # t0,t1 共同控制学习率的衰减
theta = np.random.randn(2, 1)
```

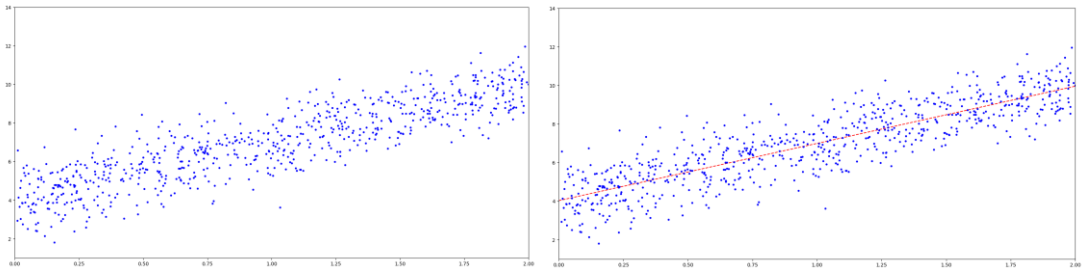
```
for epoch in range(n_epochs):
    for i in range(n):
        if epoch < 10 and i < n and i % 20 == 0:  # 绘制前十个 epoch 对应的拟合直线，每个 epoch 绘制间隔为 20 的直线(便于显示出明显的间隔)
            y_predicted = X_HeadAndTail_b.dot(theta)
            plt.plot(X_HeadAndTail, y_predicted, color=(0.1, 1, 0.5, i/n))
            if i == 0:  # 仅当每个 epoch 的第一个 i 的时候绘制出 epoch 的值
                plt.text(X_HeadAndTail[-1], y_predicted[-1], f'{epoch}', color='red')
        random_index = np.random.randint(n)  # 随机选择一个样本
        xi = X_b[random_index:random_index + 1]
        yi = y[random_index:random_index + 1]
        gradients = 2 * xi.T.dot(xi.dot(theta) - yi)
        eta = t0 / (t1+(n_epochs*n+i))  # 当迭代次数(n_epochs*n+i)增加，学习率 eta 呈降低趋势
        theta = theta - eta * gradients
        theta_path_sgd.append(theta)
plt.plot(X, y, 'b.')
plt.axis([0, 2, 1, 14])
plt.show()

# 小批量梯度下降 Mini-batch Gradient Descent
theta_path_mgd = []
n_epochs = 800  # 为什么这里迭代次数远高于上面的才能收敛？因为每次 epoch 只迭代了 n/minibatch 次，虽然每次迭代有 minibatch 个数据样本点，但 minibatch 个数据样本点真的比 1 个数据样本点强很多吗？
minibatch = 16
theta = np.random.randn(2, 1)
for epoch in range(n_epochs):
    shuffled_index = np.random.permutation(n)  # 样本洗牌
    X_b_shuffled = X_b[shuffled_index]
    y_shuffled = y[shuffled_index]
    for i in range(0, n, minibatch):  # 从 0 开始，以步长 minibatch 迭代，直到 n-1 结束。
        if epoch < 160 and epoch % 10 == 0 and i % (20*minibatch) == 0:  # 只绘制前面的 epoch。对每个 epoch 依然只绘制部分迭代过程
            y_predicted = X_HeadAndTail_b.dot(theta)
            plt.plot(X_HeadAndTail, y_predicted, color=(0.1, 1, 0.5, i / n))
            if i == 0:  # 仅当每个 epoch 的第一个 i 的时候绘制出 epoch 的值
                plt.text(X_HeadAndTail[-1], y_predicted[-1], f'{epoch}', color='red')
        xi = X_b_shuffled[i: i+minibatch]
        yi = y_shuffled[i: i+minibatch]
        gradients = 2/minibatch * xi.T.dot(xi.dot(theta) - yi)
        eta = t0 / (t1 + (n_epochs*n/minibatch + i/minibatch))
        theta = theta - eta * gradients
        theta_path_mgd.append(theta)
plt.plot(X, y, 'b.')
plt.axis([0, 2, 1, 14])
plt.show()

# BGD,SGD,MGD 三者对比
theta_path_bgd = np.array(theta_path_bgd)
theta_path_sgd = np.array(theta_path_sgd)
theta_path_mgd = np.array(theta_path_mgd)
print("BGD:", theta_path_bgd[-1], "\nSGD:",  theta_path_sgd[-1], "\nMGD:", theta_path_mgd[-1])
plt.plot(theta_path_bgd[:, 0], theta_path_bgd[:, 1], 'b-o', label='BGD')
plt.plot(theta_path_sgd[:, 0], theta_path_sgd[:, 1], 'r-s', label='SGD')
plt.plot(theta_path_mgd[:, 0], theta_path_mgd[:, 1], 'g-+', label='MGD')
plt.legend()
plt.show()  # 全局图像
plt.plot(theta_path_bgd[::10, 0], theta_path_bgd[::10, 1], 'b-o', label='BGD')  # ::n 表示以步长为 n 进行切片
plt.plot(theta_path_sgd[::100, 0], theta_path_sgd[::100, 1], 'r-s', label='SGD')
plt.plot(theta_path_mgd[::80, 0], theta_path_mgd[::80, 1], 'g-+', label='MGD')
plt.axis([3.2, 4.5, 2.3, 4.5])
plt.legend()
plt.show()  # 部分数据点的局部图像
# 理论上 BGD 会是笔直到达。SGD 是大范围的浮动，很容易出现偏离。MGD 与 BGD 隔的比较近，在 BGD 的周围浮动。
```

输出

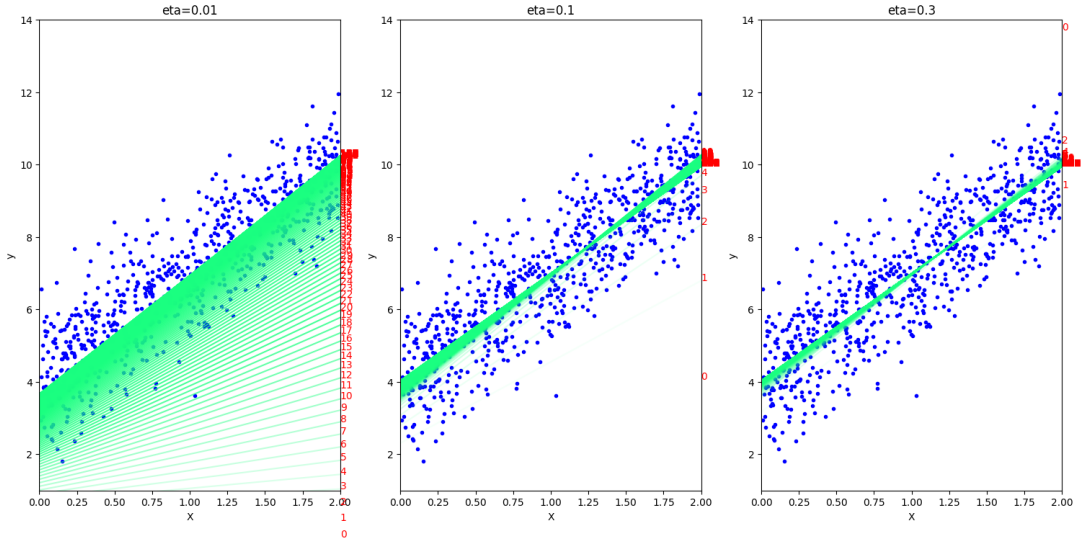
```
公式计算的 θ:  [[4.02650858]
 [2.96474178]]
偏置参数:  [4.02650858]
权重参数:  [[2.96474178]]
MSE 批量梯度下降迭代 10000 次的 θ:  [[4.02650858]
 [2.96474178]]
BGD: [[3.66635721]
 [3.2697089 ]]
SGD: [[3.93608917]
 [3.03300464]]
MGD: [[4.09271048]
 [2.90851744]]
```

图片输出如下：

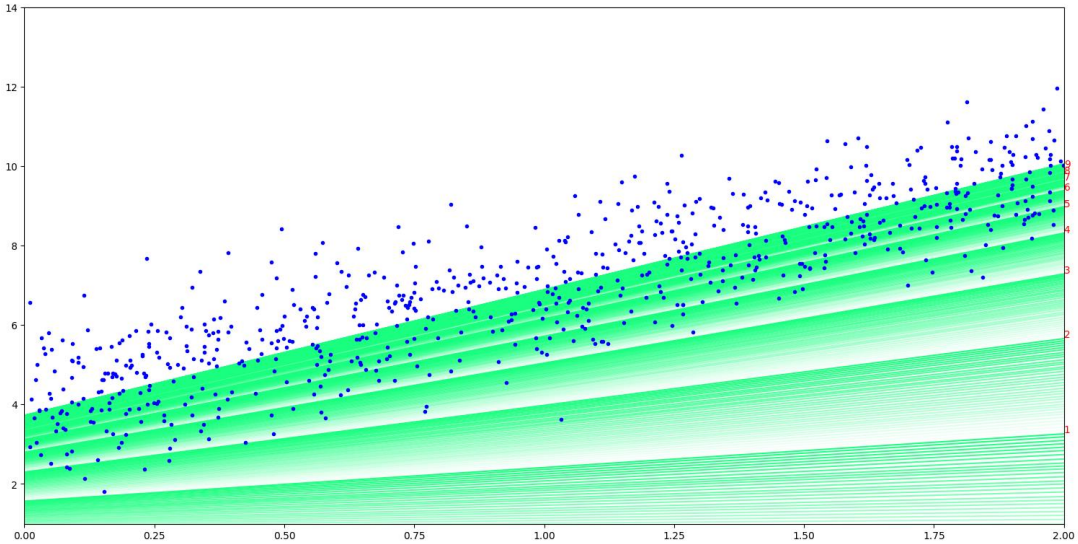
原始数据 & LR 拟合



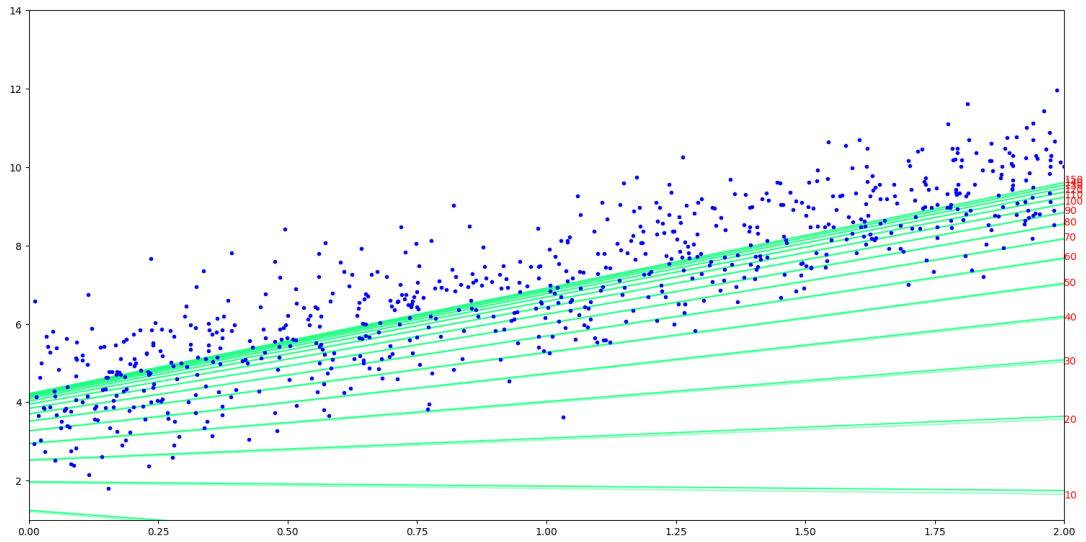
MSE-eta 不同导致的结果不同：



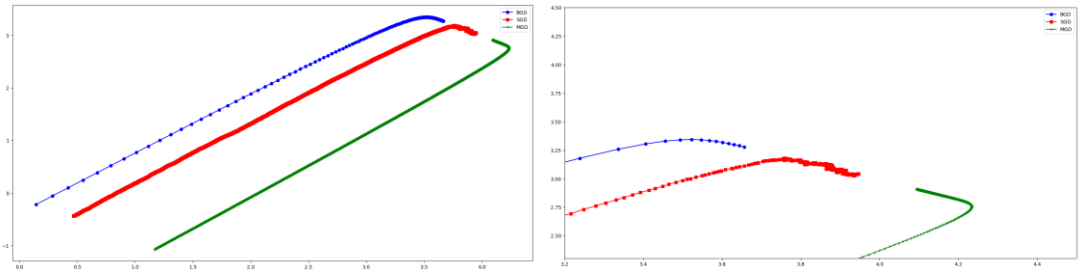

随机梯度下降：



批量梯度下降：



三者对比的完整图 & 部分图：



**多项式回归**

```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.linear_model import Lasso

np.random.seed(42)  # 设置固定种子便于实验


""" ---多项式回归--- """
""" 生成数据集 & 绘图 """
X = 6 * np.random.rand(727, 1) - 3  # 727 个[0, 1)之间均匀分布的随机数 * 6 - 3
y = 0.5*X**2 + X + np.random.randn(727, 1)  # 0.5x^2 + x + 标准正态分布误差项
plt.plot(X, y, 'b.')
plt.axis([-3.2, 3.2, -4, 11])
plt.show()

""" 将多项式化为 LinearRegression """
poly_features = PolynomialFeatures(degree=2, include_bias=False)
X_poly = poly_features.fit_transform(X)  # 将 X 变为[[X1,X1^2], [X2,X2^2], ... , [Xn,Xn^2]]，相当于将 X^2 变成了一个新的特征
print("X[0]:", X[0], "X_poly[0]", X_poly[0])  # X_poly[i] = (X[i])^2
lin_reg = LinearRegression()
lin_reg.fit(X_poly, y)
print("偏置参数: ", lin_reg.intercept_)  # 偏置参数 [0.01279778]。即常数项
print("权重参数: ", lin_reg.coef_)  # 权重参数 [[0.98806406 0.4931436 ]]。因为 X_poly 相当于 X 和 X^2，因此系数分别对应一次项和二次项
# 得到拟合曲线为 0.4931436x^2 + 0.98806406x + 0.01279778
X_curve = np.linspace(-3, 3, 100).reshape(100, 1)
X_curve_poly = poly_features.transform(X_curve)  # 从-3 到 3 的 100 个点经过变换(变为 x 与 x^2)后的值
y_curve = lin_reg.predict(X_curve_poly)
```

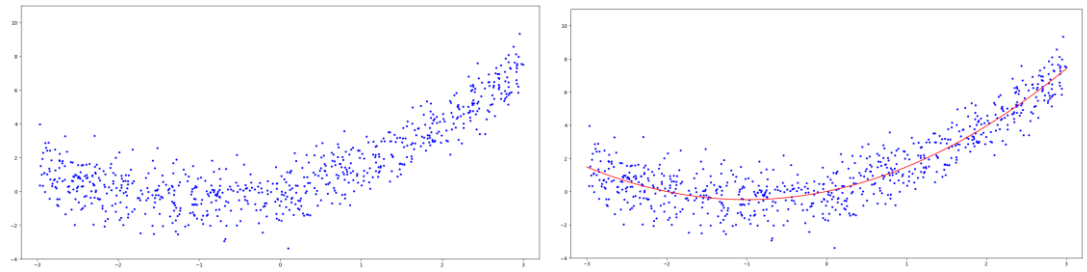
```
plt.plot(X, y, 'b.')
plt.plot(X_curve, y_curve, 'r-')
plt.axis([-3.2, 3.2, -4, 11])
plt.show()

""" degree 不同对结果的影响 """
for style, width, degree in (('g-', 3, 50), ('r-', 2, 2), ('y--', 2, 1)):
    poly_features = PolynomialFeatures(degree=degree, include_bias=False)
    std = StandardScaler()
    lin_reg = LinearRegression()
    # 流水线车间 Pipeline 包含三个流水线：poly_features,StandardScaler,LinearRegression
    polynomial_reg = Pipeline([('poly_features', poly_features),  # 多项式特征生成
                               ('StandardScaler', std),  # 标准化
                               ('LinearRegression', lin_reg)])  # 线性回归
    polynomial_reg.fit(X, y)
    y_curve_ComparativeExperiments = polynomial_reg.predict(X_curve)
    plt.plot(X_curve, y_curve_ComparativeExperiments, style, label='degree:'+str(degree),
linewidth=width)
plt.plot(X, y, 'b.')
plt.axis([-3.2, 3.2, -4, 11])
plt.legend()
plt.show()

""" 数据样本数量对结果的影响 """
def plot_learning_curves(model, X, y):
    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)  #
val:validation 验证集
    train_errors, val_errors = [], []
    for size in range(1, len(X_train)):  # 数据量从 1 到 len(X_train)
        model.fit(X_train[:size], y_train[:size])
        y_train_predicted = model.predict(X_train[:size])
        y_val_predict = model.predict(X_val)
        train_errors.append(mean_squared_error(y_train[:size], y_train_predicted[:size]))
        val_errors.append(mean_squared_error(y_val, y_val_predict))
    plt.plot(np.sqrt(train_errors), 'r--', linewidth=2, label='train_errors')
    plt.plot(np.sqrt(val_errors), 'b-', linewidth=3, label='val_errors')
    plt.xlabel("Training Size")
    plt.ylabel("RMSE")
    plt.legend()
polynomial_reg = Pipeline([('poly_features', PolynomialFeatures(degree=2, include_bias=False)),  # 多
项式特征生成
                           ('LinearRegression', LinearRegression())])  # 线性回归
plot_learning_curves(polynomial_reg, X, y)
plt.show()  # error 越小越好

""" 正则化(效果就是将拟合曲线拉平) """
def plot_model(model_choice, alphas, poly_degree):
    for alpha, style in zip(alphas, ('y-', 'm--', 'r-.')):
        ridge_model = model_choice(alpha=alpha)
        model = Pipeline([('poly_features', PolynomialFeatures(degree=poly_degree,
include_bias=False)),  # 多项式特征生成
                          ('StandardScaler', std),  # 标准化
                          ('LinearRegression', ridge_model)])  # 线性回归
        model.fit(X, y)
        y_curve_RidgeExperiments = model.predict(X_curve)
        plt.plot(X_curve, y_curve_RidgeExperiments, style, label='alpha={}'.format(alpha),
linewidth=3)
    plt.plot(X, y, 'b.')
    plt.legend()
plot_model(Ridge, alphas=(0, 10**-25, 1), poly_degree=200)  # 学习率在 10^-25 处就能比较明显地与 alpha=0
区分(alpha=10^-30 与 alpha=0 就差不多了)
plt.axis([-3.2, 3.2, -4, 11])
plt.show()

plot_model(Lasso, alphas=(0, 0.5, 1), poly_degree=2)
plt.axis([-3.2, 3.2, -4, 11])
plt.show()
```

文字输出：

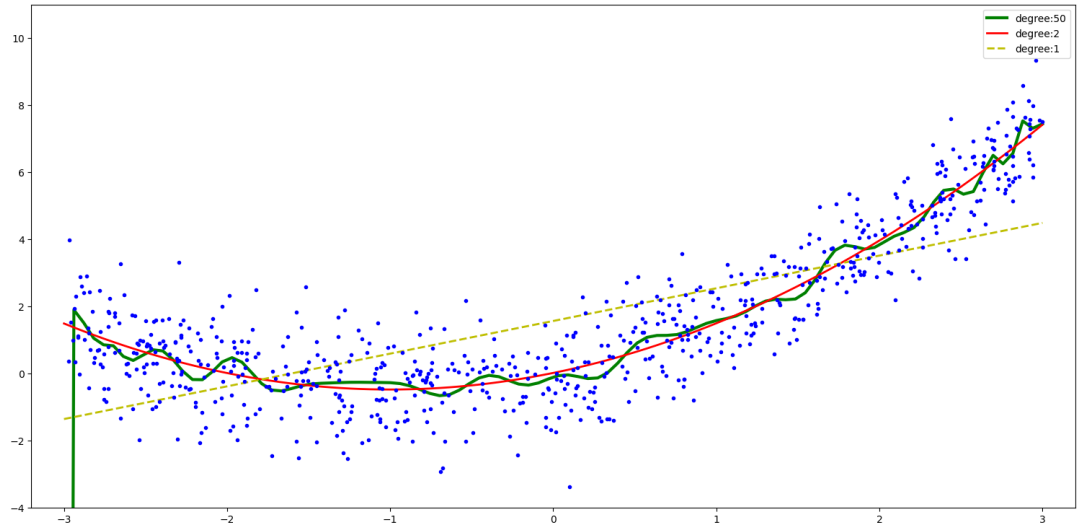
```
X[0]: [-0.75275929] X_poly[0] [-0.75275929  0.56664654]
偏置参数：  [0.01279778]
权重参数：  [[0.98806406 0.4931436 ]]
```

图片输出：

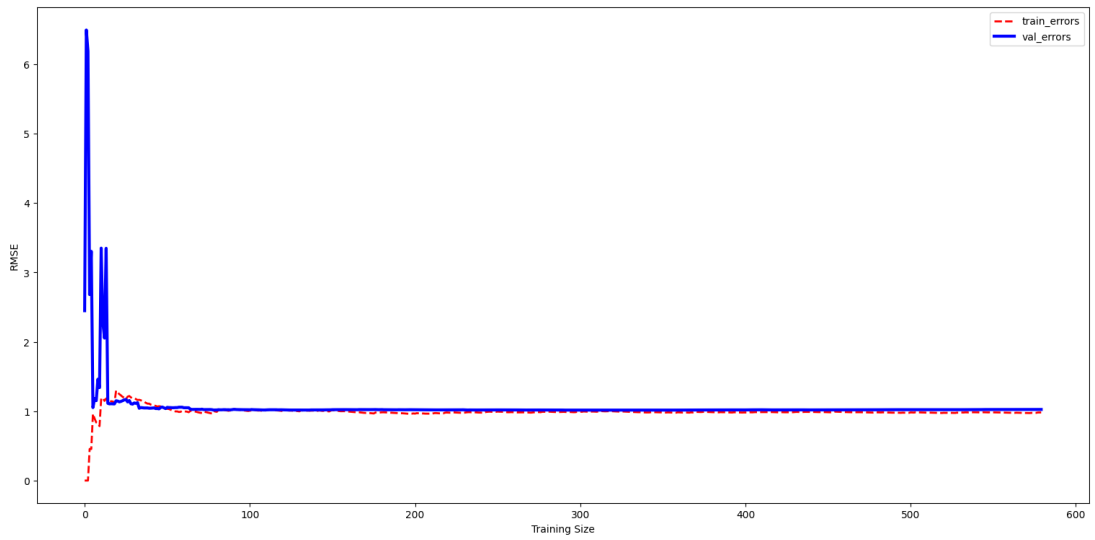
原始数据 & 二次拟合：



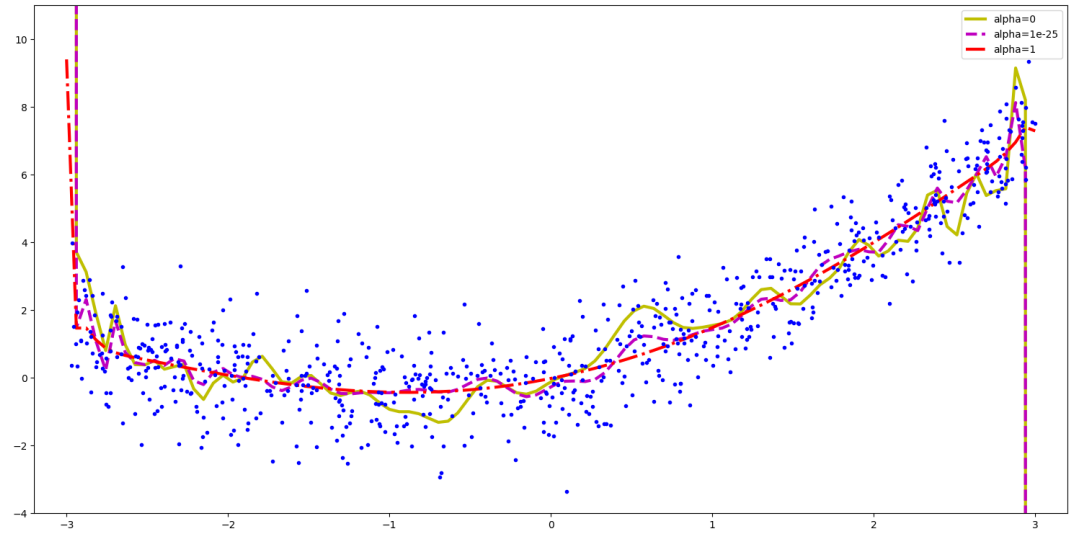
不同幂次的对比：

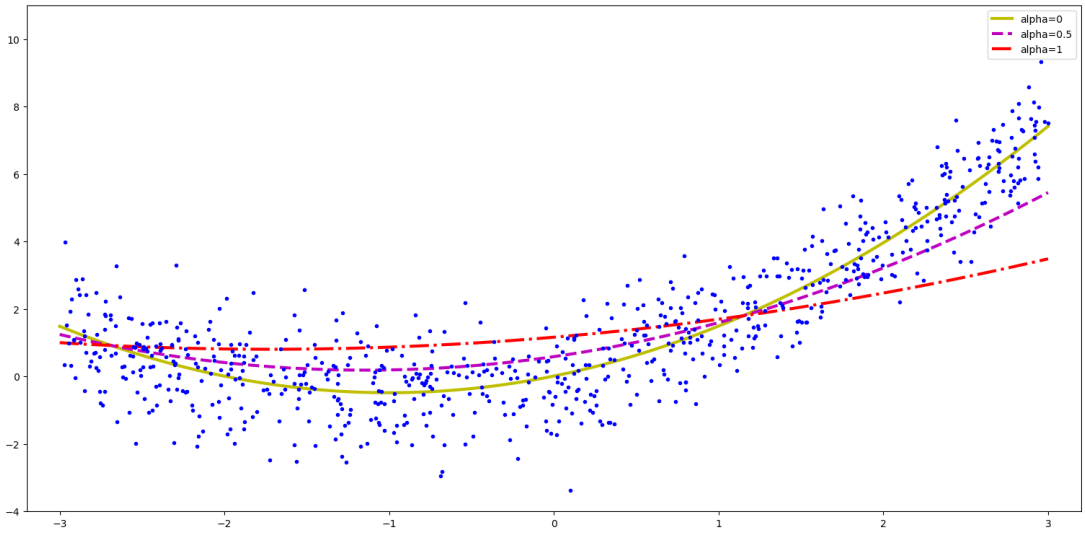



不同样本个数的对比：




L2 & L1 正则化：




**Logistic 回归**

```
from sklearn.datasets import load_iris
from sklearn.metrics import classification_report
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

Iris = load_iris()
X = Iris.data
Y = Iris.target
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.3,
random_state=0)
LR = LogisticRegression(penalty='l2', solver='newton-cg',
multi_class='multinomial')
LR.fit(x_train,y_train)
print("Logistic Regression 模型训练集的准确率: %.3f" % LR.score(x_train, y_train))
print("Logistic Regression 模型测试集的准确率: %.3f" % LR.score(x_test, y_test))
y_hat = LR.predict(x_test)
print(classification_report(y_test, y_hat, target_names=Iris.target_names))
```

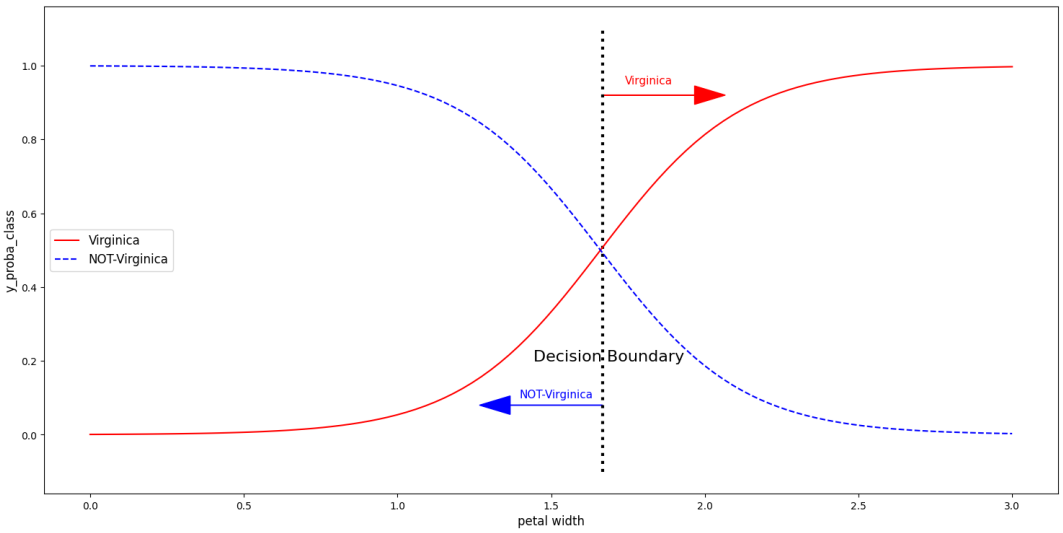
输出

```
Logistic Regression 模型训练集的准确率：0.981
Logistic Regression 模型测试集的准确率：0.978
              precision    recall  f1-score   support

      setosa       1.00      1.00      1.00        16
  versicolor       1.00      0.94      0.97        18
   virginica       0.92      1.00      0.96        11

    accuracy                           0.98        45
   macro avg       0.97      0.98      0.98        45
weighted avg       0.98      0.98      0.98        45
```

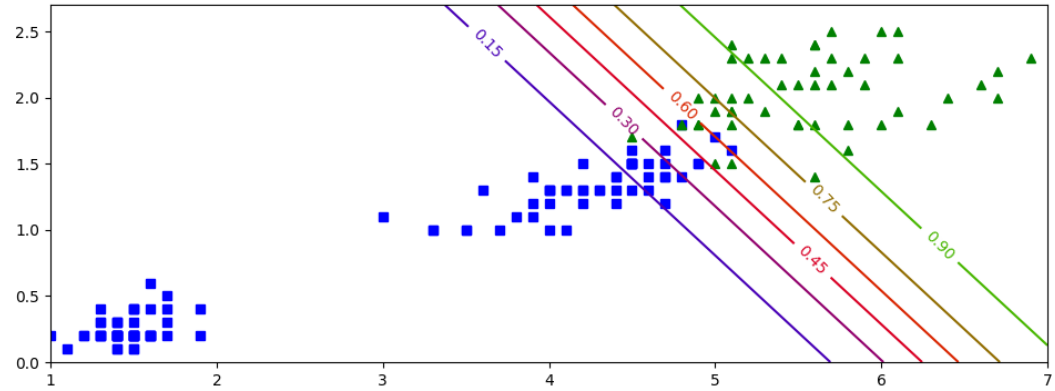
## Logistic 回归绘制决策边界

```python
Iris = datasets.load_iris()  # 导入鸢尾花数据集
print("属性列表：", list(Iris.keys()))  # 查看 Iris 有什么属性可以供我们调用
# print(Iris.DESCR)  # 查看数据集描述
X = Iris.data[:, 3:]  # 选取一个特征：花瓣宽度 petal width (cm)
y = (Iris['target'] == 2).astype(int)  # 将 virginica 设为 1
log_res = LogisticRegression()
log_res.fit(X, y)
X_curve = np.linspace(0, 3, 100).reshape(-1, 1)
y_proba = log_res.predict_proba(X_curve)
print(y_proba)  # 默认第一列是样本被预测为负类别 0 的概率，第二列是被预测为正类别 1 的概率
# 概率结果随所选特征的变化的可视化
decision_boundary = X_curve[y_proba[:, 1] >= 0.5][0]  # [1.66666667]
plt.plot([decision_boundary, decision_boundary], [-0.1, 1.1], 'k:', linewidth=3)
plt.plot(X_curve, y_proba[:, 1], 'r-', label='Virginica')
plt.plot(X_curve, y_proba[:, 0], 'b--', label='NOT-Virginica')
plt.arrow(decision_boundary[0], 0.08, -0.3, 0, head_width=0.05, head_length=0.1, fc='b', ec='b')
plt.arrow(decision_boundary[0], 0.92, 0.3, 0, head_width=0.05, head_length=0.1, fc='r', ec='r')
plt.text(decision_boundary+0.02, 0.2, 'Decision Boundary', fontsize=16, color='k', ha='center')
plt.text(decision_boundary-0.15, 0.1, 'NOT-Virginica', fontsize=11, color='b', ha='center')
plt.text(decision_boundary+0.15, 0.95, 'Virginica', fontsize=11, color='r', ha='center')
plt.xlabel('petal width', fontsize=12)
plt.ylabel('y_proba_class', fontsize=12)
plt.legend(loc='center left', fontsize=12)
plt.show()

X = Iris.data[:, (2, 3)]  # 提取了第 2、3 列 petal length (cm)  petal width (cm)
y = (Iris.target == 2).astype(int)  # 是不是 virginica
log_res.fit(X, y)
x0, x1 = np.meshgrid(np.linspace(1, 7, 500).reshape(-1, 1), np.linspace(0, 2.7, 500).reshape((-1,
1)))  # 笛卡尔积
X_new = np.c_[x0.ravel(), x1.ravel()]  # shape 为(250000, 2)
y_proba = log_res.predict_proba(X_new)
plt.figure(figsize=(10, 4))
plt.plot(X[y == 0, 0], X[y == 0, 1], 'bs')  # 类别为 0 的样本用蓝色方块
plt.plot(X[y == 1, 0], X[y == 1, 1], "g^")  # 类别为 1 的样本用绿色三角
zz = y_proba[:, 1].reshape(x0.shape)
contour = plt.contour(x0, x1, zz, cmap=plt.cm.brg)  # 绘制等高线图
plt.clabel(contour, inline=1)  # 在等高线上标注概率值
plt.show()

X = Iris.data[:, (2, 3)]  # 提取了第 2、3 列 petal length (cm)  petal width (cm)
y = Iris.target
softmax_log = LogisticRegression(multi_class='multinomial', solver='lbfgs')
softmax_log.fit(X, y)
x0, x1 = np.meshgrid(np.linspace(1, 7, 500).reshape(-1, 1), np.linspace(0, 2.7, 500).reshape((-1,
1)))  # 笛卡尔积
X_new = np.c_[x0.ravel(), x1.ravel()]  # shape 为(250000, 2)
y_proba = softmax_log.predict_proba(X_new)
y_predict = softmax_log.predict(X_new)
zzl = y_proba[:, 1].reshape(x0.shape)
zz = y_predict.reshape(x0.shape)
plt.figure(figsize=(10, 4))
plt.plot(X[y == 0, 0], X[y == 0, 1], "yo", label="setosa")
plt.plot(X[y == 1, 0], X[y == 1, 1], "bs", label="versicolor")
plt.plot(X[y == 2, 0], X[y == 2, 1], "g^", label="virginica")
plt.contourf(x0, x1, zz, cmap=matplotlib.colors.ListedColormap(['#66ccff', "#39c5bb", '#ffc0cb']))
# 绘制类别区域的等高线图
contour = plt.contour(x0, x1, zzl, cmap=plt.cm.brg)  # 绘制决策边界的等高线图
plt.clabel(contour, inline=1, fontsize=12)
plt.show()
```

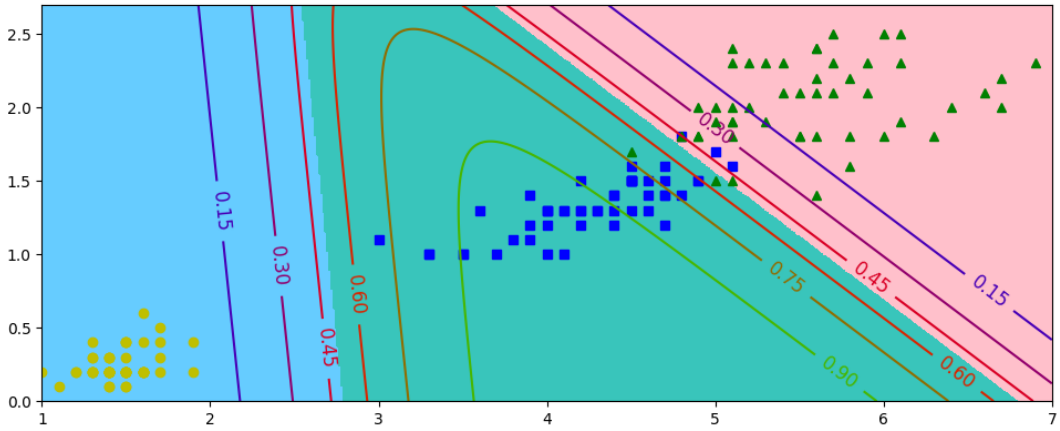
预测：




单分类边界：




多分类边界：

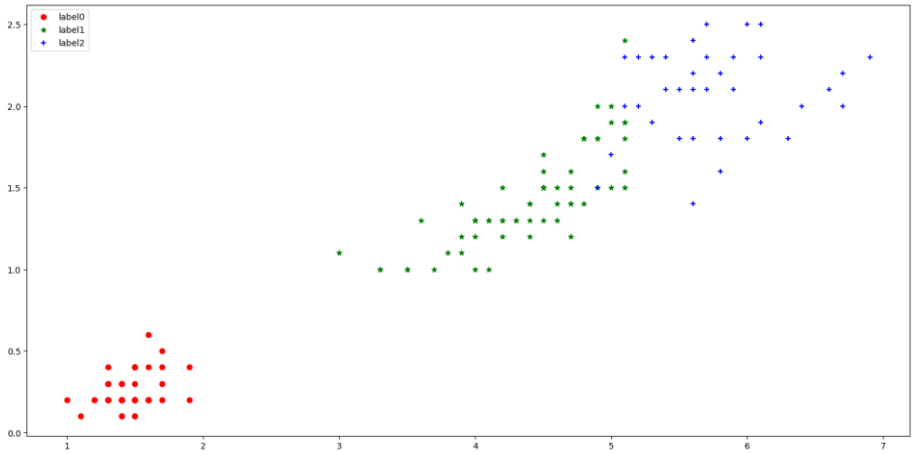


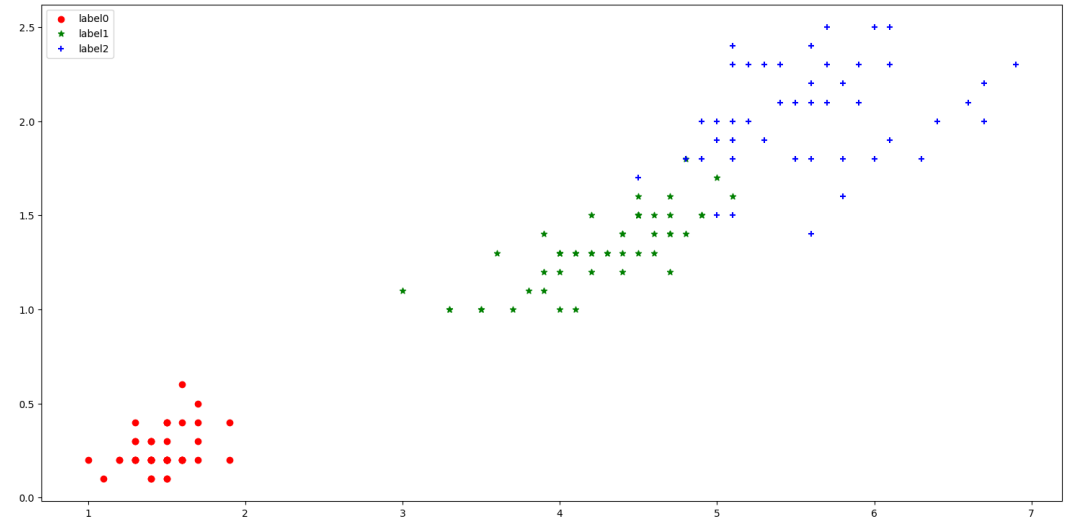
## 9.聚类

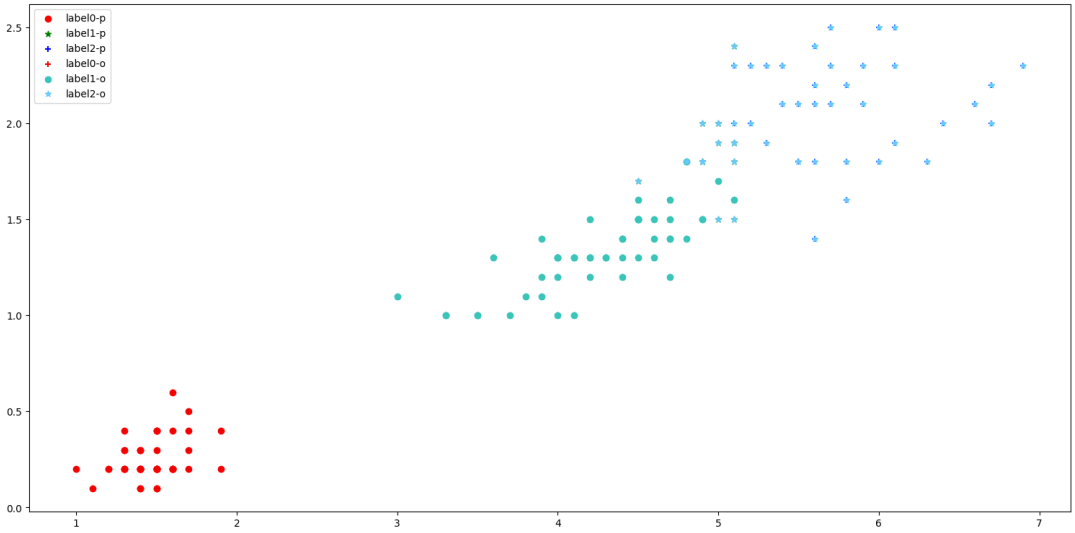
### K-means

```
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

Iris = load_iris()
X = Iris.data
Y = Iris.target

estimator = KMeans(n_clusters=3, n_init=10)  # n_init 是 KMeans 算法中用于指定随机初始化的次数的参数。
estimator.fit(X)
label_predict = estimator.labels_  # 是一个 list，包含对应的聚类标签(0,1,2)

# 绘制预测数据
xp0 = X[label_predict == 0]  # xp0 代指 predict 的 label 是 0 的 x
xp1 = X[label_predict == 1]
xp2 = X[label_predict == 2]
plt.scatter(xp0[:, 2], xp0[:, 3], c='red', marker='o', label='label0')
plt.scatter(xp1[:, 2], xp1[:, 3], c='green', marker='*', label='label1')
plt.scatter(xp2[:, 2], xp2[:, 3], c='blue', marker='+', label='label2')
plt.legend(loc=2)  # 添加图例(loc=2 代表左上角绘制)
plt.show()

# 绘制原始数据
xo0 = X[Y == 0]  # xo0 代指 original label 是 0 的 x
xo1 = X[Y == 1]
xo2 = X[Y == 2]
plt.scatter(xo0[:, 2], xo0[:, 3], c='red', marker='o', label='label0')
plt.scatter(xo1[:, 2], xo1[:, 3], c='green', marker='*', label='label1')
plt.scatter(xo2[:, 2], xo2[:, 3], c='blue', marker='+', label='label2')
plt.legend(loc=2)
plt.show()

# 比较绘制
plt.scatter(xp0[:, 2], xp0[:, 3], c='red', marker='o', label='label0-p')
plt.scatter(xp1[:, 2], xp1[:, 3], c='green', marker='*', label='label1-p')
plt.scatter(xp2[:, 2], xp2[:, 3], c='blue', marker='+', label='label2-p')
plt.scatter(xo0[:, 2], xo0[:, 3], c='#EE0000', marker='+', label='label0-o')
plt.scatter(xo1[:, 2], xo1[:, 3], c='#39C5BB', marker='o', label='label1-o')
plt.scatter(xo2[:, 2], xo2[:, 3], c='#66CCFF', marker='*', label='label2-o')
plt.legend(loc=2)
plt.show()
```

输出（依次为预测、原始、预测和原始叠加）





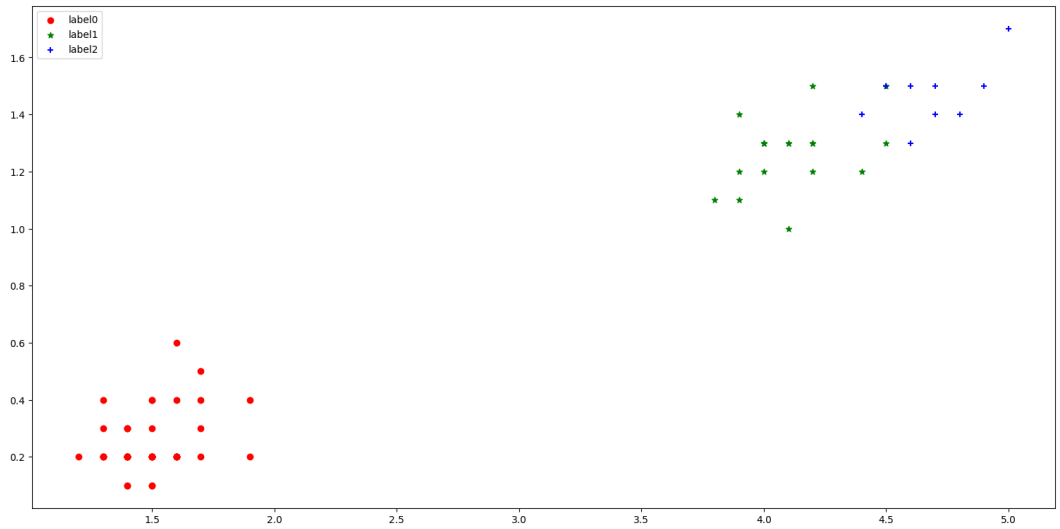

当聚类数量较多不便书写时可考虑以下两种方法:（以绘制原始数据的 target 值的图为例）

```
for target_value in set(Y):
    x = X[Y == target_value]
    if target_value == 0:
        plt.scatter(x[:, 2], x[:, 3], c='red', marker='o', label=f'Target
{target_value}')
    elif target_value == 1:
        plt.scatter(x[:, 2], x[:, 3], c='green', marker='*', label=f'Target
{target_value}')
    else:
        plt.scatter(x[:, 2], x[:, 3], c='blue', marker='+', label=f'Target
{target_value}')

# 定义颜色和标记对应关系
colors = ['red', 'green', 'blue']
markers = ['o', '*', '+']
for target_value in set(Y):
    x = X[Y == target_value]
    color = colors[target_value]
    marker = markers[target_value]
    label = f'Target {target_value}'
    plt.scatter(x[:, 2], x[:, 3], c=color, marker=marker, label=label)
```

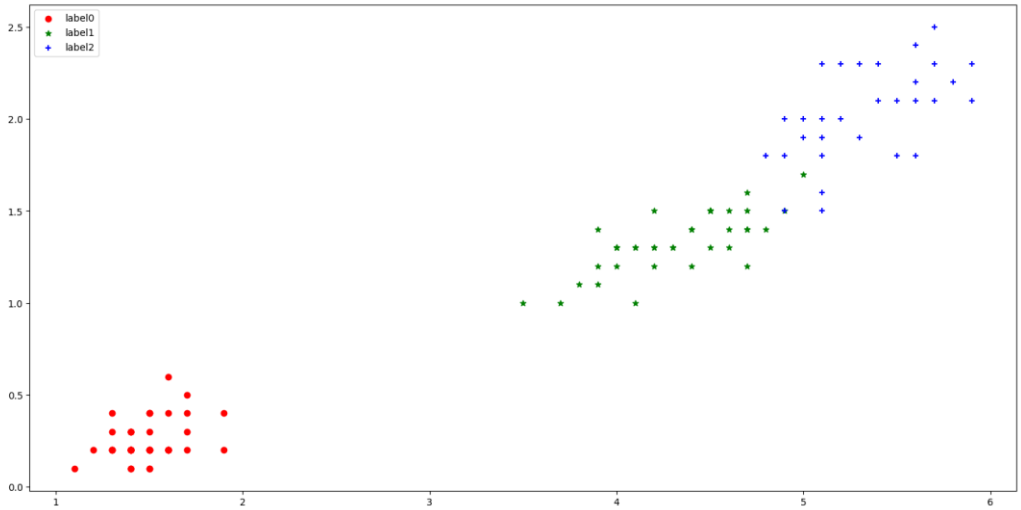
**DBSCAN**

```
from sklearn.datasets import load_iris
from sklearn.cluster import DBSCAN
import matplotlib.pyplot as plt

Iris = load_iris()
X = Iris.data
Y = Iris.target

dbscan = DBSCAN(eps=0.4, min_samples=9)  # 半径 0.4 内最少要有 9 个样本
dbscan.fit(X)
label_predict = dbscan.labels_  # 是一个 list，包含对应的聚类标签(0,1,2)

# 绘制预测数据
xp0 = X[label_predict == 0]  # xp0 代指 predict 的 label 是 0 的 x
xp1 = X[label_predict == 1]
xp2 = X[label_predict == 2]
plt.scatter(xp0[:, 2], xp0[:, 3], c='red', marker='o', label='label0')
plt.scatter(xp1[:, 2], xp1[:, 3], c='green', marker='*', label='label1')
plt.scatter(xp2[:, 2], xp2[:, 3], c='blue', marker='+', label='label2')
plt.legend(loc=2)  # 添加图例(loc=2 代表左上角绘制)
plt.show()
```

输出

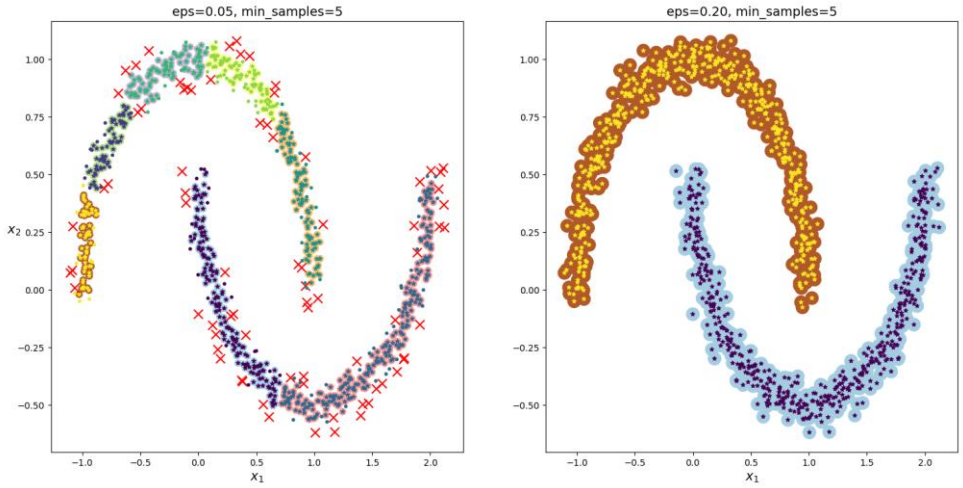



会发现和想要的结果不太一样。更改参数为 eps=0.4, min_samples=4，输出：



**DBSCAN 的可视化展示**

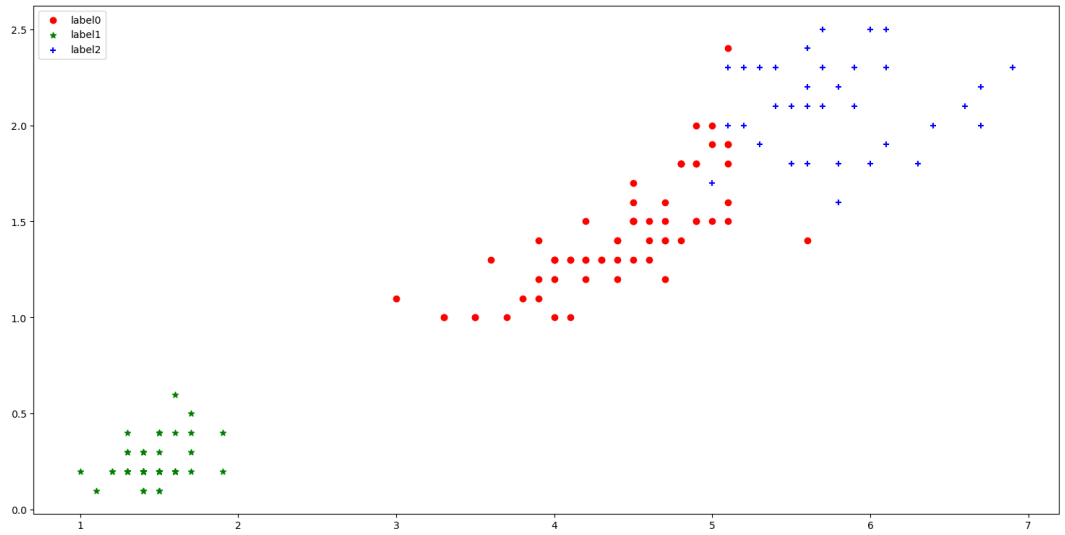
```
from sklearn.cluster import DBSCAN
from sklearn.datasets import make_moons
import matplotlib.pyplot as plt
import numpy as np

X, y = make_moons(n_samples=1000, noise=0.05, random_state=42)

dbscan = DBSCAN(eps=0.05, min_samples=5)
dbscan.fit(X)
dbscan2 = DBSCAN(eps=0.2, min_samples=5)
dbscan2.fit(X)


def plot_dbscan(dbscan, X, size, show_ylabels=True):
    core_mask = np.zeros_like(dbscan.labels_, dtype=bool)
    core_mask[dbscan.core_sample_indices_] = True
    anomalies_mask = dbscan.labels_ == -1
    non_core_mask = ~(core_mask | anomalies_mask)

    cores = dbscan.components_  # 核心点
    anomalies = X[anomalies_mask]  # 异常点
    non_cores = X[non_core_mask]  # 既不是核心点也不是异常点的数据点

    plt.scatter(cores[:, 0], cores[:, 1], c=dbscan.labels_[core_mask], marker='o',
s=size, cmap="Paired")  # 填充核心点的周围区域
    plt.scatter(cores[:, 0], cores[:, 1], marker='*', s=20,
c=dbscan.labels_[core_mask])  # 绘制核心点
    plt.scatter(anomalies[:, 0], anomalies[:, 1], c="r", marker="x", s=100)  # 绘制异
常点
    plt.scatter(non_cores[:, 0], non_cores[:, 1], c=dbscan.labels_[non_core_mask],
marker=".")  # 绘制其他点
    plt.xlabel("$x_1$", fontsize=14)
    if show_ylabels:
        plt.ylabel("$x_2$", fontsize=14, rotation=0)
    else:
        plt.tick_params(labelleft='off')
    plt.title("eps={:.2f}, min_samples={}".format(dbscan.eps, dbscan.min_samples),
fontsize=14)


plt.subplot(121)
plot_dbscan(dbscan, X, size=50)
plt.subplot(122)
plot_dbscan(dbscan2, X, size=200, show_ylabels=False)
plt.show()
```



## AGENS

```
from sklearn.datasets import load_iris
from sklearn.cluster import AgglomerativeClustering
import matplotlib.pyplot as plt

Iris = load_iris()
X = Iris.data
Y = Iris.target

AGENS = AgglomerativeClustering(linkage='ward', n_clusters=3)  # 以 ward 方差最小化的方式来计算簇的距离，分为 3 个簇。
AGENS.fit(X)
label_predict = AGENS.labels_  # 是一个 list，包含对应的聚类标签(0,1,2)

# 绘制预测数据
xp0 = X[label_predict == 0]  # xp0 代指 predict 的 label 是 0 的 x
xp1 = X[label_predict == 1]
xp2 = X[label_predict == 2]
plt.scatter(xp0[:, 2], xp0[:, 3], c='red', marker='o', label='label0')
plt.scatter(xp1[:, 2], xp1[:, 3], c='green', marker='*', label='label1')
plt.scatter(xp2[:, 2], xp2[:, 3], c='blue', marker='+', label='label2')
plt.legend(loc=2)  # 添加图例(loc=2 代表左上角绘制)
plt.show()
```

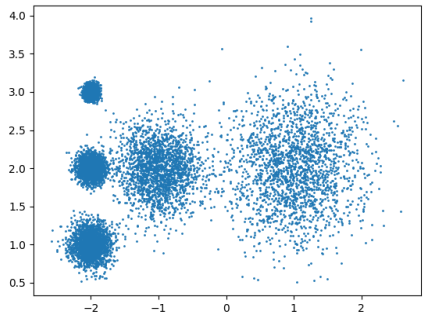
输出

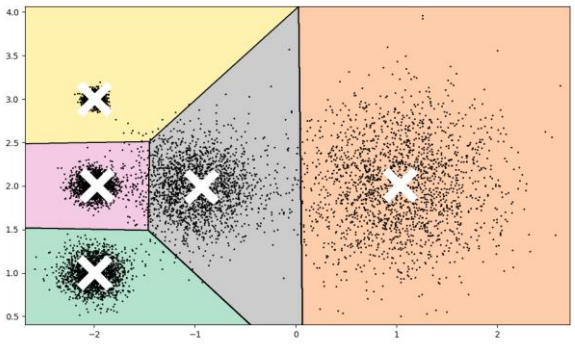


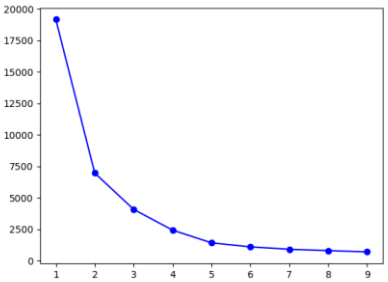

## 聚类效果的评估

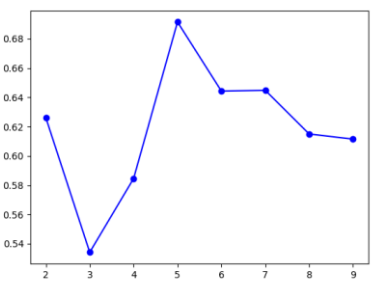
```
import matplotlib.pyplot as plt
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

blob_centers = np.array([[1, 2], [-1, 2], [-2, 1], [-2, 2], [-2, 3]])
blob_std = np.array([0.5, 0.3, 0.15, 0.1, 0.05])
X, y = make_blobs(n_samples=10000, centers=blob_centers, cluster_std=blob_std, random_state=42)


def plot_clusters(X, y=None):
    plt.scatter(X[:, 0], X[:, 1], c=y, s=1)


plot_clusters(X)
plt.show()
```

```python
kmeans = KMeans(n_clusters=5, random_state=42, n_init=10)
y_predict = kmeans.fit_predict(X)
print(y_predict)  # [0 2 0 ... 0 1 3]


def plot_data(X):
    plt.plot(X[:, 0], X[:, 1], "k.", markersize=2)


def plot_centroids(centroids, weights=None, circle_color="w", cross_color="k"):
    if weights is not None:
        centroids = centroids[weights > weights.max() / 10]
    plt.scatter(centroids[:, 0], centroids[:, 1], marker="o", s=30, linewidths=8, color=circle_color,
zorder=10, alpha=0.9)
    plt.scatter(centroids[:, 0], centroids[:, 1], marker="x", s=50, linewidths=50,
color=circle_color, zorder=11, alpha=1)


def plot_decision_boundaries(cluster, X, resolution=1000):
    X_min = X.min(axis=0) - 0.1
    X_max = X.max(axis=0) + 0.1
    xx, yy = np.meshgrid(np.linspace(X_min[0], X_max[0], resolution), np.linspace(X_min[1], X_max[1],
resolution))
    Z = cluster.predict(np.c_[xx.ravel(), yy.ravel()])
    Z = Z.reshape(xx.shape)
    plt.contourf(Z, extent=(X_min[0], X_max[0], X_min[1], X_max[1]), cmap="Pastel2")
    plt.contour(Z, extent=(X_min[0], X_max[0], X_min[1], X_max[1]), linewidths=1, colors="k")
    plot_data(X)
    plot_centroids(cluster.cluster_centers_)


plt.figure(figsize=(10, 6))
plot_decision_boundaries(kmeans, X)
plt.show()

# 评估方法：样本离最近聚类中心的总和:inertia。拐点附近的 K 更好
kmeans_per_k = [KMeans(n_clusters=k, n_init=10).fit(X) for k in range(1, 10)]
inertias = [model.inertia_ for model in kmeans_per_k]
plt.plot(range(1, 10), inertias, "bo-")
plt.show()

# 评估方法：轮廓系数。越接近 1 越好
silhouette_scores = [silhouette_score(X, model.labels_) for model in kmeans_per_k[1:]]  # 因为 k=1 无
法计算 si 的值
plt.plot(range(2, 10), silhouette_scores, "bo-")
plt.show()
```



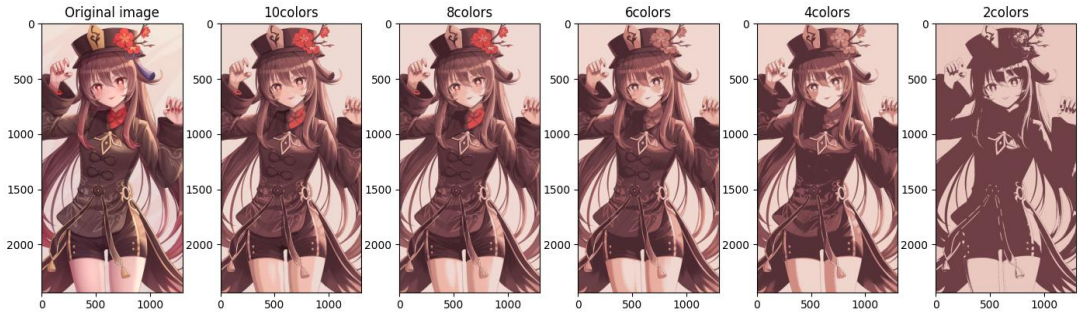






## 聚类在图像分割中的应用

```
from matplotlib.image import imread
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

image = imread("input/93088521.jpg")
print(image)
print(image.shape)  # (2434, 1300, 3)
X = image.reshape(-1, 3)  # 将图像的每个像素点作为一个样本

segmented_imgs = []
n_colors = (10, 8, 6, 4, 2)
for n_cluster in n_colors:
    kmeans = KMeans(n_clusters=n_cluster, n_init=10, random_state=42)
    kmeans.fit(X)
    segmented_img = kmeans.cluster_centers_[kmeans.labels_].reshape(image.shape) /
255
    segmented_imgs.append(segmented_img.reshape(image.shape))

plt.figure(figsize=(10, 5))
plt.subplot(161)
plt.imshow(image)
plt.title('Original image')

for idx, n_clusters in enumerate(n_colors):
    plt.subplot(162 + idx)
    plt.imshow(segmented_imgs[idx])
    plt.title('{}colors'.format(n_clusters))

plt.show()
```




`segmented_img = kmeans.cluster_centers_[kmeans.labels_].reshape(image.shape) / 255`

这个代码的作用是：将图像像素分配给最近的簇中心，并用相应的中心值替换原始像素值，以生成分割后的图像。以 `n_colors=10` 为例：

kmeans.cluster_centers_返回 10 个聚类中心，每个中心都是一个 RGB 值(红绿蓝三通道的值)的数组。

kmeans.labels_是对每个样本点分配的簇标签，表示每个像素点属于哪个聚类中心。

利用 kmeans.labels_中的标签去索引 kmeans.cluster_centers_，以找到每个像素点最接近的聚类中心。也就是，寻找图像中所有的像素点所对应的是哪个聚类中心。换句话说，就是将`(2434, 1300, 3)`个原始像素点映射到对应的`(10, 3)`的聚类中心。本次聚类中心是：

```
[[ 95.79026631  56.06818906  62.78858877]
 [239.68942344 218.24742709 211.64805813]
 [185.06630501 121.53136937 116.18169914]
 [152.30885607  90.31655274  93.93635596]
 [ 40.91918301  21.38834967  25.29352941]
 [ 75.42078651  41.28879712  48.52008189]
 [219.37994866 157.15874073 141.56587774]
 [243.17167364 190.31619774 178.98773861]
 [224.20816723  79.47032398  77.40751159]
 [119.04504141  71.37239398  77.16417315]]
```

## 采用半监督进行手写数字识别

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.cluster import KMeans
import numpy as np
import matplotlib.pyplot as plt

# 查看数据集
X_digits, y_digits = load_digits(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(X_digits, y_digits, random_state=42)
print(X_train.shape)  # (1347, 64)
print(X_test.shape)  # (450, 64)

# 选取 50 个完全随机的样本进行逻辑回归
n_labeled = 50
log_reg = LogisticRegression(max_iter=10000, random_state=42)
np.random.seed(42)
random_indices = np.random.choice(X_train.shape[0], n_labeled, replace=False)
log_reg.fit(X_train[random_indices], y_train[random_indices])
print("50 个随机样本的得分：", log_reg.score(X_test, y_test))

# 先聚类成 50 个簇，然后自行打标签，再以这些有代表性的图像来训练
k = n_labeled  # 之前的是随机选取的 50 个训练样本，现在尝试选择 50 个离簇中心最近的点
kmeans = KMeans(n_clusters=k, n_init=10, random_state=42)
X_digits_dist = kmeans.fit_transform(X_train)  # 距离矩阵 X_digits_dist 的每一行表示此数据点到每个簇中心的距离。
representative_digits_idx = np.argmin(X_digits_dist, axis=0)  # 50 个离簇中心最近的样本的索引
X_representative_digits = X_train[representative_digits_idx]
# 查看样本的样子，然后自行打标签
plt.figure(figsize=(8, 2))
for index, X_representative_digit in enumerate(X_representative_digits):
    plt.subplot(k // 10, 10, index + 1)
    plt.imshow(X_representative_digit.reshape(8, 8), cmap="binary", interpolation="bilinear")
    plt.axis('off')
plt.show()
y_representative_digits = np.array([
    3, 8, 5, 1, 0, 4, 6, 7, 1, 2,
    4, 5, 9, 9, 6, 2, 1, 7, 2, 8,
    6, 7, 9, 2, 0, 9, 2, 5, 4, 5,
    0, 7, 1, 3, 7, 1, 2, 8, 4, 3,
    7, 4, 5, 3, 3, 9, 1, 3, 6, 8])
log_reg = LogisticRegression(max_iter=10000, random_state=42)
log_reg.fit(X_representative_digits, y_representative_digits)
print("50 个有代表性样本的得分：", log_reg.score(X_test, y_test))

# 标签传播 认为在该簇内的样本的属性值就是簇中心点的属性值
y_train_propagated = np.empty(len(X_train), dtype=np.int32)  # 创建一个未初始化的数组，数组的长度是 X_train 的样本数量。
for i in range(k):
    y_train_propagated[kmeans.labels_ == i] = y_representative_digits[i]
log_reg = LogisticRegression(max_iter=10000, random_state=42)
log_reg.fit(X_train, y_train_propagated)
print("标签传播到同一簇中的所有其他实例的得分：", log_reg.score(X_test, y_test))

# 不一定要选取簇中所有的样本，只考虑前 20%试试
percentile_closest = 20
X_cluster_dist = X_digits_dist[np.arange(len(X_train)), kmeans.labels_]  # 记录每个样本到当前簇中心的距离
for i in range(k):
    cutoff_distance = np.percentile(X_cluster_dist[kmeans.labels_ == i], percentile_closest)  # 选择属于当前簇的所有样本，排序找到前 20%
    X_cluster_dist[(kmeans.labels_ == i) & (X_cluster_dist > cutoff_distance)] = -1  # &逐元素按位与，对于两个布尔数组，它返回一个新的布尔数组
X_train_partially_propagated = X_train[X_cluster_dist != -1]  # 使用布尔数组作为索引来选择数组中满足某些条件的元素
y_train_partially_propagated = y_train_propagated[X_cluster_dist != -1]
log_reg = LogisticRegression(max_iter=10000, random_state=42)
log_reg.fit(X_train_partially_propagated, y_train_partially_propagated)
print("标签传播到同一簇中的前 20%实例的得分：", log_reg.score(X_test, y_test))
```

输出得分：

```
50 个随机样本的得分： 0.8244444444444444
50 个有代表性样本的得分： 0.9022222222222223
标签传播到同一簇中的所有其他实例的得分： 0.9155555555555556
```

标签传播到同一簇中的前 20%实例的得分： 0.9244444444444444

手写数据采样的 50 个中心点是：

## 10.关联规则

**自制数据集**

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.expand_frame_repr', False)

likes_raw_data = {'ID': [1, 2, 3, 4, 5, 6, 7, 8],
                  'Likes': [['Klee', 'Hu Tao', 'Nahida', 'Sangonomiya Kokomi', 'Ganyu', 'Yoimiya'],
                            ['Keqing', 'Ganyu', 'Yoimiya', 'Hu Tao', 'Sangonomiya Kokomi'],
                            ['Sangonomiya Kokomi', 'Qiqi', 'Nahida', 'Keqing'],
                            ['Qiqi', 'Sayu', 'Klee', 'Nahida', 'Hu Tao'],
                            ['Yoimiya', 'Sangonomiya Kokomi', 'Nahida'],
                            ['Klee', 'Sigewinne', 'Yoimiya', 'Nahida'],
                            ['Klee', 'Nahida', 'Sigewinne', 'Sayu', 'Qiqi'],
                            ['Nahida', 'Ganyu', 'Klee', 'Qiqi'],
                            ]
                  }

likes_df = pd.DataFrame(likes_raw_data)
print(likes_df)
# .str 是用于处理字符串的特殊属性，用于对 Series 中的每个元素进行字符串操作
likes_Likes = likes_df['Likes'].str.join(',')
print(likes_Likes)
# 将 Series 中的每个字符串按 sep 进行分割，并返回一个由虚拟(dummy)/指示(indicator)变量组成的 DataFrame。
# 即使分割时重复出现了某个字符串，也只记为 1。
likes_Likes = likes_Likes.str.get_dummies(',')
print(likes_Likes)
# 转换成 bool 类型，以避免 DeprecationWarning。布尔类型的计算性能更好。
likes_Likes = likes_Likes.astype(bool)
# 设置最小支持度查找频繁项集
likes_frequent_itemsets = apriori(likes_Likes, min_support=0.5, use_colnames=True)
print(likes_frequent_itemsets)
# 设置其他衡量标准(metric)的最小阈值(min_threshold)的关联规则
likes_association_rules = association_rules(likes_frequent_itemsets, metric='lift')
print(likes_association_rules)
print(likes_association_rules[(likes_association_rules['lift'] > 1) &
(likes_association_rules['confidence'] > 0.7)])
```

先将[Nahida, Ganyu, Klee, Qiqi]变成这样的 Nahida,Ganyu,Klee,Qiqi，得到指示矩阵：

| | Ganyu | Hu Tao | Keqing | Klee | Nahida | Qiqi | Sangonomiya Kokomi | Sayu | Sigewinne | Yoimiya |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 0 | 1 | 1 | 0 | 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 1 | 0 | 0 | 0 | 1 | 0 | 0 | 1 |
| 2 | 0 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 0 | 0 |
| 3 | 0 | 1 | 0 | 1 | 1 | 1 | 0 | 1 | 0 | 0 |
| 4 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | 0 | 1 |
| 5 | 0 | 0 | 0 | 1 | 1 | 0 | 0 | 0 | 1 | 1 |
| 6 | 0 | 0 | 0 | 1 | 1 | 1 | 0 | 1 | 1 | 0 |
| 7 | 1 | 0 | 0 | 1 | 1 | 1 | 0 | 0 | 0 | 0 |

然后转为 bool 类型，得到频繁项集与关联规则：

| | support | itemsets |
|---|---|---|
| 0 | 0.625 | (Klee) |
| 1 | 0.875 | (Nahida) |
| 2 | 0.500 | (Qiqi) |
| 3 | 0.500 | (Sangonomiya Kokomi) |
| 4 | 0.500 | (Yoimiya) |
| 5 | 0.625 | (Klee, Nahida) |
| 6 | 0.500 | (Qiqi, Nahida) |

| | antecedents | consequents | antecedent support | consequent support | support | confidence | lift | leverage | conviction | zhangs_metric |
|---|---|---|---|---|---|---|---|---|---|---|
| 0 | (Klee) | (Nahida) | 0.625 | 0.875 | 0.625 | 1.000000 | 1.142857 | 0.078125 | inf | 0.333333 |
| 1 | (Nahida) | (Klee) | 0.875 | 0.625 | 0.625 | 0.714286 | 1.142857 | 0.078125 | 1.312500 | 1.000000 |
| 2 | (Qiqi) | (Nahida) | 0.500 | 0.875 | 0.500 | 1.000000 | 1.142857 | 0.062500 | inf | 0.250000 |
| 3 | (Nahida) | (Qiqi) | 0.875 | 0.500 | 0.500 | 0.571429 | 1.142857 | 0.062500 | 1.166667 | 1.000000 |

**电影数据集**

```
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.expand_frame_repr', False)

movies_raw_data = pd.read_csv('ml-latest-small/movies.csv')
movies_genres = movies_raw_data['genres'].str.get_dummies('|').astype(bool)
print(movies_genres.shape)  # (9742, 20)
genres_freq = apriori(movies_genres, use_colnames=True, min_support=0.03)
print(genres_freq.sort_values(by='support', ascending=False))
genres_rules = association_rules(genres_freq, metric='lift', min_threshold=2)
print(genres_rules.sort_values(by='lift', ascending=False))
```

| | antecedents | consequents | antecedent support | consequent support | support | confidence | lift | leverage | conviction | zhangs_metric |
|---|---|---|---|---|---|---|---|---|---|---|
| 11 | (Animation) | (Children) | 0.062718 | 0.068158 | 0.031000 | 0.494272 | 7.251799 | 0.026725 | 1.842573 | 0.919791 |
| 10 | (Children) | (Animation) | 0.068158 | 0.062718 | 0.031000 | 0.454819 | 7.251799 | 0.026725 | 1.719213 | 0.925161 |
| 4 | (Children) | (Adventure) | 0.068158 | 0.129645 | 0.032026 | 0.469880 | 3.624360 | 0.023190 | 1.641806 | 0.777052 |
| 5 | (Adventure) | (Children) | 0.129645 | 0.068158 | 0.032026 | 0.247031 | 3.624360 | 0.023190 | 1.237556 | 0.831947 |
| 6 | (Adventure) | (Fantasy) | 0.129645 | 0.079963 | 0.034285 | 0.264450 | 3.307149 | 0.023918 | 1.250815 | 0.801540 |
| 7 | (Fantasy) | (Adventure) | 0.079963 | 0.129645 | 0.034285 | 0.428755 | 3.307149 | 0.023918 | 1.523610 | 0.758257 |

可以看出 Animation 和 Children 有强关联性。

## 11.集成

### 集成算法

```
from sklearn.model_selection import cross_val_score
from sklearn.datasets import load_iris
from sklearn.ensemble import AdaBoostClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import BaggingClassifier
from sklearn.ensemble import RandomForestClassifier

Iris = load_iris()
X = Iris.data
Y = Iris.target
clf = AdaBoostClassifier(n_estimators=100)  # 迭代次数为 100
scores = cross_val_score(clf, X, Y)  # 交叉验证
print('AdaBoost 准确率：', scores.mean())

bagging = BaggingClassifier(KNeighborsClassifier(), max_samples=0.5,
max_features=0.5)  # 结合 KNN 分类器，每个基分类器抽样样本数量和特征数量的最大比例为 0.5
scores = cross_val_score(bagging, X, Y)
print('bagging 准确率：', scores.mean())

clf = RandomForestClassifier(n_estimators=10, max_features=2)  # 决策树的数量为 10
棵，每棵树最大的特征数量为 2
scores = cross_val_score(clf, X, Y)
print('RF 准确率：', scores.mean())
```

输出

```
AdaBoost 准确率：  0.9466666666666665
bagging 准确率：  0.9533333333333334
RF 准确率：  0.9533333333333334
```

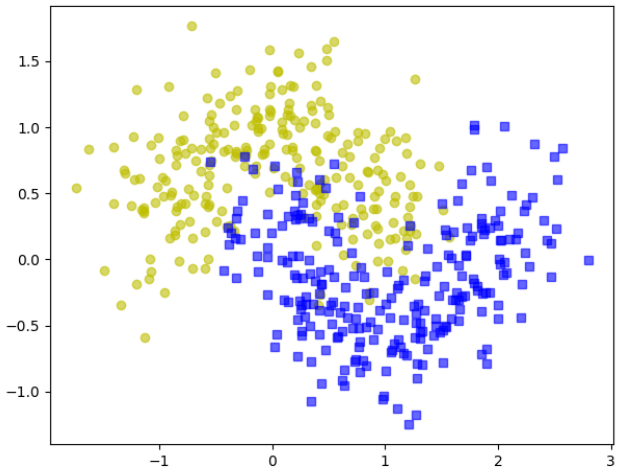
### 硬投票&软投票

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_moons
from sklearn.ensemble import RandomForestClassifier, VotingClassifier,
BaggingClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.tree import DecisionTreeClassifier
from matplotlib.colors import ListedColormap

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
plt.plot(X[:, 0][y == 0], X[:, 1][y == 0], 'yo', alpha=0.6)
plt.plot(X[:, 0][y == 1], X[:, 1][y == 1], 'bs', alpha=0.6)
plt.show()

# 硬投票
log_clf = LogisticRegression(random_state=42)
rnd_clf = RandomForestClassifier(random_state=42)
svm_clf = SVC(random_state=42)
voting_clf = VotingClassifier(estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc',
svm_clf)], voting='hard')
voting_clf.fit(X_train, y_train)
for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    print(clf.__class__.__name__, accuracy_score(y_test, y_pred))

# 软投票
```

```
log_clf = LogisticRegression(random_state=42)
rnd_clf = RandomForestClassifier(random_state=42)
svm_clf = SVC(probability=True, random_state=42)  # SVM 默认不计算概率值，但软投票需要知道概率值。
voting_clf = VotingClassifier(estimators=[('lr', log_clf), ('rf', rnd_clf), ('svc', svm_clf)], voting='soft')
voting_clf.fit(X_train, y_train)
for clf in (log_clf, rnd_clf, svm_clf, voting_clf):
    clf.fit(X_train, y_train)
    y_pred = clf.predict(X_test)
    print(clf.__class__.__name__, accuracy_score(y_test, y_pred))
```

输出

```
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.896
VotingClassifier 0.912
LogisticRegression 0.864
RandomForestClassifier 0.896
SVC 0.896
VotingClassifier 0.92
```



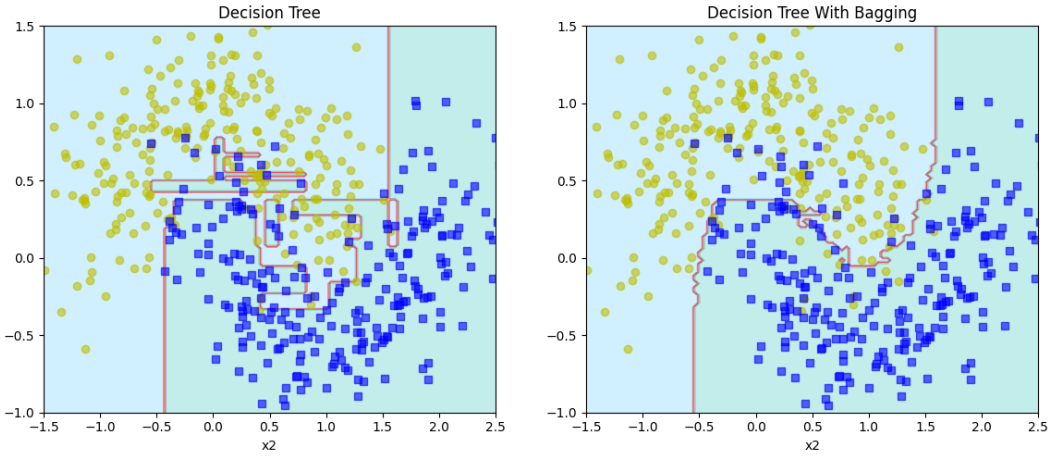

**bagging(多次采样，分别训练多个模型，预测时用所有模型结果进行集成)**

```
# bagging
tree_clf = DecisionTreeClassifier(random_state=42)
tree_clf.fit(X_train, y_train)
y_pred_tree = tree_clf.predict(X_test)
print(accuracy_score(y_test, y_pred_tree))
bag_clf = BaggingClassifier(DecisionTreeClassifier(),
                            n_estimators=500,  # 集成的基础分类器的数量
                            max_samples=100,  # （有放回抽样）从训练集中抽取的样本数量
                            bootstrap=True,  # 随机采样
                            # n_jobs=-1,  # 用全部 CPU 核心进行训练
                            random_state=42
                            )
bag_clf.fit(X_train, y_train)
y_pred = bag_clf.predict(X_test)
print(accuracy_score(y_test, y_pred))

def plot_decision_boundary(clf, X, y, axes=(-1.5, 2.5, -1, 1.5)):
    x1s = np.linspace(axes[0], axes[1], 100)
    x2s = np.linspace(axes[2], axes[3], 100)
    x1, x2 = np.meshgrid(x1s, x2s)
    X_new = np.c_[x1.ravel(), x2.ravel()]
    y_pred = clf.predict(X_new).reshape(x1.shape)
    custom_cmap = ListedColormap(['#66ccff', '#ee0000', '#39c5bb'])
```

```
    plt.contourf(x1, x2, y_pred, cmap=custom_cmap, alpha=0.3)
    plt.plot(X[:, 0][y == 0], X[:, 1][y == 0], 'yo', alpha=0.6)
    plt.plot(X[:, 0][y == 1], X[:, 1][y == 1], 'bs', alpha=0.6)
    plt.axis(axes)
    plt.xlabel('x1')
    plt.xlabel('x2')

plt.figure(figsize=(12, 5))
plt.subplot(121)
plot_decision_boundary(tree_clf, X, y)
plt.title('Decision Tree')
plt.subplot(122)
plot_decision_boundary(bag_clf, X, y)
plt.title('Decision Tree With Bagging')
plt.show()

bag_clf = BaggingClassifier(DecisionTreeClassifier(),
                            n_estimators=500,
                            max_samples=100,
                            bootstrap=True,
                            random_state=42,
                            oob_score=True
                            )
bag_clf.fit(X_train, y_train)
print(bag_clf.oob_score_)  # 在训练过程中用每次没被训练到的数据来获取模型的性能估计
y_pred = bag_clf.predict(X_test)
print(accuracy_score(y_test, y_pred))  # 使用独立的测试集来评估模型性能
# print(bag_clf.oob_decision_function_)  # 每个样本属于对应类别的概率值
```

输出

```
0.856
0.904
0.9253333333333333
0.904
```



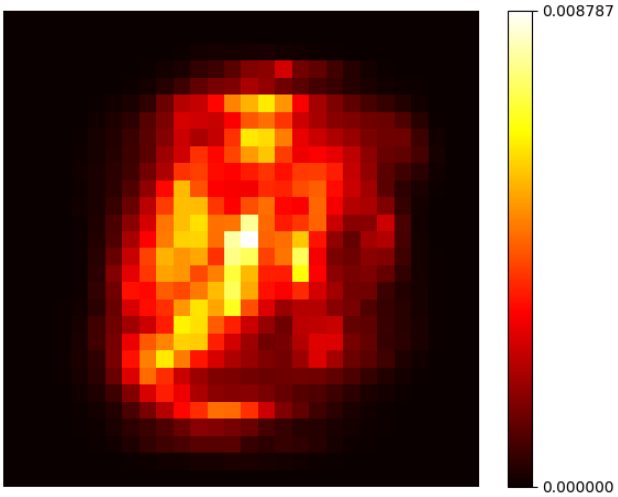

**用随机森林进行特征重要性判断**

```
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_iris, fetch_openml

iris = load_iris()
rf_clf = RandomForestClassifier(n_estimators=500)
rf_clf.fit(iris['data'], iris['target'])
for name, score in zip(iris['feature_names'], rf_clf.feature_importances_):
    print(name, score)

mnist = fetch_openml('MNIST_784', parser='auto')
rf_clf = RandomForestClassifier(n_estimators=500, n_jobs=-1) # 任务量大的时候才指定为-
```

```
1，这时候能节省时间。任务量小的时候指定为-1 反而增加开销。
rf_clf.fit(mnist['data'], mnist['target'])
print(rf_clf.feature_importances_.shape)
image = rf_clf.feature_importances_.reshape(28, 28)
plt.imshow(image, cmap='hot')
plt.axis('off')
char = plt.colorbar(ticks=[rf_clf.feature_importances_.min(),
rf_clf.feature_importances_.max()])
# char.ax.set_yticklabels(['Not important','Very important'])  # 更改坐标标签文字
plt.show()
```

输出

```
sepal length (cm) 0.10156108390408021
sepal width (cm) 0.02390628173194181
petal length (cm) 0.4432523934459005
petal width (cm) 0.43128024091807743
```

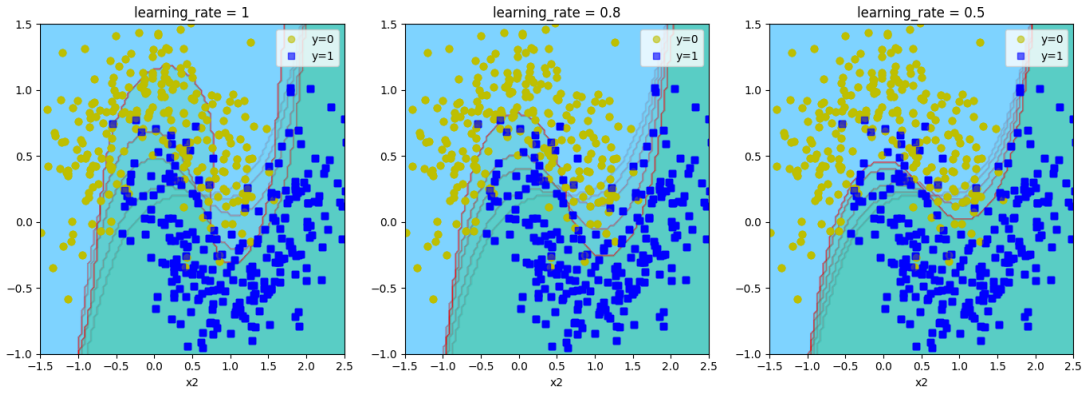


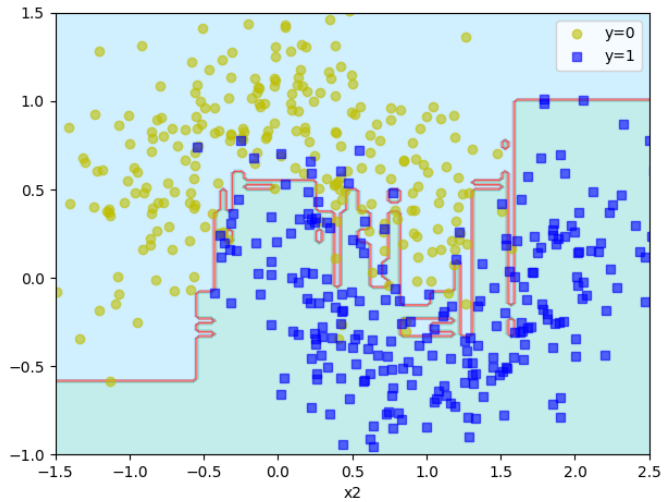

MNIST_784 是 70000 个 28*28=784 个像素点的手写数据集。

**模拟 AdaBoost 和 AdaBoost**

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from sklearn.model_selection import train_test_split
from sklearn.datasets import make_moons
from sklearn.metrics import mean_squared_error, accuracy_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier

X, y = make_moons(n_samples=500, noise=0.30, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)
def plot_decision_boundary(clf, X, y, axes=(-1.5, 2.5, -1, 1.5), show_legend=True,
show_other_label=False):
    x1s = np.linspace(axes[0], axes[1], 100)
    x2s = np.linspace(axes[2], axes[3], 100)
    x1, x2 = np.meshgrid(x1s, x2s)
    X_new = np.c_[x1.ravel(), x2.ravel()]
    y_pred = clf.predict(X_new).reshape(x1.shape)
    custom_cmap = ListedColormap(['#66ccff', '#ee0000', '#39c5bb'])
    plt.contourf(x1, x2, y_pred, cmap=custom_cmap, alpha=0.3)
    plt.plot(X[:, 0][y == 0], X[:, 1][y == 0], 'yo', alpha=0.6, label='y=0')
    plt.plot(X[:, 0][y == 1], X[:, 1][y == 1], 'bs', alpha=0.6, label='y=1')
    if show_other_label:
        plt.plot(X[:, 0][y == -1], X[:, 1][y == -1], 'g+', alpha=0.6, label='y=-1')
        plt.plot(X[:, 0][y > 1], X[:, 1][y > 1], 'mx', alpha=0.6, label='y>1')
        plt.plot(X[:, 0][y < -1], X[:, 1][y < -1], 'cx', alpha=0.6, label='y<-1')
```

```
    if show_legend:
        plt.legend()  # 显示图例
    plt.axis(axes)
    plt.xlabel('x1')
    plt.xlabel('x2')

# 模拟 AdaBoost
plt.figure(figsize=(14, 5))
for subplot, learning_rate in ((131, 1), (132, 0.8), (133, 0.5)):
    sample_weights = np.ones(len(X_train))  # 样本权重矩阵
    plt.subplot(subplot)
    for i in range(5):
        # 以 SVM 分类器模拟 5 次 AdaBoost
        svm_clf = SVC(kernel='rbf', C=0.05, random_state=42)  # 高斯核函数 rbf，正则化
参数 0.05
        svm_clf.fit(X_train, y_train, sample_weight=sample_weights)
        y_pred = svm_clf.predict(X_train)
        sample_weights[y_pred != y_train] *= (1 + learning_rate)  # 错分的权重升高
        plot_decision_boundary(svm_clf, X, y, show_legend=(bool(i == 0)))
        plt.title('learning_rate = {}'.format(learning_rate))
plt.show()

# AdaBoost
ada_clf = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1),
                             n_estimators=500,
                             learning_rate=0.5,
                             random_state=42
                             )
ada_clf.fit(X_train, y_train)
plot_decision_boundary(ada_clf, X, y)
plt.show()
```

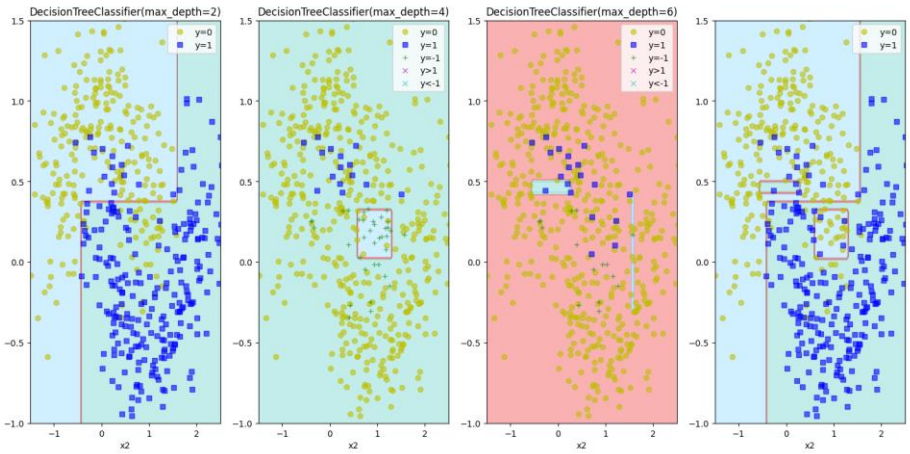


## 模拟 Gradient Boosting（同理可用于回归问题，每次都去预测新的误差）

```
tree_reg1 = DecisionTreeClassifier(max_depth=2)
tree_reg1.fit(X, y)
y2 = y - tree_reg1.predict(X)
tree_reg2 = DecisionTreeClassifier(max_depth=4)
tree_reg2.fit(X, y2)
y3 = y2 - tree_reg2.predict(X)
tree_reg3 = DecisionTreeClassifier(max_depth=6)
tree_reg3.fit(X, y3)
for subplot, tree_reg, y_n in ((141, tree_reg1, y), (142, tree_reg2, y2), (143,
tree_reg3, y3)):
    plt.subplot(subplot)
    plot_decision_boundary(tree_reg, X, y_n, show_other_label=bool(subplot != 141))
    plt.title('{}'.format(tree_reg))
plt.subplot(144)
x1s = np.linspace(-1.5, 3.5, 100)
x2s = np.linspace(-1, 1.5, 100)
x1, x2 = np.meshgrid(x1s, x2s)
X_new = np.c_[x1.ravel(), x2.ravel()]
y_pred = sum(tree.predict(X_new).reshape(x1.shape) for tree in (tree_reg1,
tree_reg2, tree_reg3))
custom_cmap = ListedColormap(['#66ccff', '#ee0000', '#39c5bb'])
plt.contourf(x1, x2, y_pred, cmap=custom_cmap, alpha=0.3)
plt.plot(X[:, 0][y == 0], X[:, 1][y == 0], 'yo', alpha=0.6, label='y=0')
plt.plot(X[:, 0][y == 1], X[:, 1][y == 1], 'bs', alpha=0.6, label='y=1')
plt.legend()  # 显示图例
plt.axis((-1.5, 2.5, -1, 1.5))
plt.xlabel('x1')
plt.xlabel('x2')
plt.show()
```

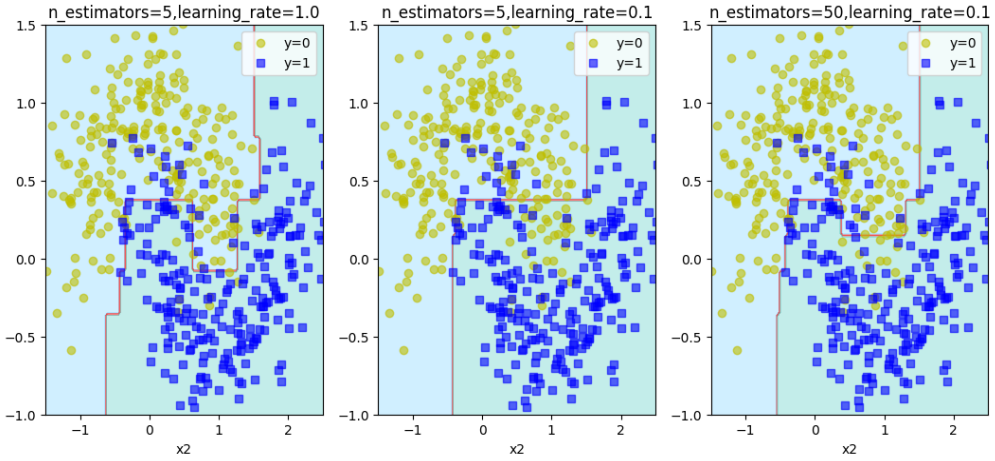
每次去预测前一个样本的误差：




## Gradient Boost

```
gbclf1 = GradientBoostingClassifier(max_depth=2,
                                    n_estimators=5,
                                    learning_rate=1.0,
                                    random_state=42
                                    )
gbclf1.fit(X, y)
gbclf2 = GradientBoostingClassifier(max_depth=2,
                                    n_estimators=5,
                                    learning_rate=0.1,
                                    random_state=42
                                    )
gbclf2.fit(X, y)
gbclf3 = GradientBoostingClassifier(max_depth=2,
```

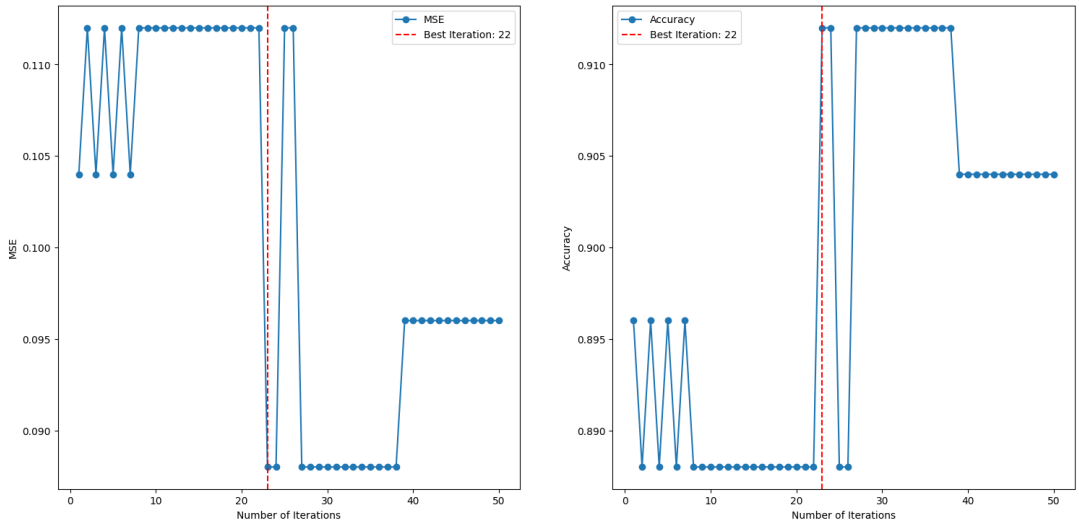
```
                                    n_estimators=50,
                                    learning_rate=0.1,
                                    random_state=42
                                    )
gbclf3.fit(X, y)
plt.figure(figsize=(11, 4))
for subplot, gbclf in ((131, gbclf1), (132, gbclf2), (133, gbclf3)):
    plt.subplot(subplot)
    plot_decision_boundary(gbclf, X, y)
    plt.title("n_estimators={},learning_rate={}".format(gbclf.n_estimators,
gbclf.learning_rate))
plt.show()

X_train, X_val, y_train, y_val = train_test_split(X, y, random_state=42)
gbclf = GradientBoostingClassifier(max_depth=2,
                                   n_estimators=50,
                                   random_state=42
                                   )
gbclf.fit(X_train, y_train)
```




**通过误差提前停止**

```
# 1.MSE
# for i, y_pred_i in enumerate(gbclf.staged_predict(X_val)):
#     # 在这里，y_pred_i 是模型在第 i 次迭代后的预测结果
#     error = mean_squared_error(y_val, y_pred_i)
#     print(f'第 {i+1} 次, MSE: {error}')
errors = [mean_squared_error(y_val, y_pred) for y_pred in
gbclf.staged_predict(X_val)]
print(np.argmin(errors))  # 22，也就是第 23 次
# 2.Accuracy
accuracies = [accuracy_score(y_val, y_pred) for y_pred in
gbclf.staged_predict(X_val)]
print(np.argmax(accuracies))  # 22，也就是第 23 次
# 绘制误差/精度曲线
plt.figure(figsize=(10, 6))
plt.subplot(121)
plt.plot(range(1, len(errors) + 1), errors, label='MSE', marker='o')
plt.axvline(x=np.argmin(errors)+1, color='red', linestyle='--', label=f'Best
Iteration: {np.argmin(errors)}')
plt.xlabel('Number of Iterations')
plt.ylabel('MSE')
plt.legend()
plt.subplot(122)
plt.plot(range(1, len(accuracies) + 1), accuracies, label='Accuracy', marker='o')
plt.axvline(x=np.argmax(accuracies)+1, color='red', linestyle='--', label=f'Best
Iteration: {np.argmax(accuracies)}')
```

```
plt.xlabel('Number of Iterations')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```




## warm_start

```
gbclf = GradientBoostingClassifier(max_depth=2,
                                   random_state=42,
                                   warm_start=True
                                   )
error_going_up = 0
max_accuracy = 0
# min_val_error = float('inf')
for n_estimators in range(1, 120):
    gbclf.n_estimators = n_estimators
    gbclf.fit(X_train, y_train)
    y_pred = gbclf.predict(X_val)
    accuracy = accuracy_score(y_val, y_pred)
    if accuracy > max_accuracy:
        max_accuracy = accuracy
        error_going_up = 0
    else:
        error_going_up += 1
        if error_going_up == 30:
            # 至少有 30 次没有新的 MAX(accuracy)才算到了 max。但是 30 次这个间隔不一定好
            break
print(gbclf.n_estimators-30)  # 23
```

warm_start 能在每次 fit 时从上一次的训练开始继续训练。

## Stacking 堆叠集成

```
import numpy as np
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.ensemble import RandomForestClassifier, ExtraTreesClassifier
from sklearn.svm import LinearSVC
from sklearn.neural_network import MLPClassifier

mnist = fetch_openml('MNIST_784', parser='auto')
# 首先将 MNIST 数据集(包括图像和标签)分成训练验证集和测试集 test。接着划分训练验证集得到训练集 train 和验证集 val。
X_train_val, X_test, y_train_val, y_test = train_test_split(mnist.data,
```

```
mnist.target, test_size=10000, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val,
test_size=10000, random_state=42)

random_forest_clf = RandomForestClassifier(random_state=42)
extra_trees_clf = ExtraTreesClassifier(random_state=42)
svm_clf = LinearSVC(random_state=42, dual=True, max_iter=2000)
mlp_clf = MLPClassifier(random_state=42)
estimators = [random_forest_clf, extra_trees_clf, svm_clf, mlp_clf]

for estimator in estimators:
    print("Training:", estimator)
    estimator.fit(X_train, y_train)

X_val_pred = np.empty((len(X_val), len(estimators)), dtype=np.float32)
for index, estimator in enumerate(estimators):
    X_val_pred[:, index] = estimator.predict(X_val)  # 对手写数字验证集的预测，每行是 4
个模型的对应预测

rdf = RandomForestClassifier(n_estimators=200, oob_score=True, random_state=42)
rdf.fit(X_val_pred, y_val)
print(rdf.oob_score_)  # 0.9663

X_test_pred = np.empty((len(X_test), len(estimators)), dtype=np.float32)
for index, estimator in enumerate(estimators):
    X_test_pred[:, index] = estimator.predict(X_test)
y_test_pred = rdf.predict(X_test_pred)
stacking_accuracy = accuracy_score(y_test, y_test_pred)
print(f"Stacking Model Accuracy: {stacking_accuracy}")  # 0.9664
```

**性能比较**

```
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.feature_extraction.text import TfidfTransformer
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import VotingClassifier

categories = ['alt.atheism', 'soc.religion.christian', 'comp.graphics', 'sci.med']
twenty_train = fetch_20newsgroups(subset='train', categories=categories,
shuffle=True, random_state=42)
count_vect = CountVectorizer()
X_train_counts = count_vect.fit_transform(twenty_train.data)
tfidf_transformer = TfidfTransformer()
X_train_tfidf = tfidf_transformer.fit_transform(X_train_counts)
twenty_test = fetch_20newsgroups(subset='test', categories=categories,
shuffle=True, random_state=42)
X_test_counts = count_vect.transform(twenty_test.data)
X_test_tfidf = tfidf_transformer.transform(X_test_counts)


def Vot_clf(voting_type):
    log_clf = LogisticRegression(penalty='l2', solver='newton-cg')
    tree_clf = DecisionTreeClassifier(criterion='entropy')
    svm_clf = SVC(kernel='linear', probability=True)
    return VotingClassifier([('lr', log_clf), ('tree', tree_clf), ('svc', svm_clf)],
voting=voting_type, weights=[2, 1, 3])


def Acc_of_clf(voting_type):
    log_clf = LogisticRegression()
    tree_clf = DecisionTreeClassifier(criterion='entropy')
```

```
    svm_clf = SVC(kernel='linear', probability=True)
    vot_clf = Vot_clf(voting_type)
    for clf in (log_clf, tree_clf, svm_clf, vot_clf):
        clf.fit(X_train_tfidf, twenty_train.target)
        print(clf.__class__.__name__, "准确率%.4f" % (clf.score(X_test_tfidf,
twenty_test.target)))


Acc_of_clf('soft')
```

输出

```
LogisticRegression 准确率 0.8975
DecisionTreeClassifier 准确率 0.6897
SVC 准确率 0.9208
VotingClassifier 准确率 0.9288
```

用于比较逻辑回归、决策树、支持向量机和集成投票分类器在 20news 文本数据集上的性能。

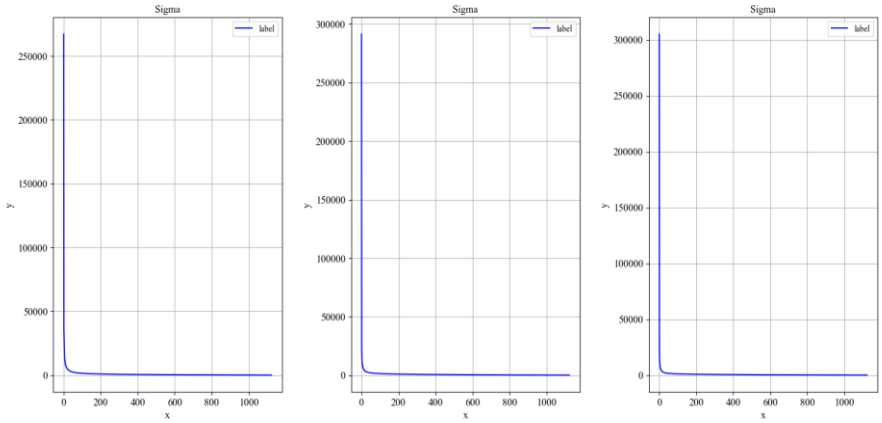
## 12.线性代数

### SVD 实现图片压缩里的颜色细节

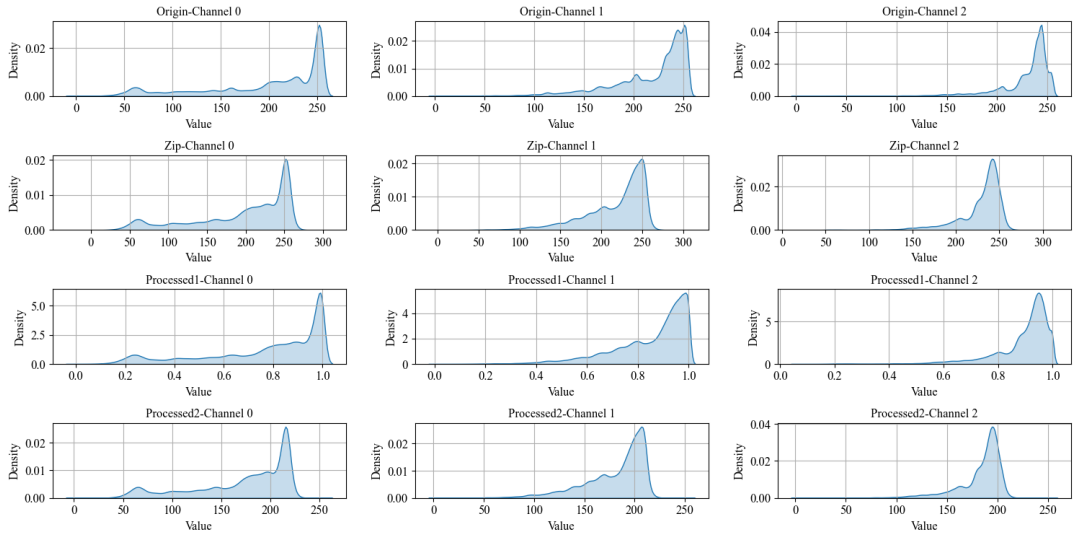
```
import cv2
import numpy as np
import matplotlib.pyplot as plt

from easier_excel.draw_data import draw_density, plot_xy

def read_image(img_path, gray_pic=False, show_details=False):
    """
    读取图片
    :param img_path: 图像路径
    :param gray_pic: 是否读取灰度图像
    :param show_details: 是否输出图片的 shape 以及显示图片
    :return: 图像数组，类型为 np.ndarray
    """
    if gray_pic:
        img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    else:
        img_gbr = cv2.imread(img_path)
        img = cv2.cvtColor(img_gbr, cv2.COLOR_BGR2RGB)
    if show_details:
        print(img.shape)
        plt.imshow(img)
        plt.show()
        plt.close()
    return img


def zip_image_by_svd(origin_image, rate=0.8, channel=3, show_img=True):
    """
    使用 SVD 对图像进行压缩
    :param origin_image: 传入 np.ndarray 类型的图像数组
    :param rate: 保留率
    :param channel: 通道数
    :param show_img: 是否显示压缩前后的图片
    :return zip_img: 压缩后的图像
    """
    zip_img = np.zeros(origin_image.shape)  # 用于存储压缩后的图像
    u_shape, s_shape, vT_shape, n_sigmas = 0, 0, 0, 0

    fig, axs = plt.subplots(1, 3)
    for chan in range(channel):
        # 对每层进行 SVD 分解
        U, Sigma, VT = np.linalg.svd(origin_image[:, :, chan])
        # 计算达到保留率需要的奇异值数量
        total_Sigma = np.sum(Sigma)
        cum_Sigma = np.cumsum(Sigma)
        n_sigmas = np.argmax(cum_Sigma >= rate * total_Sigma) + 1
        Sigma_n = np.diag(Sigma[:n_sigmas])
        zip_img[:, :, chan] = np.dot(U[:, :n_sigmas], np.dot(Sigma_n, VT[:n_sigmas, :]))
        # 记录每个矩阵的 shape
        u_shape = U[:, 0:n_sigmas].shape
        s_shape = Sigma_n.shape
        vT_shape = VT[0:n_sigmas, :].shape
        plot_xy(x=np.arange(Sigma.shape[0]), y=Sigma, title='Sigma', show_plt=False, ax=axs[chan],
use_ax=True,
                font_name='Times New Roman')
    plt.show()
    plt.close()

    fig, axs = plt.subplots(4, 3)
    for i in range(channel):
        axs[0][i] = draw_density(origin_image[:, :, i].reshape(-1), ax=axs[0][i], show_plt=False,
                                 title=f'Origin-Channel {i}', show_legend=False)
    for i in range(channel):
        axs[1][i] = draw_density(zip_img[:, :, i].reshape(-1), ax=axs[1][i], show_plt=False,
                                 title=f'Zip-Channel {i}', show_legend=False)

    # 这里暂时没想到更好的方法，应该是让 zip_img 更接近 origin_image 的值，使得颜色差异小
    # 如果使用 zip_img 就是归一化到[0, 1]，但这里使用 origin_image 的最大最小值来进行归一化，是为了减少颜色差
异
    # 使用 zip_img[:, :, i] = (O_MAX-O_MIN) * (zip_img[:, :, i] - Z_MIN) / (Z_MAX - Z_MIN) + O_MIN
    # 可能不如只使用 O_MAX 和 O_MIN，猜测是 Z_MIN 和 Z_MAX 的值是离群点
    zip_img1 = np.zeros(zip_img.shape)
```

```
    zip_img2 = np.zeros(zip_img.shape)
    for i in range(channel):
        O_MAX = np.max(origin_image[:, :, i])
        O_MIN = np.min(origin_image[:, :, i])
        Z_MIN = np.min(zip_img[:, :, i])
        Z_MAX = np.max(zip_img[:, :, i])
        zip_img1[:, :, i] = (zip_img[:, :, i] - O_MIN) / (O_MAX - O_MIN)
        # print("min-max:",  np.min(zip_img[:, :, i]),  np.max(zip_img[:, :, i]))
    zip_img1[zip_img1 < 0] = 0
    zip_img1[zip_img1 > 1] = 1
    for i in range(channel):
        O_MAX = np.max(origin_image[:, :, i])
        O_MIN = np.min(origin_image[:, :, i])
        Z_MIN = np.min(zip_img[:, :, i])
        Z_MAX = np.max(zip_img[:, :, i])
        zip_img2[:, :, i] = (O_MAX-O_MIN) * (zip_img[:, :, i] - Z_MIN) / (Z_MAX - Z_MIN) + O_MIN

    for i in range(channel):
        axs[2][i] = draw_density(zip_img1[:, :, i].reshape(-1), ax=axs[2][i], show_plt=False,
                                 title=f'Processed1-Channel {i}', show_legend=False)
    for i in range(channel):
        axs[3][i] = draw_density(zip_img2[:, :, i].reshape(-1), ax=axs[3][i], show_plt=False,
                                 title=f'Processed2-Channel {i}', show_legend=False)
    plt.show()
    plt.close()

    # # 因为数据支持 RGB data ([0..1] for floats or [0..255] for integers，所以不一定需要调整到[0, 255]
    # zip_img = np.round(zip_img * 255).astype('int')

    # 计算压缩率
    zip_rate = (origin_image.size - 3 * (u_shape[0] * u_shape[1] + s_shape[0] * s_shape[1] +
vT_shape[0] * vT_shape[1])) \
        / origin_image.size

    print(f"保留率:          {rate * 100:.1f}%")
    print(f"选择的奇异值数量: {n_sigmas}--->原来的奇异值数量: {Sigma.shape[0]}")
    print(f"原图 Shape:       {origin_image.shape}--->Size: {origin_image.size}", )
    print(f"压缩后的矩阵大小: {u_shape} , {s_shape} , {vT_shape}")
    print(f"压缩率为:        {zip_rate * 100:.3f}%")
    if show_img:
        fig, axes = plt.subplots(1, 2)
        if channel == 1:
            axes[0].imshow(origin_image[:, :, 0], cmap='gray')
            axes[1].imshow(zip_img[:, :, 0], cmap='gray')
        else:
            axes[0].imshow(origin_image)
            axes[1].imshow(zip_img)
        axes[0].set_title('Before SVD')
        axes[1].set_title(f'After SVD with rate={zip_rate * 100:.3f}% and n_sigmas={n_sigmas}')
        plt.show()
    return zip_img


path = '../input/arona.jpg'
img_gray = read_image(path, gray_pic=False, show_details=False)
# img_gray = img_gray.reshape((img_gray.shape[0], img_gray.shape[1], 1))
zip_image_by_svd(img_gray, rate=0.6, channel=3)
```

图片的奇异值的大小趋势：



两种压缩后的处理方式是①映射到与原始图像差不多的值域，剩余部分取 0 或 1②映射到[0,255]。因为 SVD 分解时存在离群值，因此①更优：



## 13.神经网络

### sklearn 神经网络

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

Iris = load_iris()
X = Iris.data
Y = Iris.target
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.3)
model = MLPClassifier(max_iter=1000)
model.fit(x_train, y_train)
print("神经网络模型训练集的准确率：%.3f" % model.score(x_train, y_train))
print("神经网络模型测试集的准确率：%.3f" % model.score(x_test, y_test))
```

输出

```
神经网络模型训练集的准确率：0.971
神经网络模型测试集的准确率：0.978
```

### torch 神经网络

### 绘制激活函数与其导数

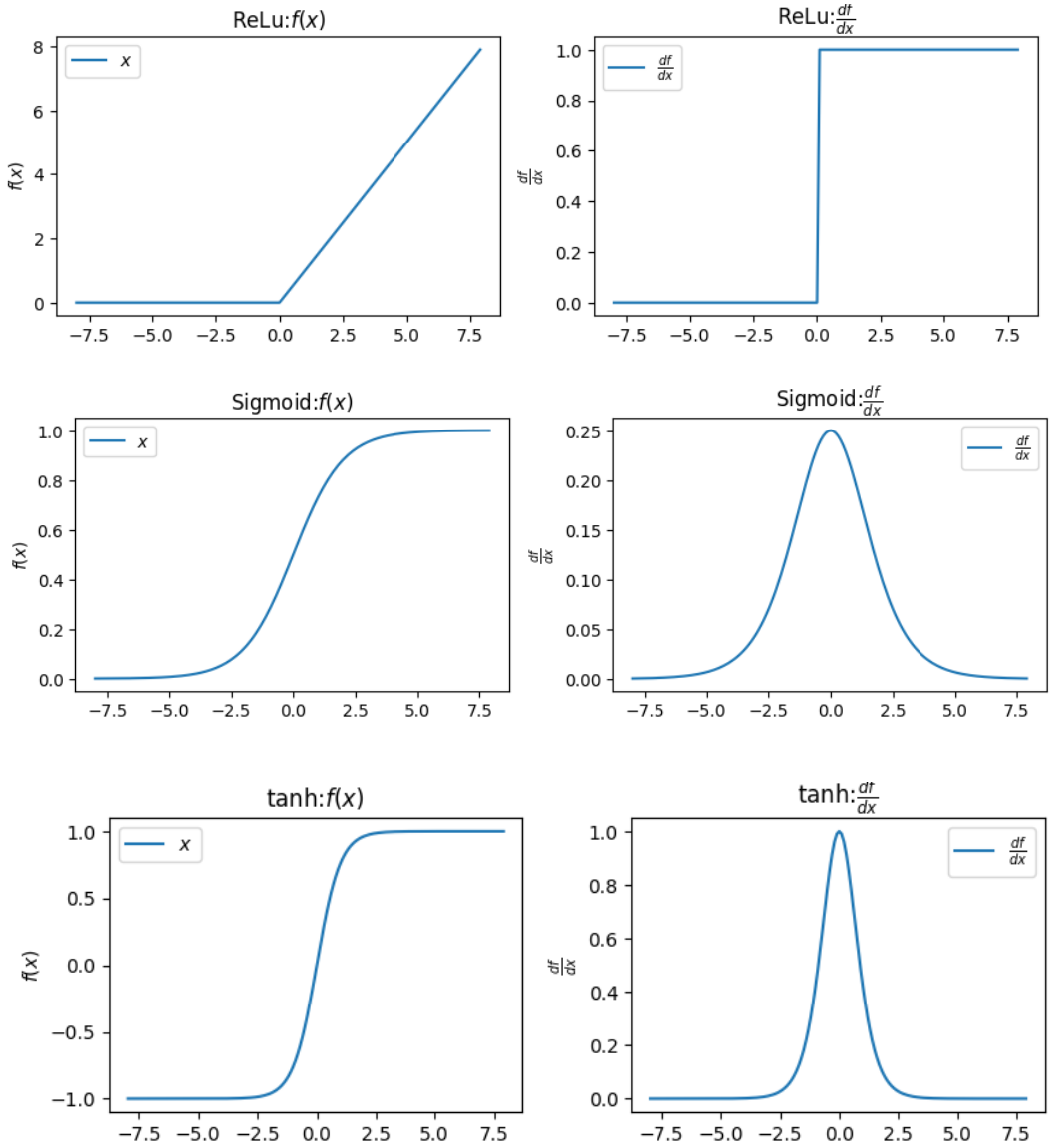
```
import torch
import xm_draw

x = torch.arange(-8.0, 8.0, 0.1, requires_grad=True)
y1 = torch.relu(x)
y2 = torch.sigmoid(x)
y3 = torch.tanh(x)

xm_draw.draw_f_and_df(x, y1, funcName_in_title="ReLu")
xm_draw.draw_f_and_df(x, y2, funcName_in_title="Sigmoid")
xm_draw.draw_f_and_df(x, y3, funcName_in_title="tanh")
```

用到的函数如下：

```
def draw_f_and_df(x, y, draw_f=True, draw_df=True, separate=True, funcName_in_title=""):
    """绘制函数与其导数，要求输入的x,y 的requires_grad=True"""
    x_latex = r'$x$'
    fx_latex = r'$f(x)$'
    dfx_latex = r'$\frac{df}{dx}$'
    if separate:
        fig, ax = plt.subplots(1, 2)
        if draw_f:
            ax[0].plot(x.detach(), y.detach(), label=x_latex)
            ax[0].set_xlabel(x_latex)
            ax[0].set_ylabel(fx_latex)
            ax[0].set_title(f"{funcName_in_title + ':' if funcName_in_title else ''}{fx_latex}")
            ax[0].legend()
        if draw_df:
            y.backward(torch.ones_like(x), retain_graph=True)
            dy = x.grad
            ax[1].plot(x.detach(), dy.detach(), label=dfx_latex)
            ax[1].set_xlabel(x_latex)
            ax[1].set_ylabel(dfx_latex)
            ax[1].set_title(f"{funcName_in_title + ':' if funcName_in_title else ''}{dfx_latex}")
            ax[1].legend()
        plt.show()
        plt.close()
    else:
        fig, ax = plt.subplots(1, 1)
        y_latex_temp, title_latex_temp = '', ''
        if draw_f:
            ax.plot(x.detach(), y.detach(), label=fx_latex)
            y_latex_temp += fx_latex
            title_latex_temp += fx_latex
        if draw_df:
            y.backward(torch.ones_like(x), retain_graph=True)
            dy = x.grad
            y_latex_temp += dfx_latex
            title_latex_temp += "and" if draw_f else "" + dfx_latex
            ax.plot(x.detach(), dy.detach(), label=dfx_latex)
```

```
        ax.set_xlabel(x_latex)
        ax.set_ylabel(y_latex_temp)
        ax.set_title(f"{funcName_in_title + ':' if funcName_in_title else ''}{title_latex_temp}")
        ax.legend()
        plt.show()
        plt.close()
    x.grad.data.zero_()  # 防止对同一个 x 多次调用此函数
```

函数图像如下：



**自己&api 实现线性回归**

```
import torch
import matplotlib.pyplot as plt
import xm_func


def synthetic_data(w, b, num_examples):
    """生成y=Xw+b+噪声"""
    X = torch.normal(0, 1, (num_examples, len(w)))  # 大小(num_examples, len(w))的张量 X，X 的元素服从 N(0, 1)
    y = xm_func.linreg(X, w, b)
    y += torch.normal(0, 0.01, y.shape)
    return X, y.reshape((-1, 1))

true_w = torch.tensor([2, -3.4])
true_b = 4.2
features, labels = synthetic_data(true_w, true_b, 1000)

print('features:', features[0], '\nlabel:', labels[0])  # 比如 features: tensor([-2.2002,  0.1260])  label: tensor([-0.6346])
plt.scatter(features[:, 1].detach().numpy(), labels.detach().numpy(), 1)
# .detach()创建张量副本。这里 x 轴选取 features 第二列，y 选取 labels
plt.show()

# 初始化
w = torch.normal(0, 0.01, size=(2, 1), requires_grad=True)  # 比如[[-0.0193], [ 0.0127]]
b = torch.zeros(1, requires_grad=True)  # [0.]
print(w, b)

# 超参数
lr = 0.03  # 学习率
num_epochs = 30  # 迭代周期
batch_size = 10  # 每个小批量样本的数量
net = xm_func.linreg  # 线性回归模型
loss = xm_func.squared_loss  # 均方损失函数

for epoch in range(num_epochs):
    # 控制训练的轮数
    for X, y in xm_func.data_iter(batch_size, features, labels):
        # 迭代获取小批量样本
        l = loss(net(X, w, b), y)  # 计算模型的损失。l 的大小是(batch_size,1)，而不是一个标量
        l.sum().backward()  # 计算关于[w,b]的梯度
        xm_func.sgd([w, b], lr, batch_size)  # 使用[w,b]的梯度更新[w,b]
    with torch.no_grad():
        train_l = loss(net(features, w, b), labels)
        print(f'epoch {epoch + 1}, loss {float(train_l.mean()):f}, w {w[0].item():.2f},{w[1].item():.2f}, b {b.item():.2f}')

print(f'w 的估计误差: {true_w - w.reshape(true_w.shape)}')
print(f'b 的估计误差: {true_b - b}')
```

用到的 xm_func 模块：

```
def linreg(X, w, b):
    # mm 要求输入矩阵的维度必须匹配，matmul 支持广播操作。
    return torch.matmul(X, w) + b

def squared_loss(y_hat, y):
    return (y_hat - y.reshape(y_hat.shape)) ** 2 / 2

def data_iter(batch_size, features, labels):
    num_examples = len(features)  # 数据集的长度
```

```
    indices = list(range(num_examples))  # 创建一个包含数据集索引的列表
    random.shuffle(indices)  # 将数据集索引随机打乱
    for i in range(0, num_examples, batch_size):
        batch_indices = torch.tensor(indices[i: min(i + batch_size, num_examples)])
# 从打乱的索引列表中取出一个批次的索引
        yield features[batch_indices], labels[batch_indices]  # 取出对应的批量数据
        # yield 不会终止函数的执行，而是会暂时挂起函数的执行，并返回一个值，保持函数的状态。每次调用函数时，都会从上次挂起的地方继续执行，直到遇到下一个 yield 语句。这使得函数可以产生一个序列的值(让函数生成一个数据流)，而不是单个值。

def sgd(params, lr, batch_size):
    with torch.no_grad():
        # with torch.no_grad()：在该模块下，所有计算得出的 tensor 的 requires_grad 都自动设置为 False。
        for param in params:
            param -= lr * param.grad / batch_size  # 更新参数（使用-=运算符进行原地操作in-place，所以修改的是全局变量）
            param.grad.zero_()  # 梯度清零(为下一个小批量样本做准备)
```

上面只用了张量来进行数据存储和线性代数以及通过自动微分来计算梯度，下面用 api 来实现（feature、labels 和超参数不变）：

```
from torch import nn  # 神经网络
net = nn.Sequential(nn.Linear(2, 1))  # 流水线里只有一个线性回归模型(输入特征有 2 个，输出为 1 个)
net[0].weight.data.normal_(0, 0.01)  # 用 N(0, 0.01^2)的正态分布来初始化权重。net[0]是第一个层(Linear)。weight 是该层的权重，data 表示获取权重的数据。
net[0].bias.data.fill_(0)  # bias 是该层的偏置，data 表示获取偏置的数据。fill_(0) 表示将偏置的值填充为 0
loss = nn.MSELoss()  # 均方损失函数(如果指定 reduction='sum'，会使得梯度值放大为原来的 num_example 倍，容易在最优解附近震荡)。还可以换成 nn.HuberLoss()等
trainer = torch.optim.SGD(net.parameters(), lr=lr)
data_iter = xm_func.load_array((features, labels), batch_size)  # 可以用 next(iter(data_iter))来查看数据

for epoch in range(num_epochs):
    for X, y in data_iter:
        l = loss(net(X), y)  # 这里的 net 返回输入 x 经过定义的网络所计算出的值
        trainer.zero_grad()  # 清除上一次的梯度值
        l.backward()  # 反向传播，求参数的梯度
        trainer.step()  # 步进 根据指定的优化算法进行参数的寻优
    l = loss(net(features), labels)
    print(f'epoch {epoch + 1}, loss {l:f}')
w = net[0].weight.data
print('w 的估计误差：', true_w - w.reshape(true_w.shape))
b = net[0].bias.data
print('b 的估计误差：', true_b - b)
```

在反向传播后可以这样查看梯度值：

```
for param in net.parameters():
    print(param.grad)
```

线性回归与 softmax 的比较：

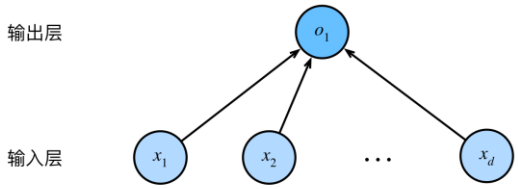



图3.1.2: 线性回归是一个单层神经网络。

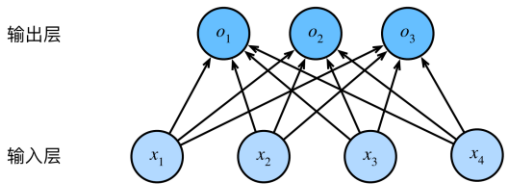



图3.4.1: softmax回归是一种单层神经网络

## 自己&api 实现 softmax 回归

### 读入数据集

```
import torch
import torchvision
import xm_func
from matplotlib import pyplot as plt
from torch.utils import data
from torchvision import transforms


# ToTensor 实例将图像数据从 PIL 类型变换成 32 位浮点数格式，并进行归一化处理(除以 255 使得所有像素的数值均在 0～1 之间)
trans = transforms.ToTensor()
mnist_train = torchvision.datasets.FashionMNIST(root="../data", train=True, transform=trans, download=True)
mnist_test = torchvision.datasets.FashionMNIST(root="../data", train=False, transform=trans, download=True)
print(len(mnist_train), len(mnist_test))  # 60000 10000
print(mnist_train[0][0].shape)  # torch.Size([1, 28, 28])

def get_fashion_mnist_labels(labels):
    """输入数值，返回文本标签：t-shirt T 恤、trouser 裤子、pullover 套衫、dress 连衣裙、coat 外套、sandal 凉鞋、shirt 衬衫、sneaker 运动鞋、bag 包、ankle boot 短靴"""
    text_labels = ['t-shirt', 'trouser', 'pullover', 'dress', 'coat', 'sandal', 'shirt', 'sneaker', 'bag', 'ankle boot']
    return [text_labels[i] for i in labels]

def show_images(imgs, num_rows, num_cols, titles=None):
    """绘制图像列表"""
    _, axes = plt.subplots(num_rows, num_cols)
    axes = axes.flatten()  # 以 num_rows=3,num_cols=6 为例，flatten 将(3, 6)的 axes 转换成(18,)，这样更方便使用单个索引来访问每个子图轴对象
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        # 通过对当前的子图轴对象 ax 的操作直接影响了 axes 中相应位置的元素
        if torch.is_tensor(img):
            # 图片张量
            ax.imshow(img.numpy())
        else:
            # PIL 图片
            ax.imshow(img)
            ax.axes.get_xaxis().set_visible(False)
            ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes

# 输出部分图片
X, y = next(iter(data.DataLoader(mnist_train, batch_size=18)))
show_images(X.reshape(18, 28, 28), 3, 6, titles=get_fashion_mnist_labels(y))
plt.show()
plt.close()

# 测试读入数据
batch_size = 256
train_iter = data.DataLoader(mnist_train, batch_size, shuffle=True)
timer = xm_func.Timer()
timer.start()
for X, y in train_iter:
    continue
print(f'{batch_size}--{timer.stop():.2f} sec')  # 5.20sec 左右

def load_data_fashion_mnist(batch_size, resize=None):
    """使用已经下载了的Fashion-MNIST 数据集(将其加载到内存中)"""
    trans = [transforms.ToTensor()]  # 加上[]是为了便于后续的列表操作
    if resize:
        trans.insert(0, transforms.Resize(resize))  # 将 resize 变换插入到变成 tensor 之前
    trans = transforms.Compose(trans)  # 把这个变换列表 trans 按顺序组合成一个整体的变换操作。
    mnist_train = torchvision.datasets.FashionMNIST(root="../data", train=True, transform=trans)
    mnist_test = torchvision.datasets.FashionMNIST(root="../data", train=False, transform=trans)
    return data.DataLoader(mnist_train, batch_size, shuffle=True), data.DataLoader(mnist_test, batch_size, shuffle=False)

train_iter, test_iter = load_data_fashion_mnist(32, resize=64)
for X, y in train_iter:
    print(X.shape, X.dtype, y.shape, y.dtype)  # torch.Size([32, 1, 64, 64]) torch.float32
```

```
torch.Size([32]) torch.int64
    break
```

**手动(部分)**

```
def softmax(X):
    """softmax"""
    X_exp = torch.exp(X)  # 形状(n, D)，n 为样本数量(如 batch_size),D 即类别数量
    partition = X_exp.sum(1, keepdim=True)  # 形状(n, 1)
    return X_exp / partition  # 广播机制

def softmax_net(X):
    """softmax 回归模型"""
    return softmax(torch.matmul(X.reshape((-1, W.shape[0])), W) + b)

def cross_entropy(y_hat, y):
    """交叉熵损失"""
    return - torch.log(y_hat[range(len(y_hat)), y])  # range(len(y_hat))遍历每个样本，y 是对应样本的正确类别
```

**API**

```
from torch import nn
from xm_draw import draw_LossAndAccuracy
from xm_func import evaluate_accuracy, bool_accuracy

batch_size = 256
train_iter, test_iter = load_data_fashion_mnist(batch_size)

# PyTorch 不会隐式地调整输入的形状。因此在线性层前定义了展平层（flatten），来调整网络输入的形状
net = nn.Sequential(nn.Flatten(), nn.Linear(784, 10))
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, mean=0, std=0.01)
net.apply(init_weights)  # 对 Flatten,Linear,Sequential 依次 init_weights
loss = nn.CrossEntropyLoss(reduction='none')
trainer = torch.optim.SGD(net.parameters(), lr=0.1)
num_epochs = 10

def train(net, train_iter, test_iter, loss, trainer, num_epochs):
    train_loss_values = []
    train_acc_values = []
    test_acc_values = []
    for epoch in range(num_epochs):
        train_loss_sum, train_acc_sum, n = 0.0, 0.0, 0
        for X, y in train_iter:
            trainer.zero_grad()
            y_hat = net(X)
            l = loss(y_hat, y)
            l.sum().backward()
            trainer.step()
            n += y.shape[0]
            train_loss_sum += l.sum().item()
            train_acc_sum += bool_accuracy(y_hat, y)
        print(f"epoch:{epoch}, 样本数 n:{n}")
        train_loss_values.append(train_loss_sum / n)
        train_acc_values.append(train_acc_sum / n)
        test_acc_values.append(evaluate_accuracy(net, test_iter))
    return train_loss_values, train_acc_values, test_acc_values

train_loss_values, train_acc_values, test_acc_values = train(net, train_iter,
test_iter, loss, trainer, num_epochs)
draw_LossAndAccuracy(y=[train_loss_values, train_acc_values, test_acc_values],
epochs=num_epochs)
```

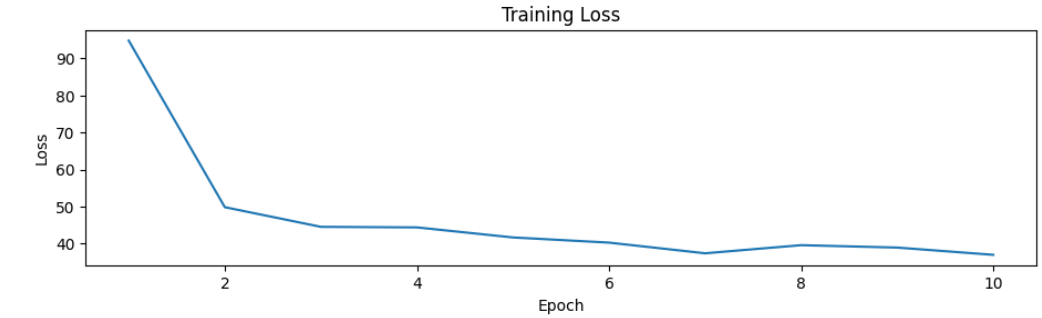
其中用到：

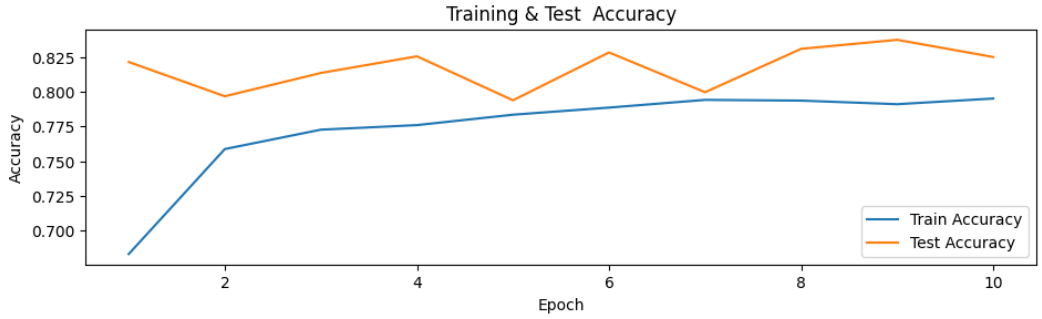
```python
def draw_LossAndAccuracy(y, epochs, x=None):
    """传入y=[train_loss_values, train_acc_values, test_acc_values]和epochs的值，绘制Loss和Accuracy"""
    epochs = range(1, epochs + 1)
    fig, ax = plt.subplots(2, 1, figsize=(10, 7))

    ax[0].plot(epochs, y[0], label='Train Loss')
    ax[0].set_xlabel('Epoch')
    ax[0].set_ylabel('Loss')
    ax[0].set_title('Training Loss')

    ax[1].plot(epochs, y[1], label='Train Accuracy')
    ax[1].plot(epochs, y[2], label='Test Accuracy')
    ax[1].set_xlabel('Epoch')
    ax[1].set_ylabel('Accuracy')
    ax[1].set_title('Training & Test  Accuracy')
    ax[1].legend()

    plt.show()
    plt.close()

def evaluate_accuracy(net, data_iter):
    """计算分类模型的准确率"""
    if isinstance(net, torch.nn.Module):
        net.eval()  # 将模型设置为评估模式
    accuracy_sum, n = 0, 0
    with torch.no_grad():
        for X, y in data_iter:
            accuracy_sum += bool_accuracy(net(X), y)
            n += y.numel()
    return accuracy_sum / n


def bool_accuracy(y_hat, y):
    """计算分类正确的数量"""
    if len(y_hat.shape) > 1 and y_hat.shape[1] > 1:
        cmp = y_hat.argmax(axis=1).type(y.dtype) == y
        return float(cmp.sum())
```

还可以进行预测：

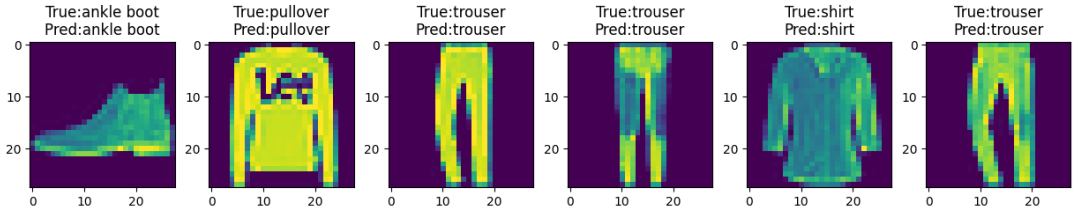
```python
def predict(net, test_iter, n=6):
    """对前n个验证集样本进行预测"""
    X, y = next(iter(test_iter))
    trues = get_fashion_mnist_labels(y)
    preds = get_fashion_mnist_labels(net(X).argmax(axis=1))
    titles = ["True:" + true + '\n' + "Pred:" + pred for true, pred in zip(trues, preds)]
    show_images(X[0:n].reshape((n, 28, 28)), 1, n, titles=titles[0:n])

predict(net, test_iter, n=6)
plt.show()
plt.close()
```

训练结果：




预测结果：




**多层感知机的实现：**

```
net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784, 256),
                    nn.ReLU(),
                    nn.Linear(256, 10))
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, mean=0, std=0.01)
net.apply(init_weights)
```

其中 net 的使用方法可以参考 softmax 的 API 实现中的过程。

但建议用 tanh 作为激活函数，这里用 relu 会让 loss 值很小而准确率极低(11%)，原因不知。

另外 nn.Linear 的 weight 和 bias 会自动初始化，这里使用 init_weight 只是不希望采用对 weight 的默认初始化，对 bias 则采用默认初始化。

**多项式回归的实现：**

```
import numpy as np
import torch
import math
import matplotlib.pylab as plt
from xm_func import evaluate_loss, load_array
from torch import nn
from xm_draw import draw_LossAndAccuracy

max_degree = 20  # 多项式的最大阶数
n_train, n_test = 100, 100  # 训练和测试数据集大小
```

```
true_w = np.zeros(max_degree)
true_w[0:4] = np.array([5, 1.2, -3.4, 5.6])
# 生成一个维度为(n_train + n_test, 1)的随机正态分布样本特征矩阵
features = np.random.normal(size=(n_train + n_test, 1))
np.random.shuffle(features)
# 对于 features 的每一行的那个元素 x，进行 x^0, x, x^2,..., x^(max_degree-1)，生成维度为(n_train + n_test,
max_degree)的多项式特征矩阵
# .reshape(1, -1)让维度从(max_degree,)变成(1, max_degree)利于广播原则（不 reshape 其实也行）
poly_features = np.power(features, np.arange(max_degree).reshape(1, -1))
for i in range(max_degree):
    poly_features[:, i] /= math.gamma(i + 1)  # 通过 gamma(n)=(n-1)!标准化，避免非常大的梯度值或损失值
# labels 维度(n_train+n_test,)
labels = np.dot(poly_features, true_w)
labels += np.random.normal(scale=0.1, size=labels.shape)  # 自定义误差
# ndarray 转换为 tensor
true_w, features, poly_features, labels = [torch.tensor(x, dtype=torch.float32) for x in [true_w,
features, poly_features, labels]]

def train(train_features, test_features, train_labels, test_labels, num_epochs=200):
    loss = nn.MSELoss(reduction='mean')
    input_shape = train_features.shape[-1]  # 以 x^0~x^3 为例，train_features 大小是[100, 4]，[-1]取出的
是 4(也就是多少次幂)
    net = nn.Sequential(nn.Linear(input_shape, 1, bias=False))  # 不设置偏置，因为已经在多项式中实现了它
    batch_size = 10
    train_iter = load_array((train_features, train_labels.reshape(-1, 1)), batch_size)
    test_iter = load_array((test_features, test_labels.reshape(-1, 1)), batch_size, is_train=False)
    trainer = torch.optim.SGD(net.parameters(), lr=0.01)

    train_loss_values = []
    train_acc_values = []
    test_acc_values = []
    for epoch in range(num_epochs):
        train_loss_sum, train_acc_sum, n = 0.0, 0.0, 0
        for X, y in train_iter:
            trainer.zero_grad()
            y_hat = net(X)
            l = loss(y_hat, y)
            l.sum().backward()
            trainer.step()
            n += y.shape[0]
            with torch.no_grad():
                train_loss_sum += l.sum().item()
                train_acc_sum += loss(y_hat, y)
        if epoch % 100 == 0:
            print(f"epoch:{epoch}, 样本数 n:{n}")
        train_loss_values.append(train_loss_sum / n)
        train_acc_values.append(train_acc_sum / n)
        test_acc_values.append(evaluate_loss(net, test_iter, loss))
    print('weight:', net[0].weight.data.numpy())
    print('loss:', train_loss_values[-1])
    return train_loss_values, train_acc_values, test_acc_values, net

# 从多项式特征中选择前 4 个维度，即 1,x,x^2/2!,x^3/3!
train_loss_values, train_acc_values, test_acc_values, net1 = \
    train(poly_features[:n_train, :4], poly_features[n_train:, :4], labels[:n_train],
labels[n_train:])
draw_LossAndAccuracy([train_loss_values, train_acc_values, test_acc_values], epochs=200,
loss_log=True, Accuracy_log=True)
train_loss_values, train_acc_values, test_acc_values, net2 = \
    train(poly_features[:n_train, :2], poly_features[n_train:, :2], labels[:n_train],
labels[n_train:])
draw_LossAndAccuracy([train_loss_values, train_acc_values, test_acc_values], epochs=200,
loss_log=True, Accuracy_log=True)
train_loss_values, train_acc_values, test_acc_values, net3 = \
    train(poly_features[:n_train, :], poly_features[n_train:, :], labels[:n_train], labels[n_train:],
num_epochs=1500)
draw_LossAndAccuracy([train_loss_values, train_acc_values, test_acc_values], epochs=1500,
loss_log=True, Accuracy_log=True)

# 绘制 y 和 y_hat
for X, net in zip([poly_features[:, :4], poly_features[:, :2], poly_features[:, :]], [net1, net2,
net3]):
    with torch.no_grad():
        x = features
        y = np.dot(poly_features, true_w)
        y_hat = net(X)
    f, ax = plt.subplots()
```

```
    ax.scatter(x, y, label='y', color='black')
    ax.scatter(x, y_hat, label='y_hat', color='red')
    plt.show()
    plt.close()
```

用到的模块（其他的函数同之前的代码）：

```
def evaluate_loss(net, data_iter, loss):
    """计算回归模型的损失"""
    loss_sum, n = 0, 0
    with torch.no_grad():
        for X, y in data_iter:
            out = net(X)
            y = y.reshape(out.shape)
            l = loss(out, y)
            loss_sum += l.sum()
            n += l.numel()
    return loss_sum / n
```

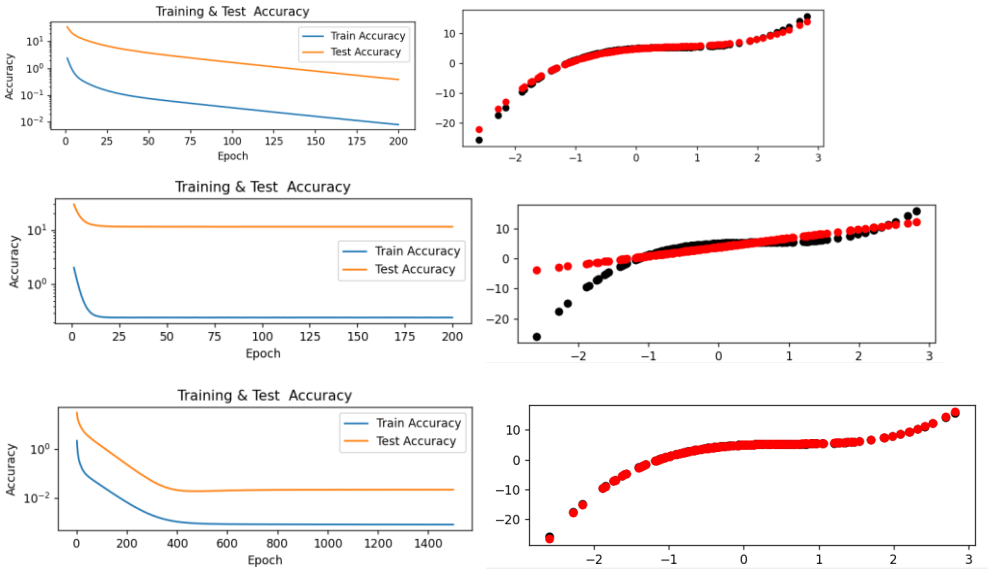
**训练数据：**

weight: [[ 4.859618  1.6995777 -2.9753466  4.3367476]]  loss: 0.007932368274778128

weight: [[3.7808843 2.954084 ]]  loss: 0.24653663396835326

weight: [[ 4.976041  1.281252  -3.3253098  5.287895  -0.30464238  1.139414  0.2866924  0.3230021  0.14926858 -0.13491264 -0.06434472  0.01017338
-0.17069647 -0.17043759 -0.17870218 -0.08004908 -0.10076946  0.20569256  0.09011322 -0.19666234]]  loss: 0.0007883880054578186

仅展示部分图片（这里的 Accuracy 用 loss 来代替）：




**为什么没有过拟合的感觉？因为大部分 feature 都在 0 附近，高阶项很小。**

生成数据集的时候还可以只用 tensor：

```
max_degree = 20
n_train, n_test = 100, 100
true_w = torch.zeros(max_degree)
true_w[:4] = torch.tensor([5, 1.2, -3.4, 5.6])
features = torch.randn((n_train + n_test, 1))
features = features[torch.randperm(len(features))]
poly_features = torch.pow(features, torch.arange(max_degree))
poly_features /= torch.tensor([math.gamma(i + 1) for i in range(max_degree)])
labels = poly_features @ true_w
labels += torch.randn(labels.shape) * 0.1
labels = labels.reshape(-1, 1)
```

**还可以直接算出解析解：**

```
w = torch.linalg.inv(poly_features[:, :4].t() @ poly_features[:, :4]) @
poly_features[:, :4].t() @ labels.reshape(200, 1)
```

某次得到的是 tensor([[ 5.0035], [ 1.1972], [-3.4066], [ 5.6050]])。原理是：对 $A\boldsymbol{x}=\boldsymbol{b}$ 凑广

又逆 $A^TA\boldsymbol{x}=A^T\boldsymbol{b}$ 得 $\boldsymbol{x}=(A^TA)^{-1}A^T\boldsymbol{b}$。

**添加正则化(权重衰减)**只需要更改 trainer：

```
trainer = torch.optim.SGD([
    {"params": net[0].weight, 'weight_decay': 3},
    {"params": net[0].bias}], lr=0.01)
```

注意是给 weight 正则化，bias 不需要正则化。(而且如果之前指定 bias=False 了，这里也不应该出现 bias)

**暂退法的实现：**

```
dropout1, dropout2 = 0.2, 0.5
net = nn.Sequential(nn.Flatten(),
                    nn.Linear(784, 256),
                    nn.ReLU(),
                    # 在第一个全连接层之后添加一个 dropout 层
                    nn.Dropout(dropout1),
                    nn.Linear(256, 256),
                    nn.ReLU(),
                    # 在第二个全连接层之后添加一个 dropout 层
                    nn.Dropout(dropout2),
                    nn.Linear(256, 10))
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight)
net.apply(init_weights)
```

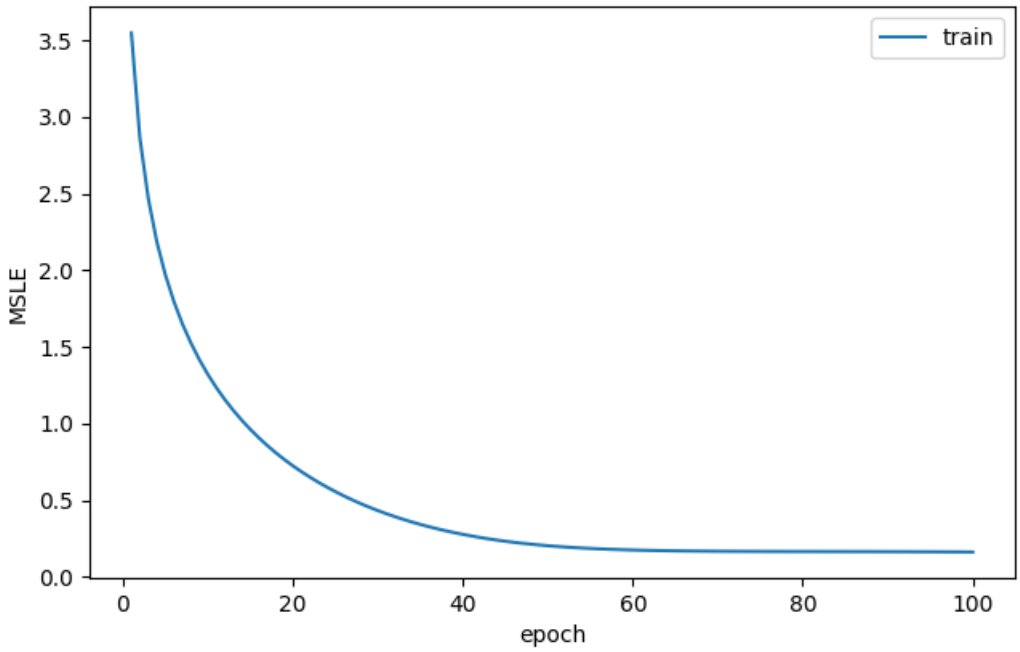
可以将暂退法应用于每个隐藏层的输出（**在激活函数之后**），并且可以为每一层分别设置暂退概率。常见的技巧是**在靠近输入层的地方设置较低的暂退概率**。

**kaggle-房价**

```
import pandas as pd
import numpy as np
import torch
import xm_func
import matplotlib.pylab as plt
from torch import nn

# 1.读入
train_data = pd.read_csv(r"..\data\kaggle_house_pred_train.csv")  # shape(1460, 81)
test_data = pd.read_csv(r"..\data\kaggle_house_pred_test.csv")  # shape(1459, 80)
all_features = pd.concat((train_data.iloc[:, 1:-1], test_data.iloc[:, 1:]))  # shape(2919, 79)，去除了 ID 和 train 里的 SalePrice

# 2.数据处理
# .dtypes 返回一个 Series(索引是列名，值是对应列的数据类型)
numeric_features = all_features.dtypes[all_features.dtypes != 'object'].index
# 对每一列 x 进行标准化处理
all_features[numeric_features] = all_features[numeric_features].apply(lambda x: (x - x.mean()) / (x.std()))
# # 若无法获得测试数据，则可根据训练数据计算均值和标准差
# train_mean, train_std = all_features[numeric_features].mean(), all_features[numeric_features].std()
# all_features = (all_features - train_mean) / train_std
# 在标准化数据之后，所有均值消失，因此我们可以将缺失值设置为 0
all_features[numeric_features] = all_features[numeric_features].fillna(0)
all_features = pd.get_dummies(all_features, dummy_na=True)  # 将 object 的数据转为指示性变量，此时 shape 为(2919, 330)
all_features = all_features.astype(np.float32)  # 将 True,False 变成 float。也可以 all_features = all_features * 1
# 转为 tensor
n_train = train_data.shape[0]
train_features = torch.tensor(all_features[:n_train].values, dtype=torch.float32)
test_features = torch.tensor(all_features[n_train:].values, dtype=torch.float32)
train_labels = torch.tensor(train_data['SalePrice'].values.reshape(-1, 1), dtype=torch.float32)
```

```python
# 3.训练
loss = nn.MSELoss()
in_features = train_features.shape[1]

def log_rmse(net, features, labels):
    # 为了在取对数时进一步稳定该值，将数值限定在 1~inf 之间，避免小于 1 时出现负数
    clipped_preds = torch.clamp(net(features), 1, float('inf'))
    rmse = torch.sqrt(loss(torch.log(clipped_preds), torch.log(labels)))
    return rmse.item()

def train(net, train_features, train_labels, test_features, test_labels, num_epochs, learning_rate,
weight_decay, batch_size):
    train_ls, test_ls = [], []
    train_iter = xm_func.load_array((train_features, train_labels), batch_size)
    # 这里使用的是 Adam 优化算法
    optimizer = torch.optim.Adam(net.parameters(), lr=learning_rate, weight_decay=weight_decay)
    for epoch in range(num_epochs):
        for X, y in train_iter:
            optimizer.zero_grad()
            l = loss(net(X), y)
            l.backward()
            optimizer.step()
        train_ls.append(log_rmse(net, train_features, train_labels))
        if test_labels is not None:
            test_ls.append(log_rmse(net, test_features, test_labels))
    return train_ls, test_ls

def get_k_fold_data(k, i, X, y):
    assert k > 1
    fold_size = X.shape[0] // k
    X_train, y_train,  X_valid, y_valid = None, None, None, None
    for j in range(k):
        idx = slice(j * fold_size, (j + 1) * fold_size)  # [j*fold_size, (j+1)*fold_size)
        X_part, y_part = X[idx, :], y[idx]
        if j == i:
            X_valid, y_valid = X_part, y_part
        elif X_train is None:
            X_train, y_train = X_part, y_part
        else:
            X_train = torch.cat([X_train, X_part], 0)  # 按行拼接
            y_train = torch.cat([y_train, y_part], 0)
    return X_train, y_train, X_valid, y_valid

def k_fold(k, X_train, y_train, num_epochs, learning_rate, weight_decay, batch_size):
    """K 折交叉验证，有助于模型选择和超参数调整。如果训练误差非常低但 K 折交叉验证的误差要高得多，表明模型过拟
合了"""
    train_l_sum, valid_l_sum = 0, 0
    for i in range(k):
        data = get_k_fold_data(k, i, X_train, y_train)
        net = nn.Sequential(nn.Linear(in_features, 1))
        train_ls, valid_ls = train(net, *data, num_epochs, learning_rate, weight_decay, batch_size)
        train_l_sum += train_ls[-1]
        valid_l_sum += valid_ls[-1]
        if i == 0:
            plt.plot(list(range(1, num_epochs + 1)), train_ls, label='train')
            plt.plot(list(range(1, num_epochs + 1)), valid_ls, label='valid')
            plt.gca().set_xlabel('epoch')  # gca:Get Current Axes 获取当前轴对象
            plt.gca().set_ylabel('MSLE')
            plt.ylim([0, 0.5])
            plt.legend()
            plt.show()
            plt.close()
        print(f'折{i + 1}，训练 MSLE{float(train_ls[-1]):f}, 'f'验证 MSLE{float(valid_ls[-1]):f}')
    return train_l_sum / k, valid_l_sum / k

k, num_epochs, lr, weight_decay, batch_size = 5, 100, 5, 0, 64
# train_l, valid_l = k_fold(k, train_features, train_labels, num_epochs, lr, weight_decay,
batch_size)
# print(f'{k}-折验证: 平均训练 MSLE: {float(train_l):f}, 'f'平均验证 MSLE: {float(valid_l):f}')

def train_and_pred(train_features, test_features, train_labels, test_data, num_epochs, lr,
weight_decay, batch_size):
    """应该选择什么样的超参数后，再使用所有数据对其进行训练"""
    net = nn.Sequential(nn.Linear(in_features, 1))
    train_ls, _ = train(net, train_features, train_labels, None, None, num_epochs, lr, weight_decay,
batch_size)
    plt.plot(list(range(1, num_epochs + 1)), train_ls, label='train')
    plt.gca().set_xlabel('epoch')
```

```
    plt.gca().set_ylabel('MSLE')
    plt.legend()
    plt.show()
    plt.close()
    print(f'训练 MSLE: {float(train_ls[-1]):f}')  # 某次是 0.162501
    # 将网络应用于测试集。
    preds = net(test_features).detach().numpy()  # 大小(1459, 1)
    # 将其重新格式化以导出到 Kaggle
    test_data['SalePrice'] = pd.Series(preds.reshape(1, -1)[0])
    submission = pd.concat([test_data['Id'], test_data['SalePrice']], axis=1)
    submission.to_csv('submission.csv', index=False)

train_and_pred(train_features, test_features, train_labels, test_data, num_epochs, lr, weight_decay,
batch_size)
```




另外，根据别人的经验，可以选用：

```
net = nn.Sequential(nn.Flatten(),
                    nn.Linear(in_features, 1024),
                    nn.ReLU(),
                    nn.Linear(1024, 1))
k, num_epochs, lr, weight_decay, batch_size = 5, 100, 0.1, 35, 256
```

## 访问参数

```
# 一般的 Sequential 的参数访问
net = nn.Sequential(nn.Linear(4, 2), nn.ReLU(), nn.Linear(2, 1))
X = torch.rand(size=(3, 4))
print(X)
print(net(X))
print(net.state_dict())  # 参数访问
print(net[2].bias)  # 提取偏置，提取后返回的是一个参数类实例
print(net[2].bias.data)  # 进一步访问该参数的值
print(net[2].bias.grad)  # 梯度值（因为这里并没有反向传播，所以是 None）
print(*[(name, param.shape) for name, param in net[0].named_parameters()])  # *将整个列表解包成单独的元素(这里是 name,shape 构成的元组)
print([(name, param.shape) for name, param in net.named_parameters()])print(net.state_dict()['2.bias'].data)  # 另一种访问参数的方法
net[2].bias.data = torch.tensor([7.27])  # 修改参数时 data set to a tensor that requires gradients must be floating point or complex dtype

# 嵌套 net 的参数访问
def block_small():
    return nn.Sequential(nn.Linear(4, 8), nn.ReLU(), nn.Linear(8, 4), nn.ReLU())
def block_main():
    net = nn.Sequential()
    for i in range(4):
        # 在这里嵌套
        net.add_module(f'block {i}', block_small())
    return net
rgnet = nn.Sequential(block_main(), nn.Linear(4, 1))
rgnet(X)
print(rgnet)
print(rgnet[0][1][0].bias.data)  # 获取 Sequential 里的第 1 个主块(block_main)的第 2 个子块(block 1)的第 1 层(Linear(4, 8))的偏置项
```

从 net.state_dict()到[(name, param.shape) ……]的输出：

```
OrderedDict([('0.weight', tensor([[-0.3908,  0.1140, -0.1552,  0.3524],
        [-0.2434,  0.4896, -0.4983, -0.1692]])), ('0.bias', tensor([-0.1386,  0.0155])), ('2.weight', tensor([[-0.1979, -0.5240]])), ('2.bias', tensor([-0.5439]))])
Parameter containing:
tensor([-0.5439], requires_grad=True)
tensor([-0.5439])
None
('weight', torch.Size([2, 4])) ('bias', torch.Size([2]))
[('0.weight', torch.Size([2, 4])), ('0.bias', torch.Size([2])), ('2.weight', torch.Size([1, 2])), ('2.bias', torch.Size([1]))]
```

## 参数初始化

初始化权重矩阵还可以改为：torch.nn.init.xavier_normal_(tensor, gain=1)等。

xavier 的思想是让 $\frac{1}{2}(n_{in}+n_{out})\sigma^2=1$，即 $\sigma=\sqrt{\frac{2}{n_{in}+n_{out}}}$，也就是从 $\mu=0,\sigma^2=\frac{2}{n_{in}+n_{out}}$ 的高斯分布中采样权重。因为均匀分布 $U(-a,a)$ 的方差是 $a^2/3$，所以初始化值域是 $U\left(-\sqrt{\frac{6}{n_{in}+n_{out}}},\sqrt{\frac{6}{n_{in}+n_{out}}}\right)$。这里 gain 参数表示 $\text{std}=\text{gain}\times\sqrt{\frac{2}{n_{in}+n_{out}}}$。

```
# 内置初始化
net = nn.Sequential(nn.Linear(4, 8), nn.Linear(8, 6), nn.Linear(6, 2))
def init_xavier(m):
    if type(m) == nn.Linear:
        nn.init.xavier_uniform_(m.weight)
def init_normal(m):
    if type(m) == nn.Linear:
        nn.init.normal_(m.weight, mean=0, std=0.01)
        nn.init.zeros_(m.bias)
def init_constant(m, c):
```

```
    if type(m) == nn.Linear:
        nn.init.constant_(m.weight, c)
        nn.init.constant_(m.bias, 0)
net[0].apply(init_xavier)
net[1].apply(init_normal)  # 也可以 net[1].weight.data.normal_(0, 0.01)
net[1].bias.data.fill_(0)
net[2].apply(lambda m: init_constant(m, 0.5))  # 该匿名函数接受参数 m，将 c 设置为 0.5

# 自定义初始化
def my_init(m):
    if type(m) == nn.Linear:
        print("Init", *[(name, param.shape) for name, param in
m.named_parameters()][0])
        nn.init.uniform_(m.weight, -10, 10)
        m.weight.data *= m.weight.data.abs() >= 5
net.apply(my_init)
print(net[0].weight)
```

**参数绑定**

```
shared = nn.Linear(8, 8)  # 给共享层一个名称，以便可以引用它的参数
net = nn.Sequential(nn.Linear(4, 8), nn.ReLU(),
                    shared, nn.ReLU(),
                    shared, nn.ReLU(),
                    nn.Linear(8, 1))
net(X)
print(net[2].weight.data[0] == net[4].weight.data[0])  # 输出都是 True。
net[2].weight.data[0, 0] = 100
print(net[2].weight.data[0] == net[4].weight.data[0])  # 输出都是 True。它们实际上是同
一个对象，而不只是有相同的值
```

参数绑定时，在反向传播期间 net[2]和 net[4]的梯度会加在一起(因为传播的梯度还没有清零)。

共享参数可以减少计算成本(共享的参数只需要计算一次梯度)。还可以用于处理变长输入序列(如 nlp)。

**延后初始化**

```
net = nn.Sequential(nn.LazyLinear(256), nn.ReLU(), nn.LazyLinear(10))
print(net[0].weight)  # 尚未初始化，输出的是<UninitializedParameter>
print(net)  # LazyLinear(in_features=0, out_features=256, bias=True) 和
LazyLinear(in_features=0, out_features=10, bias=True)
X = torch.rand(2, 20)
net(X)
print(net)  # Linear(in_features=20, out_features=256, bias=True) 和
Linear(in_features=256, out_features=10, bias=True)
print(net(X))  # 能够输出了
```

延后初始化 defers initialization：直到数据第一次通过模型传递时，框架才会动态地推断出每个层的大小。

现在使用会提示：UserWarning: Lazy modules are a new feature under heavy development so changes to the API or functionality can happen at any moment.

## 卷积核的使用与训练(图像中的目标边缘检测)

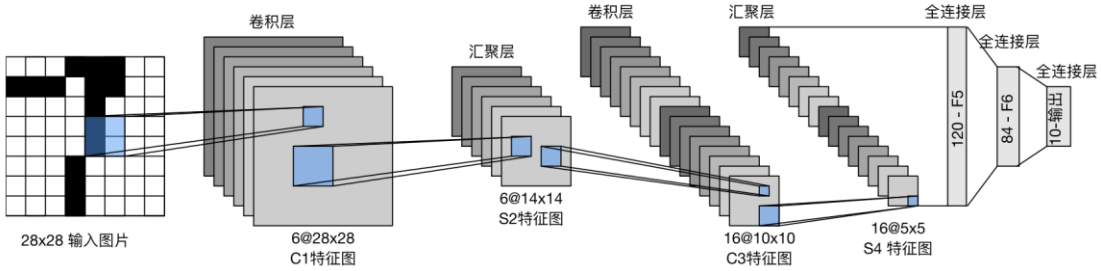
```
def corr2d(X, K):
    """计算二维互相关运算 cross-correlation"""
    h, w = K.shape
    Y = torch.zeros((X.shape[0] - h + 1, X.shape[1] - w + 1))
    for i in range(Y.shape[0]):
        for j in range(Y.shape[1]):
            Y[i, j] = (X[i:i + h, j:j + w] * K).sum()
    return Y

X = torch.ones((6, 8))
X[:, 2:6] = 0  # 3~6 列是 1，其他是 0
K = torch.tensor([[1.0, -1.0]])  # 当进行互相关运算时，如果水平相邻的两元素相同，则输出为零，否则输出为非零（这个卷积核 K 只可以检测垂直边缘，无法检测水平边缘）
Y = corr2d(X, K)  # Y 中的 1 代表从白色到黑色的边缘，-1 代表从黑色到白色的边缘，其他情况的输出为 0

# 1 个输入通道,1 个输出通道，形状为(1, 2)的卷积核
conv2d = nn.Conv2d(1, 1, kernel_size=(1, 2), bias=False)
# 这个二维卷积层使用四维输入和输出格式(batch_size 批量大小，channels 通道数，height, width)
X = X.reshape((1, 1, 6, 8))
Y = Y.reshape((1, 1, 6, 7))
lr = 0.01  # 学习率
for i in range(50):
    Y_hat = conv2d(X)
    l = (Y_hat - Y) ** 2
    conv2d.zero_grad()
    l.sum().backward()
    conv2d.weight.data[:] -= lr * conv2d.weight.grad  # 迭代卷积核
    print(f'epoch {i+1}, loss {l.sum():.6f}')
print(conv2d.weight.data.reshape((1, 2)))  # 将[1, 1, 1, 2]大小的 weight 变成[1, 2]
```

最后得到的 weight 是 tensor([[ 0.9980, -0.9980]])与垂直边缘检测核 K=[1,-1]很接近了。

## 各种 CNN 网络

**LeNet** (代码实现时对网络有修改，没有全连接层，sigmoid 改为了 relu)

```
net = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.ReLU(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.ReLU(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120), nn.ReLU(),
    nn.Linear(120, 84), nn.ReLU(),
    nn.Linear(84, 10))
def init_weights(m):
    if type(m) == nn.Linear or type(m) == nn.Conv2d:
        nn.init.xavier_uniform_(m.weight)
net.apply(init_weights)
```



**AlexNet** (对部分数据更改以适应本数据集，更改的标注在了注释里)

```
net = nn.Sequential(
    nn.Conv2d(1, 96, kernel_size=3, stride=2, padding=1), nn.ReLU(),  # 11->3, 4->2
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Conv2d(96, 256, kernel_size=5, padding=2), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=1),  # 2->1
    nn.Conv2d(256, 384, kernel_size=3, padding=1), nn.ReLU(),
    nn.Conv2d(384, 384, kernel_size=3, padding=1), nn.ReLU(),
    nn.Conv2d(384, 256, kernel_size=3, padding=1), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Flatten(),
    nn.Linear(256, 4096), nn.ReLU(),  # 6400->256
    nn.Dropout(p=0.5),
    nn.Linear(4096, 4096), nn.ReLU(),
    nn.Dropout(p=0.5),
    nn.Linear(4096, 10))  # 1000->10
```

该 net 需要的 GPU 空间太大了，我运行不出来...

**VGG** (请注意这里的输入图像大小改为 224*224 了，下同)

```
batch_size, channels, height, width = 64, 1, 224, 224
X = torch.randn(batch_size, channels, height, width)

def vgg_block(num_convs, in_channels, out_channels):
    """卷积层的数量、输入通道的数量、输出通道的数量
        3×3 卷积核、填充为 1 的卷积层->保持高度和宽度，
        2×2 汇聚窗口、步幅为 2 的最大汇聚层->每个块后的分辨率减半"""
    layers = []
    for _ in range(num_convs):
        layers.append(nn.Conv2d(in_channels, out_channels, kernel_size=3,
padding=1))
        layers.append(nn.ReLU())
        in_channels = out_channels
    layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
    return nn.Sequential(*layers)
def vgg(conv_arch):
    """conv_arch 指定了每个 VGG 块里卷积层个数和输出通道数"""
    conv_blks = []
    in_channels = 1
    # 卷积层部分
    for (num_convs, out_channels) in conv_arch:
        conv_blks.append(vgg_block(num_convs, in_channels, out_channels))
        in_channels = out_channels
    return nn.Sequential(
        *conv_blks, nn.Flatten(),
        # 全连接层部分
        nn.Linear(out_channels * 7 * 7, 4096), nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(4096, 4096), nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(4096, 10))

conv_arch = ((1, 64), (1, 128), (2, 256), (2, 512), (2, 512))
net = vgg(conv_arch)
```

其中 7 是根据输入得出来的，可以将 nn.AdaptiveMaxPool2d(output_size=(7, 7))应用于最后一个卷积层的输出，然后将其 Flatten 作为全连接层的输入。这样就不需要手动计算特征图大小，而是根据输出的固定大小来自适应地处理输入。

**NiN**

```
def nin_block(in_channels, out_channels, kernel_size, strides, padding):
    return nn.Sequential(
        nn.Conv2d(in_channels, out_channels, kernel_size, strides, padding),
        nn.ReLU(),
```

```
        nn.Conv2d(out_channels, out_channels, kernel_size=1), nn.ReLU(),
        nn.Conv2d(out_channels, out_channels, kernel_size=1), nn.ReLU())


net = nn.Sequential(
    nin_block(1, 96, kernel_size=11, strides=4, padding=0),
    nn.MaxPool2d(3, stride=2),
    nin_block(96, 256, kernel_size=5, strides=1, padding=2),
    nn.MaxPool2d(3, stride=2),
    nin_block(256, 384, kernel_size=3, strides=1, padding=1),
    nn.MaxPool2d(3, stride=2),
    nn.Dropout(0.5),
    # 标签类别数是 10
    nin_block(384, 10, kernel_size=3, strides=1, padding=1),
    nn.AdaptiveAvgPool2d((1, 1)),
    # 将四维的输出(N,C,1,1)转成二维的输出(N,C)，其形状为(N=批量大小,C=10)
    nn.Flatten())
```

将空间维度中的每个像素视为单个样本，将通道维度视为不同特征。也就是在每个像素位置(对每个高度和宽度)应用一个全连接层，也就是用 1×1 卷积层将权重连接到每个空间位置。

**GoogLeNet 的 Inception 块**

```
class Inception(nn.Module):
    def __init__(self, in_channels, c1, c2, c3, c4, **kwargs):
        """c1--c4 是每条路径的输出通道数"""
        super(Inception, self).__init__(**kwargs)
        # 线路 1，单 1x1 卷积层
        self.p1_1 = nn.Conv2d(in_channels, c1, kernel_size=1)
        # 线路 2，1x1 卷积层后接 3x3 卷积层
        self.p2_1 = nn.Conv2d(in_channels, c2[0], kernel_size=1)
        self.p2_2 = nn.Conv2d(c2[0], c2[1], kernel_size=3, padding=1)
        # 线路 3，1x1 卷积层后接 5x5 卷积层
        self.p3_1 = nn.Conv2d(in_channels, c3[0], kernel_size=1)
        self.p3_2 = nn.Conv2d(c3[0], c3[1], kernel_size=5, padding=2)
        # 线路 4，3x3 最大汇聚层后接 1x1 卷积层
        self.p4_1 = nn.MaxPool2d(kernel_size=3, stride=1, padding=1)
        self.p4_2 = nn.Conv2d(in_channels, c4, kernel_size=1)

    def forward(self, x):
        p1 = F.relu(self.p1_1(x))
        p2 = F.relu(self.p2_2(F.relu(self.p2_1(x))))
        p3 = F.relu(self.p3_2(F.relu(self.p3_1(x))))
        p4 = F.relu(self.p4_2(self.p4_1(x)))
        # 在通道维度上连结输出
        return torch.cat((p1, p2, p3, p4), dim=1)

b1 = nn.Sequential(nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3),
                   nn.ReLU(),
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
b2 = nn.Sequential(nn.Conv2d(64, 64, kernel_size=1),
                   nn.ReLU(),
                   nn.Conv2d(64, 192, kernel_size=3, padding=1),
                   nn.ReLU(),
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
b3 = nn.Sequential(Inception(192, 64, (96, 128), (16, 32), 32),
                   Inception(256, 128, (128, 192), (32, 96), 64),
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
b4 = nn.Sequential(Inception(480, 192, (96, 208), (16, 48), 64),
                   Inception(512, 160, (112, 224), (24, 64), 64),
                   Inception(512, 128, (128, 256), (24, 64), 64),
                   Inception(512, 112, (144, 288), (32, 64), 64),
                   Inception(528, 256, (160, 320), (32, 128), 128),
```

```
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
b5 = nn.Sequential(Inception(832, 256, (160, 320), (32, 128), 128),
                   Inception(832, 384, (192, 384), (48, 128), 128),
                   nn.AdaptiveAvgPool2d((1, 1)),
                   nn.Flatten())
net = nn.Sequential(b1, b2, b3, b4, b5, nn.Linear(1024, 10))
```

Inception 块通过填充使得输出的 shape 能够在 dim=1 上联结，也就是 shape 均满足[N, *, H, W]，仅在 shape[1]上不同。

GoogLeNet 就像用不同大小的滤波器识别不同范围的图像细节，还能为不同的滤波器分配不同数量的参数，因此效果好。

三种块的对比如下：

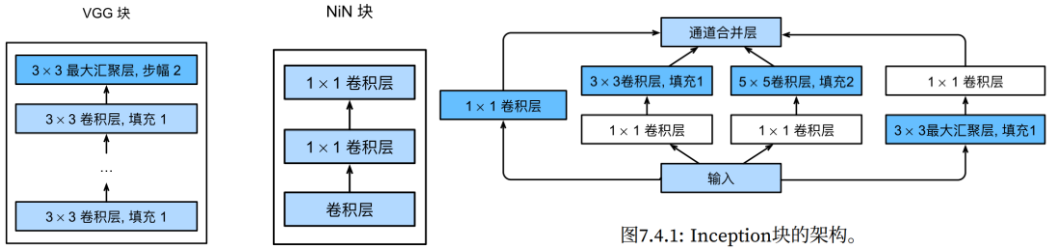


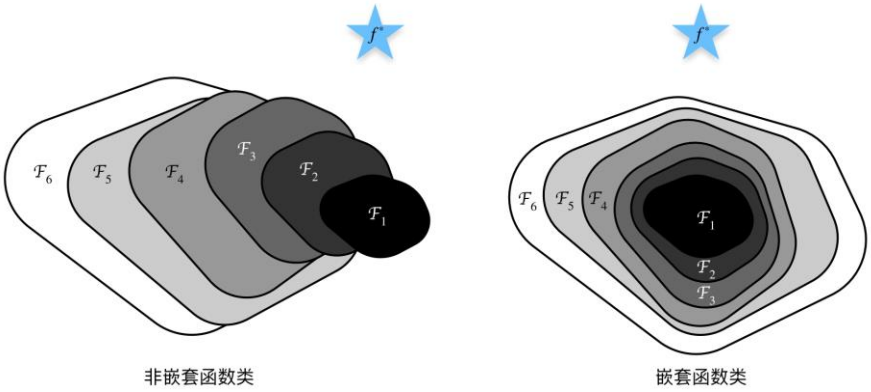

图7.4.1: Inception块的架构。

**ResNet 残差网络**

```
class Residual(nn.Module):
    def __init__(self, input_channels, num_channels, use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.Conv2d(input_channels, num_channels, kernel_size=3,
padding=1, stride=strides)
        self.conv2 = nn.Conv2d(num_channels, num_channels, kernel_size=3, padding=1)
        if use_1x1conv:
            self.conv3 = nn.Conv2d(input_channels, num_channels, kernel_size=1,
stride=strides)
        else:
            self.conv3 = None
        self.bn1 = nn.BatchNorm2d(num_channels)
        self.bn2 = nn.BatchNorm2d(num_channels)

    def forward(self, X):
        Y = F.relu(self.bn1(self.conv1(X)))
        Y = self.bn2(self.conv2(Y))
        if self.conv3:
            X = self.conv3(X)
        Y += X
        return F.relu(Y)

def resnet_block(input_channels, num_channels, num_residuals,
                 first_block=False):
    blk = []
    for i in range(num_residuals):
        if i == 0 and not first_block:
            blk.append(Residual(input_channels, num_channels,
                                use_1x1conv=True, strides=2))
    else:
        blk.append(Residual(num_channels, num_channels))
    return blk

b1 = nn.Sequential(nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3),
                   nn.BatchNorm2d(64), nn.ReLU(),
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
b2 = nn.Sequential(*resnet_block(64, 64, 2, first_block=True))
b3 = nn.Sequential(*resnet_block(64, 128, 2))
b4 = nn.Sequential(*resnet_block(128, 256, 2))
b5 = nn.Sequential(*resnet_block(256, 512, 2))
```

```
net = nn.Sequential(b1, b2, b3, b4, b5,
                    nn.AdaptiveAvgPool2d((1, 1)),
                    nn.Flatten(), nn.Linear(512, 10))
```

当较复杂的函数类包含较小的函数类时，我们才能确保提高它们的性能，如下图，非嵌套函数不能保证越来越接近目标函数 $f^*$(因为显然 $F_6$ 是不如 $F_3$ 的)。也就是当我们预测更强大的架构 $F'$ 时，应该保证原有的函数 $F_i \in F'$。




非嵌套函数类　　嵌套函数类

什么是残差，如下左图：我们定义当前块的最终输出是 $f(x)=L(x)+x$，也就是当前层的输出与上一块的输出的并集。当 $L(x)\to 0$ 时，$f(x)=x$，称为恒等映射。

当 use_1x1conv=True 时，会添加通过 1×1 卷积调整通道和分辨率，如下右图：

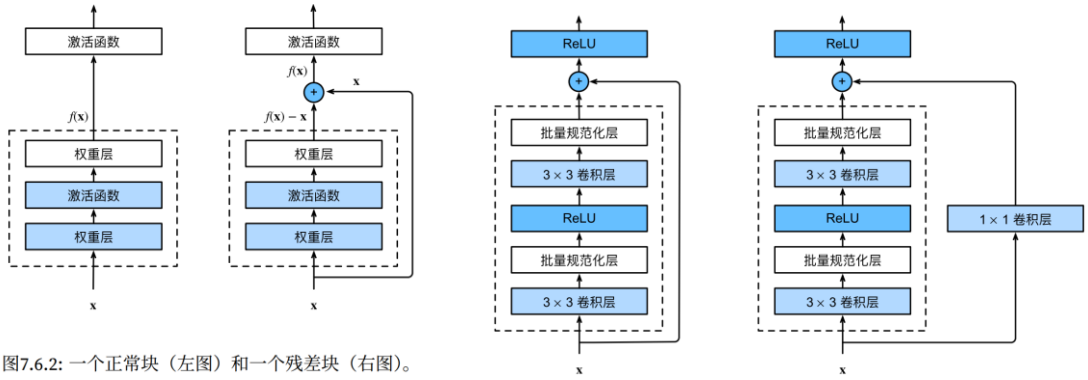


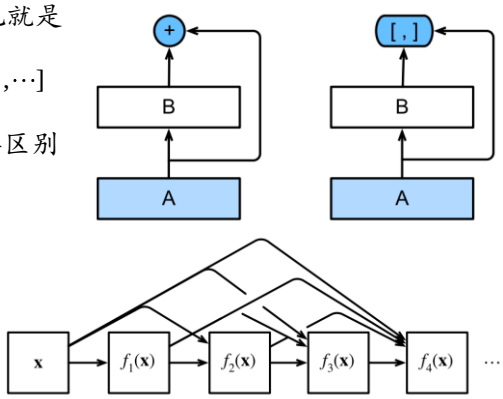

图7.6.2: 一个正常块（左图）和一个残差块（右图）。

## DenseNet 稠密连接网络

```
def conv_block(input_channels, num_channels):
    return nn.Sequential(
        nn.BatchNorm2d(input_channels), nn.ReLU(),
        nn.Conv2d(input_channels, num_channels, kernel_size=3, padding=1))
# 稠密块体
class DenseBlock(nn.Module):
    def __init__(self, num_convs, input_channels, num_channels):
        super(DenseBlock, self).__init__()
        layer = []
        for i in range(num_convs):
            layer.append(conv_block(num_channels * i + input_channels, num_channels))
        self.net = nn.Sequential(*layer)
    def forward(self, X):
        for blk in self.net:
            Y = blk(X)
            # 连接通道维度上每个块的输入和输出
            X = torch.cat((X, Y), dim=1)
        return X
# 过渡层
def transition_block(input_channels, num_channels):
    return nn.Sequential(
        nn.BatchNorm2d(input_channels), nn.ReLU(),
```

```
        nn.Conv2d(input_channels, num_channels, kernel_size=1),
        nn.AvgPool2d(kernel_size=2, stride=2))
# DenseNet 模型
b1 = nn.Sequential(nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3),
                   nn.BatchNorm2d(64), nn.ReLU(),
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
num_channels, growth_rate = 64, 32  # num_channels 为当前的通道数
num_convs_in_dense_blocks = [4, 4, 4, 4]
blks = []
for i, num_convs in enumerate(num_convs_in_dense_blocks):
    blks.append(DenseBlock(num_convs, num_channels, growth_rate))
    # 上一个稠密块的输出通道数
    num_channels += num_convs * growth_rate
    # 在稠密块之间添加一个转换层，使通道数量减半
    if i != len(num_convs_in_dense_blocks) - 1:
        blks.append(transition_block(num_channels, num_channels // 2))
        num_channels = num_channels // 2
net = nn.Sequential(
    b1, *blks,
    nn.BatchNorm2d(num_channels), nn.ReLU(),
    nn.AdaptiveAvgPool2d((1, 1)),
    nn.Flatten(),
    nn.Linear(num_channels, 10))
```

相对于残差网络 $f(x)=L(x)+x$，这里将+变成×，也就是使用连接：$x=[x, f_1(x), f_2\left(x, f_1(x)\right), f_3\left(x, f_1(x), f_2\left(x, f_1(x)\right)\right), \cdots]$




如右图，ResNet(左)与 DenseNet(右)在跨层连接上的主要区别是使用相加和使用连结。

于是定义：稠密块 dense block(如右图)和过渡层 transition layer。前者定义如何连接输入和输出，而后者则控制通道数量，使其不会太复杂。

## 为什么需要 RNN

这里以模拟的数据集 y=sinx+噪声为例：

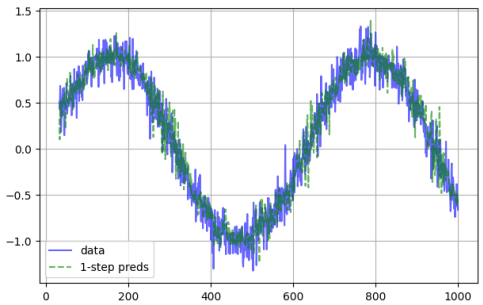
```
from easier_nn.classic_dataset import VirtualDataset as VD
vd = VD(end=1000)
vd.sinx(noise_sigma=0.15, show_plt=False)

def get_train_iter(train_class, tau=4, batch_size=16, n_train=600):
    features = torch.zeros((train_class.num_points - tau, tau))
    for i in range(tau):
        features[:, i] = train_class.y[i: train_class.num_points - tau + i]
    labels = train_class.y[tau:].reshape((-1, 1))
    train_iter = load_array((features[:n_train], labels[:n_train]), batch_size,
if_shuffle=True)
    return train_iter, features, labels

tau = 32   # 书上是 tau=4，但预测的结果不够好
n_train = 600
train_iter, features, labels = get_train_iter(vd, tau=tau, n_train=n_train)  # 大小
分别是[968, 32]), [968, 1]
```

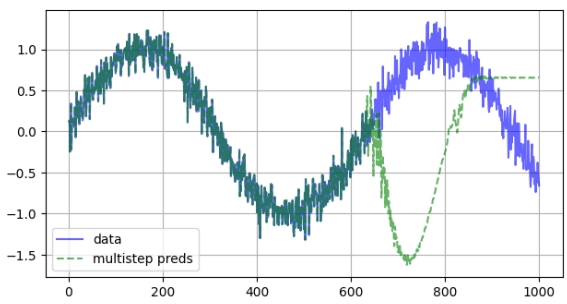
训练网络（网络是输入是 tau 个数，输出是预测的一个数）：

```
net = nn.Sequential(nn.Linear(tau, 20), nn.ReLU(), nn.Linear(20, 8), nn.ReLU(),
nn.Linear(8, 1))
def init_weights(m):
    if type(m) == nn.Linear:
        nn.init.xavier_uniform_(m.weight)
net.apply(init_weights)
loss = nn.MSELoss(reduction='none')
optimizer = torch.optim.Adam(net.parameters(), lr=0.001)
net = train_net(features, labels, data_iter=train_iter, net=net, loss=loss,
optimizer=optimizer, num_epochs=300)
# 保存 net 模型
torch.save(net, '../model/test/rnn_predict_net_tau=32.pth')
# 加载 net 模型
net = torch.load('../model/test/rnn_predict_net_tau=32.pth')
```

单步预测和多步预测：

```
onestep_preds = net(features)
plot_xys(x=vd.x.detach().numpy()[tau:], y_list=[vd.y.detach().numpy()[tau:],
onestep_preds.detach().numpy()],
         labels=['data', '1-step preds'], alpha=0.6)
multistep_preds = torch.zeros(vd.num_points)
multistep_preds[: n_train + tau] = vd.y[: n_train + tau]
for i in range(n_train + tau, vd.num_points):
    multistep_preds[i] = net(multistep_preds[i - tau:i].reshape((1, -1)))
plot_xys(x=vd.x.detach().numpy(), y_list=[vd.y.detach().numpy(),
multistep_preds.detach().numpy()],
         labels=['data', 'multistep preds'], alpha=0.6)
```



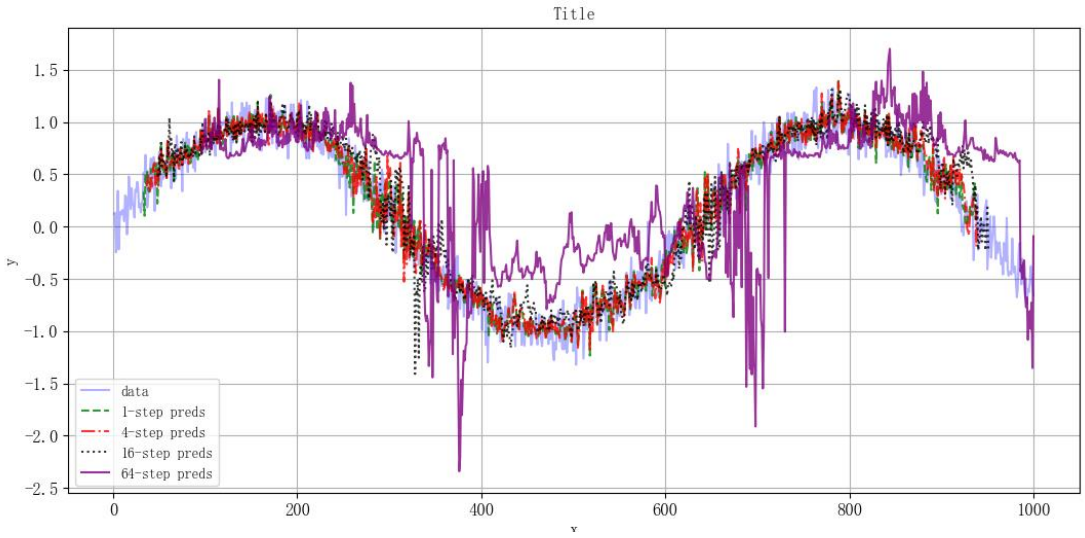




对于直到时间步 t 的观测序列，其在时间步 t+k 的预测输出是 k 步预测。

单步预测比多步更好，因为多步预测在 600+tau 之后的预测依赖于之前的预测，一旦出现误差，误差就会被累积。

对比步数：

```
max_steps = 64
features = torch.zeros((vd.num_points - tau - max_steps + 1, tau + max_steps))
# 列 i（i<tau）是来自 x 的观测，其时间步从（i）到（i+T-tau-max_steps+1）
for i in range(tau):
    features[:, i] = vd.y[i: i + vd.num_points - tau - max_steps + 1]
# 列 i（i>=tau）是来自（i-tau+1）步的预测，其时间步从（i）到（i+T-tau-max_steps+1）
for i in range(tau, tau + max_steps):
    features[:, i] = net(features[:, i - tau:i]).reshape(-1)

steps = (1, 4, 16, 64)
colors = ['blue', 'green', 'red', 'black', 'purple', 'pink', 'orange', 'cyan']
linstyles = ['-', '--', '-.', ':', 'solid', 'dashed', 'dashdot', 'dotted']
# y_list 的列表的第 1 个元素是 vd.y，后面的 2~i+1 个元素是 features 的第 i 列
y_list = [vd.y.detach().numpy()] + [features[:, tau + i - 1].detach().numpy() for i
in steps]
fig, ax = plt.subplots(figsize=(10, 6))
ax = plot_xy(x=vd.x.detach().numpy(), y=y_list[0], label='data', use_ax=True,
ax=ax, show_plt=False, alpha=0.3)
for i, n in enumerate(steps):
    ax = plot_xy(x=vd.x.detach().numpy()[tau + n - 1: vd.num_points - max_steps +
n], y=y_list[i+1], alpha=0.8,
                 label=f'{n}-step preds', use_ax=True, ax=ax, show_plt=False,
                 color=colors[i+1], linestyle=linstyles[i+1])
```




步数越大预测越不准确，因为误差的累积更大。